

การปรับปรุงการรู้จำเสียงพูดอัตโนมัติโดยการแบ่งส่วนเสียงพูด
IMPROVING AUTOMATIC SPEECH RECOGNITION BY
SPEECH SEGMENTATION

กัมปนาท ชินราช
ธนภัทร ประภัทรพรกุล
วิภาดา วรสัจจา
ศรัณย์ธร พรชัยวัฒนากร

ปริญญานิพนธ์นี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรปริญญาวิศวกรรมศาสตรบัณฑิต
สาขาวิชาวิศวกรรมคอมพิวเตอร์ คณะวิศวกรรมศาสตร์
สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง
ปีการศึกษา 2567

ปริญญานิพนธ์ปีการศึกษา 2567

สาขาวิชาวิศวกรรมคอมพิวเตอร์

คณะวิศวกรรมศาสตร์ สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง

เรื่อง การปรับปรุงการรู้จำเสียงพูดอัตโนมัติโดยการแบ่งส่วนเสียงพูด

IMPROVING AUTOMATIC SPEECH RECOGNITION BY SPEECH SEGMENTATION

ผู้จัดทำ

- | | |
|-----------------------------|-----------------------|
| 1. นายกัมปนาท ชินราช | รหัสนักศึกษา 64010042 |
| 2. นายธนภัทร ประภัทรพรกุล | รหัสนักศึกษา 64010327 |
| 3. นางสาววิภาดา วรสัจจา | รหัสนักศึกษา 64010808 |
| 4. นายศรัณย์ธร พรชัยวัฒนากร | รหัสนักศึกษา 64010827 |

อาจารย์ที่ปรึกษา

(รศ. ดร. เกียรติกุล เกียรนัยธนกิจ)

การปรับปรุงการรู้จำเสียงพูดอัตโนมัติโดยการแบ่งส่วนเสียงพูด

นายกัมปนาท	ชินราช	64010042
นายธนภัทร	ประภัทรพรกุล	64010327
นางสาววิภาดา	วรศักดิ์	64010808
นายศรัณย์ธร	พรชัยวัฒนากร	64010827
รศ. ดร. เกียรติกุล	เจียรนัยธนกิจ	อาจารย์ที่ปรึกษา
ปีการศึกษา 2567		

บทคัดย่อ

กระบวนการถอดเสียงพูดให้เป็นข้อความนั้นยังคงเป็นความท้าทาย เนื่องจากข้อจำกัดของโมเดลการรู้จำเสียงพูดอัตโนมัติ (ASR) เช่น Whisper และ Wav2Vec2 ซึ่งมักจะแปลคำผิดพลาดในช่วงต้นและท้ายของไฟล์เสียง สาเหตุหลักมาจากข้อจำกัดด้านความยาวของเสียงในโมเดล ASR โดยส่วนใหญ่ อย่างเช่น Whisper ที่จำกัดความยาวไว้ที่ 30 วินาที ส่งผลให้คำที่อยู่บริเวณรอยต่อของส่วนเสียงถูกตัดแบ่งอย่างไม่เหมาะสม เนื่องจากการแบ่งส่วนมักเกิดขึ้นกลางคำหรือวลี จึงทำให้อัตราความผิดพลาดในการรู้จำคำสูงขึ้น

งานวิจัยนี้นำเสนอวิธีการใหม่ที่จะช่วยให้สามารถแบ่งส่วนไฟล์เสียงพูดได้อย่างยืดหยุ่นในความยาวที่หลากหลาย โดยยังคงอยู่ภายใต้ข้อจำกัด 30 วินาที โดยมีเป้าหมายหลักเพื่อลดอัตราความผิดพลาดในการรู้จำคำของโมเดล ASR นอกจากนี้ยังได้เปรียบเทียบเทคนิคที่นำเสนอกับวิธีการแบ่งส่วนเสียงแบบอื่น ๆ โดยทดสอบกับโมเดล ASR ที่เป็นที่รู้จัก เช่น Whisper และ Wav2Vec2 ผลการทดลองที่ดำเนินการกับคลิปวิดีโอ 10 คลิป ที่มีความยาวต่อเสียงประมาณ 30 นาที พร้อมคำบรรยายแสดงให้เห็นว่าวิธีการแบ่งส่วนเสียงที่นำเสนอสามารถปรับปรุงอัตราความผิดพลาดในการรู้จำคำได้อย่างมีนัยสำคัญในทุกโมเดล ASR และมีประสิทธิภาพเหนือกว่าวิธีการแบ่งส่วนเสียงแบบอื่น ๆ

IMPROVING AUTOMATIC SPEECH RECOGNITION BY SPEECH SEGMENTATION

Mr. Kampanat	Chinnarach	64010042
Mr. Thanaphat	Praphaatchornkul	64010327
Ms. Vipada	Varasajja	64010808
Mr. Saranthorn	Pornchaiwattanakorn	64010827
Assoc. Prof. Dr. Kietikul	Jearanaitanakij	Advisor
Academic Year 2024		

ABSTRACT

The process of transcribing speech files into text can be challenging due to the limitations of Automatic Speech Recognition (ASR) models like Whisper and Wav2Vec2. These models often misinterpret words at the start and end of an audio file. This is primarily due to the audio-length constraint of most ASR models, for instance, Whisper has a limit of 30 seconds. Consequently, words at the edges of an audio segment are incorrectly segmented as the segmentation often occurs within a single word or phrase, resulting in a higher Word Error Rate.

We introduce a novel method that allows for flexible segmentation of a speech file into varying lengths, all within the 30-second limit. Our primary objective is to decrease the Word Error Rate produced by the ASR models. We also compare our proposed technique with other speech segmentation strategies using well-known ASR models such as Whisper and Wav2Vec2. Our experiments, conducted on 10 audios with a length around 30 minutes and their corresponding subtitles, demonstrate that our speech segmentation approach significantly improves the Word Error Rate across all ASR models and outperforms other speech segmentation strategies.

กิตติกรรมประกาศ

ปริญญานิพนธ์นี้สำเร็จลุล่วงไปได้ด้วยความกรุณาจาก รศ. ดร.เกียรติกุล เจียรนัยระกิจ อาจารย์ที่ปรึกษา ที่ได้ให้คำแนะนำข้อเสนอแนะ และให้ความช่วยเหลือในการทำโครงงานนี้มาโดยตลอด ทำให้การทำงานต่าง ๆ เป็นไปอย่างราบรื่นและสำเร็จลุล่วงไปได้ด้วยดี คณะผู้จัดทำขอกราบขอบพระคุณเป็นอย่างสูงไว้ ณ ที่นี้

ขอขอบพระคุณคณาจารย์คณะวิศวกรรมศาสตร์ สาขาวิชาวิศวกรรมคอมพิวเตอร์ทุกท่าน ที่ได้อบรมสั่งสอนและให้ความรู้ต่าง ๆ แก่คณะผู้จัดทำ ตลอดช่วงการศึกษาที่ผ่านมา ส่งผลให้ปริญญานิพนธ์ฉบับนี้สำเร็จลงด้วยดี

สุดท้ายนี้คณะผู้จัดทำมีความซาบซึ้งในความกรุณาของทุกท่านที่กล่าวถึงและผู้ที่ไม่ได้เอ่ยนามในที่นี้ที่มีส่วนให้ความช่วยเหลือและสนับสนุนด้วยดีมาโดยตลอด จึงขอกราบขอบพระคุณด้วยความจริงใจ

กัมปนาท ชินราช
ธนภัทร ประภัทรพรกุล
วิภาดา วรสังจา
ศรัณย์ธณ พรชัยวัฒนากร

สารบัญ

หน้า

บทคัดย่อภาษาไทย	I
บทคัดย่อภาษาอังกฤษ	II
กิตติกรรมประกาศ.....	III
สารบัญ	IV
สารบัญตาราง	VI
สารบัญรูปภาพ	VII
บทที่ 1 บทนำ	1
1.1 ความเป็นมาของปัญหา	1
1.2 วัตถุประสงค์ของโครงการ	2
1.3 ขอบเขตของโครงการ	2
1.4 ประโยชน์ที่คาดว่าจะได้รับ.....	2
บทที่ 2 ทฤษฎีและงานวิจัยที่เกี่ยวข้อง	3
2.1 ทฤษฎี.....	3
2.2 งานวิจัยที่เกี่ยวข้อง	11
บทที่ 3 การดำเนินงาน	14
3.1 เตรียมการพัฒนาโมเดล.....	14
3.2 ขั้นตอนการพัฒนาการตัดแบ่งเสียง.....	14
3.3 การออกแบบและพัฒนาเว็บแอปพลิเคชัน	25
บทที่ 4 ผลการดำเนินงาน	45
4.1 ประสิทธิภาพของโมเดลจากค่า WER และ CER	44
4.2 ประสิทธิภาพเวลาที่ใช้ในการทำงานของแต่ละการตัดแบ่งเสียง.....	46
4.3 ตัวอย่างการเปรียบเทียบจุดตัดและข้อความจากแต่ละวิธีการตัดแบ่งเสียง	47

บทที่ 5 สรุปผลการดำเนินงาน	52
5.1 ผลลัพธ์การทดลอง.....	52
บรรณานุกรม	54
ภาคผนวก	56

สารบัญตาราง

ตาราง	หน้า
4.1 ตารางตารางเปรียบเทียบประสิทธิภาพการถอดความของโมเดล Whisper ด้วยวิธีการตัดแบ่งเสียงวิธีต่าง ๆ.....	45
4.2 ตารางเปรียบเทียบประสิทธิภาพการถอดความของโมเดล Wav2Vec2 ด้วยวิธีการตัดแบ่งเสียงวิธีต่าง ๆ.....	46
4.3 ตารางเปรียบเทียบระยะเวลาในการถอดความของโมเดล Whisper ด้วยวิธีการตัดแบ่งเสียงวิธีต่าง ๆ.....	46
4.4 ตารางเปรียบเทียบระยะเวลาในการถอดความของโมเดล Wav2Vec2 ด้วยวิธีการตัดแบ่งเสียงวิธีต่าง ๆ.....	47

สารบัญรูป

รูป	หน้า
2.1 Whisper Architecture	3
2.2 โครงสร้างของ Neural Network	4
2.3 Deep Learning	5
2.4 การคำนวณ Levenshtein distance	11
2.5 ผลลัพธ์เปรียบเทียบประสิทธิภาพระหว่าง E2E segmenter, VAD segmenter และแบบเวลาคงที่	12
2.6 ผลลัพธ์ BLEU Score ของ Smart Speech segmentation	13
3.1 ตัวอย่างการตัดแบ่งเสียงแบบ Fix-sized Audio chunk	15
3.2 ตัวอย่างการตัดแบ่งเสียงแบบ Sliding Windows ขั้นตอนที่ 1	15
3.3 ตัวอย่างการตัดแบ่งเสียงแบบ Sliding Windows ขั้นตอนที่ 2	16
3.4 ตัวอย่างการตัดแบ่งเสียงแบบ Sliding Windows ขั้นตอนที่ 3	16
3.5 ตัวอย่างการตัดแบ่งเสียงแบบ Slide and split ขั้นตอนที่ 1	17
3.6 ตัวอย่างการตัดแบ่งเสียงแบบ Slide and split ขั้นตอนที่ 2	18
3.7 ตัวอย่างการตัดแบ่งเสียงแบบ Slide and split ขั้นตอนที่ 3	18
3.8 ตัวอย่างการตัดแบ่งเสียงแบบ Slide and split ขั้นตอนที่ 4	19
3.9 ตัวอย่างการตัดแบ่งเสียงแบบ Slide and split ขั้นตอนที่ 5	19
3.10 ตัวอย่างการตัดแบ่งเสียงแบบ Slide and split ขั้นตอนที่ 6	19
3.11 ตัวอย่างการตัดแบ่งเสียงแบบ Overlapping Windows ขั้นตอนที่ 1	20
3.12 ตัวอย่างประโยค	21
3.13 ตัวอย่างการตัดแบ่งเสียงแบบ Overlapping Windows ขั้นตอนที่ 2	21
3.14 ตัวอย่างการตัดแบ่งเสียงแบบ Overlapping Windows ขั้นตอนที่ 3	21
3.15 ตัวอย่างการตัดแบ่งเสียงแบบ Overlapping Windows ขั้นตอนที่ 4	22
3.16 ตัวอย่างการตัดแบ่งเสียงแบบ Overlapping Windows ขั้นตอนที่ 5	22
3.17 ตัวอย่างการตัดแบ่งเสียงแบบ Minimum Average Amplitude	22
3.18 ตัวอย่างการตัดแบ่งเสียงแบบ Minimum Average Amplitude	23
3.19 ตัวอย่างการตัดแบ่งเสียงแบบ Minimum Average Amplitude	23
3.20 ตัวอย่างการตัดแบ่งเสียงแบบ Minimum Average Amplitude	24

3.21 ตัวอย่างการตัดแบ่งเสียงแบบ Minimum Average Amplitude	24
3.22 ตัวอย่างการตัดแบ่งเสียงแบบ Minimum Average Amplitude	26
3.23 เว็บแอปพลิเคชันหน้า Home	27
3.24 เว็บแอปพลิเคชันส่วน DropZone	27
3.25 เว็บแอปพลิเคชันส่วน File name	27
3.26 เว็บแอปพลิเคชันส่วน Waveform	28
3.27 ตัวอย่างการ Trim ของ Waveform	28
3.28 ตัวอย่างการ Crop ของ Waveform	28
3.29 ตัวอย่างการ Reset Region	29
3.30 ปุ่ม info ของเสียงที่กำลังแสดงอยู่.....	29
3.31 ตัวอย่าง Pop-up ข้อมูลของไฟล์เสียง	29
3.32 ปุ่ม Select All และ ปุ่ม Select this audio	30
3.33 ตัวอย่าง Pop-up การกด Select All	30
3.34 ตัวอย่าง Pop-up การกด Select this audio.....	31
3.35 เว็บแอปพลิเคชันหน้า List File	31
3.36 เว็บแอปพลิเคชันส่วน filter ข้อมูลไฟล์	32
3.37 เว็บแอปพลิเคชันส่วน Search ข้อมูลไฟล์	32
3.38 เว็บแอปพลิเคชันส่วนรายชื่อไฟล์.....	33
3.39 ปุ่ม Use selected audio	33
3.40 ตัวอย่าง Pop-up หลังกดปุ่ม Use select audio	34
3.41 ปุ่มสำหรับแสดง Waveform เสียงนั้น ๆ	34
3.42 หน้าต่าง Loading	34
3.43 เว็บแอปพลิเคชันส่วน Waveform ของหน้า List File	35
3.44 ปุ่ม Select this audio	35
3.45 ตัวอย่าง Pop-up หลังกดปุ่ม Select this audio	36
3.46 เว็บแอปพลิเคชันหน้า Result	36
3.47 เว็บแอปพลิเคชันหน้า Result ส่วน filter ข้อมูลไฟล์	37
3.48 เว็บแอปพลิเคชันหน้า Result ส่วน Search ข้อมูลไฟล์	37
3.49 เว็บแอปพลิเคชันหน้า Result ส่วนการแสดงผลค่าเฉลี่ยของ WER.....	37
3.50 เว็บแอปพลิเคชันหน้า Result ส่วนรายชื่อไฟล์	38
3.51 เว็บแอปพลิเคชันหน้า File result details	38

3.53	เว็บแอปพลิเคชันส่วนข้อความที่ได้รับการถอดความ	39
------	---	----

บทที่ 1

บทนำ

1.1 ความเป็นมาของปัญหา

ในปัจจุบันเทคโนโลยีการประมวลผลภาษาธรรมชาติ (Natural Language Processing - NLP) มีการพัฒนาอย่างรวดเร็ว ส่งผลให้เกิดเครื่องมือและโมเดลต่าง ๆ มากมายที่สามารถนำมาใช้ประโยชน์ได้หลากหลาย หนึ่งในนั้นคือโมเดลถอดความจากเสียงพูด (Speech-to-Text) หรือที่เรียกว่าโมเดล ASR (Automatic Speech Recognition) ซึ่งมีความสำคัญต่อองค์กรในหลาย ๆ ด้าน ไม่ว่าจะเป็นการติดตามข้อมูลข่าวสารที่องค์กรสามารถใช้โมเดลถอดความจากเสียงพูดเพื่อติดตามว่าองค์กรถูกอ้างอิงถึงในสื่อใดบ้าง ช่วยให้วิเคราะห์กระแสข่าวและความคิดเห็นของสาธารณะชนที่มีต่อองค์กรได้ หรือจะเป็นการติดตามความนิยมที่องค์กรสามารถใช้โมเดลถอดความจากเสียงพูดเพื่อติดตามความนิยมของตัวบุคคล สินค้าหรือบริการ ช่วยให้สามารถวิเคราะห์ตลาดและกลยุทธ์ทางการตลาดได้อย่างมีประสิทธิภาพ ซึ่งโมเดลถอดความจากเสียงพูดที่มีอยู่ในปัจจุบันมีให้ใช้งานอยู่หลายตัว อย่างไรก็ตาม โมเดลที่มีอยู่ในปัจจุบันยังคงมีข้อจำกัดในการประมวลผลเสียงที่มีความยาว จึงต้องมีการตัดแบ่งเสียงที่จะนำไปใช้งานในโมเดลในความยาวไม่เกิน 30 วินาที เพื่อให้โมเดลทำงานได้เร็วขึ้น แต่ยังพบปัญหาในบางกรณีซึ่งส่งผลต่อความแม่นยำของการถอดความ คือกรณีที่มีบางคำที่ถูกตัดแบ่งกลางระหว่างคำ ทำให้คำสูญเสียความหมายและ โมเดลไม่ทราบความหมายของคำ ส่งผลให้ประสิทธิภาพของโมเดลลดลง

เราจึงมีความสนใจที่จะศึกษาและพัฒนาในส่วนที่เรียกว่า Speech Segmentation เพื่อให้โมเดล ASR มีประสิทธิภาพมากยิ่งขึ้น โดยทำการศึกษาเทคโนโลยี ASR ที่มีอยู่ เพื่อให้ทราบถึงหลักการและการทำงานของแต่ละโมเดลที่จะนำมาเปรียบเทียบ การศึกษาและพัฒนาเทคนิคใหม่ ๆ ในการทำ speech segmentation เพื่อเพิ่มประสิทธิภาพให้กับการตัดแบ่งเสียงโดยไม่สูญเสียความหมายในการนำไปใช้ประมวลผลในโมเดล และเปรียบเทียบผลลัพธ์ที่ได้จากโมเดลที่เลือกใช้งาน ก่อนและหลังจากการทำ speech segmentation ที่เราพัฒนาขึ้น แล้วจึงจะสร้างเว็บ API เพื่อทดสอบว่าการพัฒนา Speech segmentation สามารถนำไปใช้ได้จริงกับโมเดลที่เลือกนำมาพัฒนา

1.2 วัตถุประสงค์ของโครงการ

- 1) เพื่อศึกษาเกี่ยวกับเทคโนโลยีต่าง ๆ ที่ใช้ในแบ่งตัดเสียงและการถอดความจากไฟล์เสียงพูดในวิทยุ, โทรศัพท์ และสื่ออื่น ๆ
- 2) เพื่อศึกษาและพัฒนาวิธีการทำ speech segmentation
- 3) เพื่อเปรียบเทียบประสิทธิภาพในการถอดความจากไฟล์เสียงของโมเดล Whisper และ Wav2Vec2 หลังจากการทำ speech segmentation ที่ได้พัฒนาขึ้นด้วยเทคนิคใหม่ ๆ
- 4) เพื่อสร้างโปรแกรมถอดความจากไฟล์เสียงพูดในรูปแบบของเว็บแอปพลิเคชัน เพื่อใช้ในการเปรียบเทียบประสิทธิภาพของโมเดล
- 5) จัดทำและเผยแพร่บทความวิจัยสู่สาธารณะ ในรูปแบบของการตีพิมพ์บทความวิจัย

1.3 ขอบเขตของโครงการ

- 1) ศึกษาและทำการพัฒนา Algorithm การตัดแบ่งเสียงของโมเดล ASR บน 2 โมเดล คือ Whisper และ Wav2Vec2
- 2) พัฒนาโปรแกรมถอดความจากไฟล์เสียงพูดในรูปแบบของ Web application
- 3) วัดผลประสิทธิภาพของ Algorithm ที่ได้รับการพัฒนาแล้วด้วยการเปรียบเทียบโมเดลที่ใช้ Algorithm การตัดคำตั้งต้น กับ โมเดลที่ใช้ Algorithm การตัดคำที่พัฒนาขึ้นมา
- 4) ดำเนินการวิธีการตัดแบ่งเสียงด้วยภาษา python
- 5) ทดสอบประสิทธิภาพของโมเดลด้วยคลิปวิดีโอ กับ subtitles ของคลิปก้น ๆ จำนวน 10 คลิป ยาวประมาณเสียง 30 นาที
- 6) การตัดแบ่งเสียงแต่ละท่อนจะมีความยาวไม่เกิน 30 วินาที
- 7) ลักษณะของไฟล์เสียงที่รับเข้าเป็น .wav และ .mp3 เท่านั้น
- 8) ในการดำเนินการทดสอบ จะใช้ข้อมูลเสียงที่เป็นภาษาอังกฤษเท่านั้น
- 9) จัดทำบทความวิจัยที่พร้อมสำหรับการส่งตีพิมพ์

1.4 ประโยชน์ที่คาดว่าจะได้รับ

- 1) ได้ผลการเปรียบเทียบประสิทธิภาพในการถอดความจากไฟล์เสียงพูดของแต่ละโมเดลที่เทียบกับโมเดลดั้งเดิม เพื่อเป็นแนวทางในการพัฒนาความสามารถของโมเดล
- 2) ได้ต้นแบบโมเดลที่มีประสิทธิภาพมากที่สุด และสามารถนำ Web application ของต้นแบบโมเดลถอดความจากไฟล์เสียงเพื่อนำไปใช้งานต่อ
- 3) ได้รับองค์ความรู้เกี่ยวกับเทคนิควิธีการทำ Speech segmentation ให้มีประสิทธิภาพ
- 4) ได้เผยแพร่งานวิจัยสู่สาธารณะในรูปแบบของการตีพิมพ์บทความวิจัย

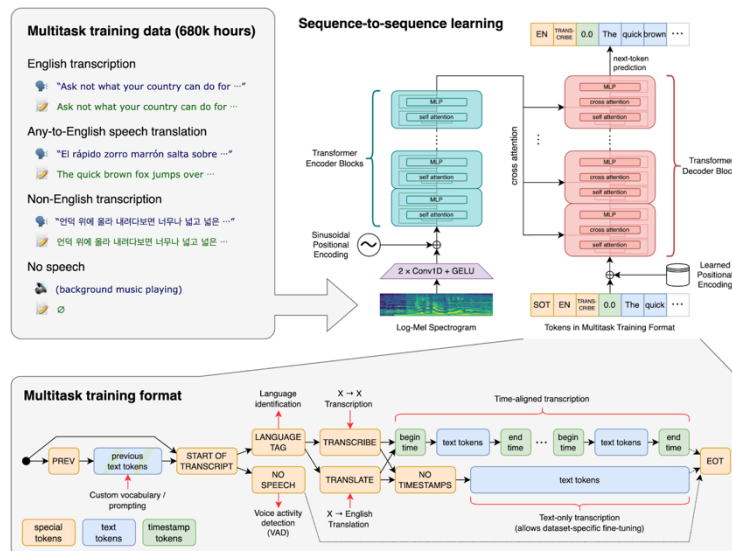
บทที่ 2

ทฤษฎีและงานวิจัยที่เกี่ยวข้อง

2.1 ทฤษฎี

2.1.1 Whisper [1]

Whisper เป็นเทคโนโลยีการประมวลผลเสียงแบบเรียลไทม์ที่พัฒนาโดย OpenAI เป็นโมเดลการรู้จำคำพูด (ASR) ที่ใช้งานสำหรับเสียงทั่ว ๆ ไป ทำงานโดยใช้โมเดล Transformer ขนาดใหญ่ที่ได้รับการฝึกฝนโดยใช้ข้อมูล Multitask training กว่า 680,00 ชั่วโมง ในการเรียนรู้ของโมเดล ซึ่งโมเดลนี้มีการเรียนรู้แบบ supervised learning ทำให้นับว่าเป็น Multilingual Speech Recognition ด้วย ซึ่งสามารถทำการรู้จำคำพูดหลายภาษา การแปลคำพูด และการระบุภาษา



รูปที่ 2.1 Whisper Architecture

2.1.2 Wav2vec2 [2]

Wav2Vec2 เป็นโมเดลปัญญาประดิษฐ์ (AI) จาก Facebook Research ที่ถูกออกแบบมาเพื่อแปลง เสียงพูดให้เป็นข้อความ (Speech-to-Text) ด้วยความแม่นยำสูง โมเดลนี้ทำงานบนพื้นฐานของเทคนิคการ เรียนรู้เชิงลึก (Deep Learning) โดยใช้โมเดล Transformer ที่มีความสามารถในการประมวลผล ภาษาธรรมชาติ (Natural Language Processing) ได้อย่างมีประสิทธิภาพ โมเดล Wav2Vec2 ประกอบไปด้วย 2 ส่วนหลัก คือ

2.1.2.1 Encoder

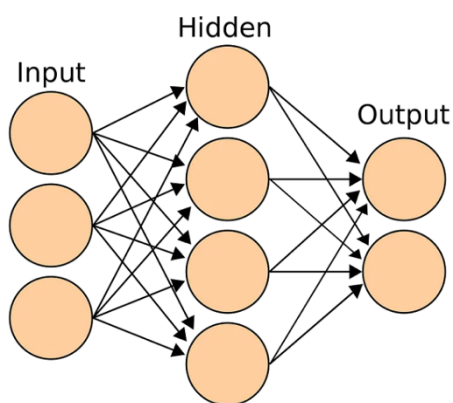
ทำหน้าที่แปลงเสียงพูด (Speech) ไปเป็นรูปแบบข้อมูลที่โมเดลสามารถเข้าใจได้ เปรียบเสมือนการแปลงเสียงพูดให้เป็นภาษาที่คอมพิวเตอร์สามารถอ่านได้

2.1.2.2 Decoder

ทำหน้าที่แปลงข้อมูลที่ได้จาก Encoder ไปเป็นข้อความ (Text) ที่มนุษย์สามารถอ่านเข้าใจได้ เปรียบเสมือนการแปลภาษาคอมพิวเตอร์ให้กลับมาเป็นภาษาที่มนุษย์คุ้นเคย

2.1.3 Deep learning

Deep Learning คือสาขาแขนงหนึ่งของ Machine Learning ที่เลียนแบบการทำงานจากระบบเซลล์ประสาทในสมองของมนุษย์ (Neural Network) โดยโครงสร้างของ Neural Network จะมีลักษณะเหมือนเซลล์ประสาทหลาย ๆ เซลล์เชื่อมต่อกัน เป็นโหนดเชื่อมกัน เรียกว่า Perceptron (เพอร์เซปตรอน)



รูปที่ 2.2 โครงสร้างของ Neural Network

2.1.3.1 ชั้นข้อมูลเข้า (Input Layer)

ทำหน้าที่รับข้อมูลป้อนเข้าสู่ระบบ เปรียบเสมือนประสาทรับความรู้สึกที่ส่งข้อมูลไปยังสมอง

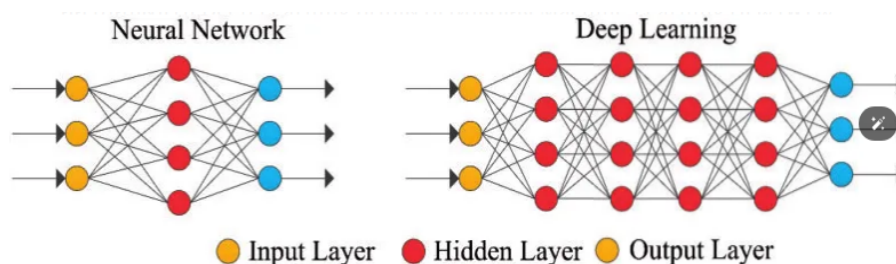
2.1.3.2 ชั้นข้อมูลซ่อน (Hidden Layer)

ทำหน้าที่ประมวลผลข้อมูล เปรียบเสมือนระบบเซลล์ประสาทในสมองที่วิเคราะห์และตีความข้อมูล

2.1.3.3 ชั้นข้อมูลออก (Output Layer)

ทำหน้าที่แสดงผลลัพธ์ที่ได้จากการประมวลผล เปรียบเสมือนการส่งสัญญาณออกจากสมองเพื่อควบคุมการกระทำ

ดังนั้น สมรรถนะของมนุษย์เปรียบเสมือนชั้นข้อมูลซ้อนที่เชื่อมต่อระหว่างชั้นข้อมูลเข้าและชั้นข้อมูลออก แต่ในความเป็นจริง สมรรถนะของมนุษย์มีความซับซ้อนมากกว่าภาพเปรียบเทียบนี้มาก เพื่อให้ Neural Network ทำงานได้อย่างชาญฉลาดและประมวลผลข้อมูลที่ซับซ้อนได้ จำเป็นต้องมีชั้นข้อมูลซ้อนหลายชั้น เรียงซ้อนกันเพื่อส่งข้อมูลและประมวลผลต่อ ๆ กันไป วิธีการนี้เรียกว่า Deep Learning



รูปที่ 2.3 Deep Learning

Deep Learning เป็นเทคนิคการฝึกฝน Neural Network ให้เรียนรู้และวิเคราะห์ข้อมูลได้อย่างลึกซึ้ง ช่วยให้ Neural Network ทำงานได้อย่างชาญฉลาดและมีประสิทธิภาพ

2.1.4 Conformer [5]

Conformer เป็นโมเดลการเรียนรู้เชิงลึกที่ถูกพัฒนาขึ้นมาเพื่อการประมวลผลเสียง (audio processing) โดยเฉพาะ โมเดลนี้ได้รับความนิยมมากในงาน Speech Recognition และ NLP (Natural Language Processing) โดย หลักการของ Conformer แต่ละส่วนของโมเดลมีหน้าที่และหลักการทำงานดังนี้ :

- 1) Encoder : Conformer มีส่วนของ Encoder ที่เป็นโครงสร้าง Transformer ที่ถูกปรับปรุง (modified Transformer) ในแต่ละเฟรม (frame) ของเสียง, Encoder จะดึงคุณลักษณะ (features) ที่สำคัญออกมา เพื่อให้ได้ข้อมูลที่เหมาะสมสำหรับการทำนาย
- 2) Depthwise Separable Convolution (DWSC) : Conformer ใช้ DWSC เพื่อทำการประมวลผลคุณลักษณะที่ได้มาจาก Encoder โดยใช้ท่าทางของ Convolutional Neural Network (CNN) แต่ทำให้มีการใช้ทรัพยากรน้อยลง
- 3) Transformer Block : มี Transformer Block ที่ประกอบไปด้วย Multi-Head Self-Attention Mechanism, Feedforward Neural Network, และ Layer Normalization

เพื่อให้โมเดลสามารถเรียนรู้และรู้สึกถึงความสัมพันธ์ทางต่อเนื่องในข้อมูลเสียง

- 4) Convolutional Block : ในบางเวอร์ชันของ Conformer, มีการใช้ Convolutional Block เพื่อช่วยในการทำนายที่ถูกต้อง
- 5) Linear Layer : มีเลเยอร์ที่ทำหน้าที่ในการแปลงคุณลักษณะที่ได้มาจากขั้นตอนก่อนหน้านี้ในการแปลงคุณลักษณะที่เหมาะสมสำหรับการทำนาย ถูกออกแบบมาเพื่อดึงเสียงและแปลงเป็นข้อความได้ดี โดยมีประสิทธิภาพที่ดีในงาน Speech Recognition และงานที่เกี่ยวข้อง

2.1.5 Transformer [6]

Transformer เป็นส่วนของโครงสร้างโมเดล โดยโครงสร้าง Transformer ให้ความสำคัญและได้รับความนิยมมากในหลายงานที่เกี่ยวข้องกับการประมวลผลลำดับข้อมูล เช่น การแปลภาษา, การจดบันทึกการประชุม, และภารกิจอื่น ๆ โดย หลักการทำงานของ Transformer นั้นมีลักษณะหลักดังนี้ :

- 1) Self-Attention Mechanism: Transformer ใช้ Self-Attention Mechanism เพื่อให้โมเดลสามารถใส่ความสำคัญในลำดับข้อมูลต่าง ๆ ในตัวเอง โดยไม่ขึ้นอยู่กับตำแหน่งที่ตายตัว หรือระดับ
- 2) Multi-Head Attention: เพื่อเพิ่มประสิทธิภาพ, Transformer ใช้หลาย Head ของ Attention ในแต่ละ Layer เพื่อเรียนรู้และควบคุมได้ดีขึ้น
- 3) Positional Encoding: เนื่องจาก Transformer ไม่ได้ใช้ลำดับหรือตำแหน่งของคำในประโยคเพื่อทำนาย, ดังนั้นต้องมีการเพิ่ม positional encoding เข้าไปในลำดับข้อมูลเพื่อให้โมเดลรู้ถึงลำดับของข้อมูล
- 4) Feedforward Neural Network (FFN): ในท้ายที่สุดของแต่ละ Layer, มี Feedforward Neural Network เพื่อปรับปรุงและแปลงลำดับข้อมูล
- 5) Layer Normalization and Residual Connections: การใช้ Layer Normalization และ Residual Connections ช่วยในการสร้างโมเดลที่มีการเรียนรู้ได้ดีและง่ายต่อการฝึก

Transformer ได้ถูกนำมาใช้ในหลายงานและมีหลายเวอร์ชันที่ถูกพัฒนาต่อยอด เช่น BERT (Bidirectional Encoder Representations from Transformers) ที่ได้รับความนิยมในงานประมวลผลภาษาธรรมชาติ (NLP)

2.1.6 Feature Extraction

Feature Extraction หรือ การสกัดคุณลักษณะ จะเป็นการใช้การคำนวณคณิตศาสตร์เพื่อสกัดเอาคุณลักษณะจากรูปภาพออกมา เรียกว่า เคอร์เนล (Kernel) หรือ ตัวกรอง (Filter) โดยตัวกรอง 1 ตัวจะดึงคุณลักษณะที่สนใจออกมาได้ 1 อย่าง ดังนั้นจึงต้องมีตัวกรองหลายตัว เพื่อสกัดหาคุณลักษณะทางพื้นที่มาประกอบกัน โดยตัวกรองจะถูกทาบบลงในพิกเซลภาพแรกมีลักษณะเป็นตาราง 2 มิติที่มีขนาดตามพื้นที่ย่อย ๆ ที่พิจารณา ตัวกรองจะเลื่อนไปจนครบทุกพิกเซลในภาพ และจะได้ฟังก์ชันคุณลักษณะ (Feature map) 3 โดยการสกัดคุณลักษณะจะใช้ข้อมูลปริมาณมากและเวลานาน ดังนั้นจึงใช้การเรียนรู้แบบถ่ายโอน (Transfer Learning) เป็นเทคนิคการเรียนรู้ของเครื่อง จะใช้การเรียนรู้ด้วยการนำโครงสร้างของโมเดลที่ฝึกเรียบร้อยแล้วนำข้อมูลของใหม่มาฝึก จะช่วยเรื่องในการประหยัดเวลาอย่างมากเนื่องจากระยะเวลาในการฝึกตั้งแต่ต้นจนจบกระบวนการใช้เวลานานและมีความซับซ้อน อีกทั้งยังต้องใช้ Data set ที่มีขนาดใหญ่และใช้เวลาในการประมวลผลหลายวันจนถึงหลายสัปดาห์

2.1.7 Log-Mel Spectrogram

เป็นการต่อยอดแนวความคิดจาก Mel Spectrogram โดยที่ Log Mel Spectrogram จะเป็นการแสดงความถี่บนมาตราส่วน Mel และแสดงแอมพลิจูดแบบ logarithm แปลงเป็น เดซิเบล (dB) ซึ่งสะท้อนการรับรู้ความดังของมนุษย์ได้ดีขึ้น โดยการใช้วิธีนี้จะเป็นการเน้นช่วงไดนามิกของเสียง ทำให้เสียงที่เบามีความเด่นชัดมากขึ้น และมีจุดเด่นคือ สามารถใช้เป็นคุณสมบัติสำหรับการฝึกโมเดลในงานเสียงต่าง ๆ โดยเฉพาะงานที่มีข้อมูลเดซิเบลเป็นสำคัญ

2.1.8 Speech segmentation

Speech segmentation คือ การแบ่งส่วนคำพูด ซึ่งเป็นกระบวนการแบ่งลำดับเสียงพูดต่อเนื่องออกเป็นหน่วยย่อย ๆ ที่มีความหมาย เช่น คำ วลี หรือประโยค เปรียบเสมือนการแยกตัวอักษรออกจากประโยคเพื่ออ่าน ซึ่งการแบ่งส่วนคำพูดมีความสำคัญต่อระบบประมวลผลภาษาพูด

หลายประเภท เช่น ระบบจดจำเสียงพูด (Automatic Speech Recognition) และ ระบบสังเคราะห์เสียงพูด (Text to speech) โดย การแบ่งส่วนคำพูดมีหลากหลายวิธี เช่น

- 1) การแบ่งส่วนตามเสียง (Acoustic segmentation) วิธีนี้ใช้วิเคราะห์คุณสมบัติทางเสียง เช่น ความถี่ พลังงาน และ formant เพื่อแบ่งลำดับเสียงออกเป็นหน่วยย่อย ๆ ที่มีความคล้ายคลึงกัน
- 2) การแบ่งส่วนตามพยางค์ (Syllabic segmentation) วิธีนี้ใช้วิเคราะห์โครงสร้างของพยางค์เพื่อแบ่งลำดับเสียงออกเป็นหน่วยย่อย ๆ ที่เป็นพยางค์
- 3) การแบ่งส่วนตามคำศัพท์ (Lexical segmentation) วิธีนี้ใช้พจนานุกรมและหลักภาษาเพื่อแบ่งลำดับเสียงออกเป็นหน่วยย่อย ๆ ที่เป็นคำ
- 4) การแบ่งส่วนตามไวยากรณ์ (Grammatical segmentation) วิธีนี้ใช้หลักไวยากรณ์เพื่อแบ่งลำดับเสียงออกเป็นหน่วยย่อย ๆ ที่เป็นวลีหรือประโยค
- 5) การแบ่งส่วนแบบผสมผสาน (Hybrid segmentation) วิธีนี้ผสมผสานเทคนิคหลายวิธีเข้าด้วยกันเพื่อเพิ่มประสิทธิภาพในการแบ่งส่วนคำพูด

2.1.9 Voice Activity Detection (VAD) [11]

Voice Activity Detection (VAD) เป็นเทคโนโลยีที่จำเป็นในระบบการสื่อสารและประมวลผลเสียง มีจุดประสงค์เพื่อระบุว่าเสียงใดคือคำพูดของมนุษย์ ทำงานโดยการวิเคราะห์สัญญาณเสียงเพื่อแยกความแตกต่างระหว่างส่วนที่เป็นคำพูดและส่วนที่ไม่ใช่คำพูด โดยทั่วไปจะทำการวัดระดับพลังงาน ปริมาณความถี่ และแพทเทิร์นในเสียง โดย ปัจจัยหลักที่สำคัญใน Voice Activity Detection คือ

- 1) Energy Thresholds สำหรับตรวจสอบว่าสัญญาณที่ได้รับนั้นเป็นเสียงพูดหรือไม่
- 2) Zero Crossing Rate (ZCR) วัดอัตราที่สัญญาณเปลี่ยนจากบวกเป็นลบและลบเป็นบวก ซึ่งบ่งบอกถึงความถี่และการแสดงเสียงพูด
- 3) Mel-Frequency Cepstral Coefficients (MFCC) ใช้เพื่อบันทึกลักษณะเฉพาะของคำพูดของมนุษย์ ทำให้ระบบเรียนรู้แพทเทิร์นและสามารถคาดการณ์ได้อย่างแม่นยำ

2.1.10 Levenshtein Distance

Levenshtein distance เป็นการวัดความแตกต่างระหว่างสตริงสองตัว (หรือข้อความสองชุด) โดยนับจำนวนขั้นตอนขั้นต่ำที่ต้องใช้ในการแก้ไขข้อความหนึ่งให้กลายเป็นข้อความอีกข้อความหนึ่ง ขั้นตอนเหล่านี้ประกอบด้วย การแทรก (insert), ลบ (delete), หรือ แทนที่ (substitute) อักษร

2.1.10.1 หลักการทำงานของ Levenshtein Distance

- 1) การแทรก (Insert) : เพิ่มอักษรใหม่ในตำแหน่งใดตำแหน่งหนึ่งของข้อความ เช่น การเปลี่ยนคำว่า "cat" เป็น "cats" ต้องเพิ่มตัว "s" จึงใช้ขั้นตอนการแทรก 1 ครั้ง
- 2) การลบ (Delete) : ลบอักษรออกจากข้อความ เช่น การเปลี่ยนคำว่า "cats" เป็น "cat" ต้องลบตัว "s" ออก จึงใช้ขั้นตอนการลบ 1 ครั้ง
- 3) การแทนที่ (Substitute) : เปลี่ยนอักษรในตำแหน่งหนึ่งให้เป็นอักษรตัวอื่น เช่น การเปลี่ยนคำว่า "cat" เป็น "cut" ต้องเปลี่ยนตัว "a" เป็น "u" จึงใช้ขั้นตอนการแทนที่ 1 ครั้ง

2.1.10.2 ตัวอย่างการคำนวณ Levenshtein Distance

สมมติว่าต้องการคำนวณ Levenshtein distance ระหว่างคำว่า "kitten" และ "sitting"

- 1) แทนที่ 'k' ด้วย 's': จาก "kitten" เป็น "sitten" (1 ขั้นตอน)
 - 2) แทนที่ 'e' ด้วย 'i': จาก "sitten" เป็น "sittin" (2 ขั้นตอน)
 - 3) เพิ่ม 'g' ที่ท้ายข้อความ: จาก "sittin" เป็น "sitting" (3 ขั้นตอน)
- ดังนั้น Levenshtein distance ระหว่าง "kitten" และ "sitting" คือ 3

2.1.10.3 การใช้ตารางในการคำนวณ

การคำนวณ Levenshtein distance สามารถทำได้โดยการสร้างตารางสองมิติ ซึ่งแต่ละช่องในตารางจะเก็บระยะทางขั้นต่ำจากการแก้ไขข้อความเริ่มต้นให้กลายเป็นข้อความเป้าหมายตามขั้นตอนที่กล่าวมา

- 1) ตั้งค่าแถวและคอลัมน์แรก ค่าของช่องแรกในแถวและคอลัมน์แต่ละช่องจะเป็นตัวเลขลำดับที่เพิ่มขึ้น ซึ่งแสดงถึงจำนวนการแก้ไขที่ต้องทำเพื่อเปลี่ยนข้อความเริ่มต้นหรือข้อความเป้าหมายให้กลายเป็นข้อความว่าง

- 2) คำนวณค่าของแต่ละช่องในตาราง แต่ละช่องจะมีค่าที่คำนวณจากค่าในช่องที่อยู่ทางซ้ายบน ซ้าย หรือบน โดยเลือกค่าน้อยที่สุดและบวกเพิ่ม 1 สำหรับการแทรก ลบ หรือแทนที่

Computing Levenshtein distance – 4

$$D(i,j) = \min \begin{cases} D(i-1,j-1) + d(s_i,t_j) & //subst/copy \\ D(i-1,j)+1 & //insert \\ D(i,j-1)+1 & //delete \end{cases}$$

A *trace* indicates where the min value came from, and can be used to find edit operations and/or a best *alignment* (may be more than 1)

	C	O	H	E	N
M	1	2	3	4	5
C	1	2	3	4	5
C	2	3	3	4	5
O	3	2	3	4	5
H	4	3	2	3	4
N	5	4	3	3	3

รูปที่ 2.4 การคำนวณ Levenshtein distance

2.1.11 การวัดประสิทธิภาพ

2.1.11.1 WER

WER ย่อมาจาก Word Error Rate เป็นตัวชี้วัดที่ใช้ประเมินความแม่นยำของ ASR (Automatic Speech Recognition) โดยคำนวณจากสมการดังนี้

$$\text{WER} = \frac{S + D + I}{N} \times 100$$

$$\text{Accuracy} = 100 - \text{WER} \quad (2.1)$$

N คือ จำนวนคำของ reference (ประโยคจริง)

I คือ คำที่ต้องใส่เพิ่มใน prediction เพื่อให้มันเหมือนประโยคจริง

D คือ คำที่ต้องลบออกจาก prediction เพื่อให้มันเหมือนประโยคจริง

S คือ คำที่ต้องแทนที่ใน prediction เพื่อให้มันเหมือนประโยคจริง

เลือก combination $I + D + S$ ที่ต่ำที่สุด (แก่น้อยสุด) แล้วหารด้วย N เพื่อให้ได้ WER

ผลลัพธ์ของ WER ที่มีค่าต่ำจะมีความหมายว่า โมเดล ASR นั้น ๆ มีความแม่นยำสูง และ WER ที่มีค่าสูงจะมีความหมายว่า โมเดล ASR นั้น ๆ มีความแม่นยำต่ำ

2.1.11.2 CER

CER ย่อมาจาก Character Error Rate เป็นตัวชี้วัดที่ใช้ประเมินความแม่นยำของ ASR (Automatic Speech Recognition) model โดยคำนวณจากสมการ ดังนี้

$$CER = \frac{S + D + I}{N} \quad (2.2)$$

N คือจำนวนอักษรของ reference (ประโยคจริง)

I คืออักษรที่ต้องใส่เพิ่มใน prediction เพื่อให้มันเหมือนประโยคจริง

D คืออักษรที่ต้องลบออกจาก prediction เพื่อให้มันเหมือนประโยคจริง

S คืออักษรที่ต้องแทนที่ใน prediction เพื่อให้มันเหมือนประโยคจริง

เลือก combination I + D + S ที่ต่ำที่สุด (แก่น้อยสุด) แล้วหารด้วย

เพื่อให้ได้ CER ผลลัพธ์ของ CER ที่มีค่าต่ำจะมีความหมายว่า โมเดล ASR นั้น ๆ มีความแม่นยำสูง และ CER ที่มีค่าสูงจะมีความหมายว่า โมเดล ASR นั้น ๆ มีความแม่นยำต่ำ

2.2 งานวิจัยที่เกี่ยวข้อง

2.2.1 E2E Segmenter: Joint Segmenting and Decoding for Long-Form ASR

[8]

งานวิจัยนี้เสนอโมเดล E2E Segmenter สำหรับโมเดล Long-Form ASR (Automatic Speech Recognition) โดยโมเดลนี้สามารถแบ่งไฟล์เสียงออกเป็นส่วนย่อย ๆ และถอดเสียงแต่ละส่วนพร้อมกัน ซึ่งจะเป็นวิธีที่ช่วยเพิ่มความแม่นยำและประสิทธิภาพของ Long-Form ASR และแก้ปัญหาของระบบการรู้จำเสียงพูดแบบเดิม คือ การแบ่งเสียงออกเป็นส่วน ๆ โดยใช้ Voice Activity Detector (VAD) ที่แยกแยะเสียงพูดกับเสียงรบกวนจากคุณสมบัติของเสียงเท่านั้น ข้อจำกัดของ VAD คือ มันอาจแบ่งประโยคที่สมบูรณ์ออกเป็นหลายส่วนจนเกิดเป็นผลลัพธ์ที่ไม่ถูกต้อง

Segmenter	YT.LONG					YT.SHORT				
	WER	EOS50	EOS75	# Seg.	# State	WER	EOS50	EOS75	# Seg.	# State
(a) B1: Fixed-10s	20.05	-	-	99.8	5496	13.63	-	-	33.9	3965
B2: Fixed-20s	18.22	-	-	51.3	5753	11.57	-	-	17.5	4048
B3: VAD	18.16	260	490	95.2	8396	11.46	460	660	28.3	5689
E1: E2E (best)	17.05	130	353	56.2	8098	10.49	210	365	18.2	5672
⇒ E1 vs. B3	-6.1%	-130	-137			-8.5%	-250	-265		
(b) E2: E2E-0.0	17.60	180	550	16.4	7244	11.10	510	855	5.8	5416
E3: E2E-0.5	17.38	180	550	18.7	7958	10.91	460	800	6.8	5705
E4: E2E-1.0	17.19	220	535	26.3	8513	10.68	390	745	9.6	5771
E5: E2E-1.5	17.09	195	445	38.6	8366	10.63	280	395	13.3	5700
E6: E2E-2.0	17.05	130	353	56.2	8098	10.49	210	365	18.2	5672
E7: E2E-2.5	17.08	100	355	80.7	7917	10.49	200	345	25.7	5371
E8: E2E-3.0	17.06	100	415	116.3	7362	10.58	180	230	35.6	5039
E9: E2E-3.5	17.22	90	280	178.5	6926	10.56	180	450	49.8	4790
E10: E2E-4.0	17.48	75	210	300.0	6237	10.58	180	380	74.7	4569
E11: E2E-4.5	17.76	90	255	477.6	5410	10.78	140	280	119.1	3975

รูปที่ 2.5 ผลลัพธ์เปรียบเทียบประสิทธิภาพระหว่าง E2E segmenter, VAD segmenter และแบบ
เวลาคงที่

โดยงานวิจัยแสดงถึงความเป็นไปได้ที่จะพัฒนา Speech segmentation เพื่อเพื่อประสิทธิภาพของโมเดล ASR ซึ่งจากผลการทดลองงานวิจัยนี้แสดงให้เห็นว่า E2E Segmenter มีประสิทธิภาพดีกว่า VAD โดยมีทั้งอัตราความผิดพลาดของคำ (WER) ที่น้อยกว่า และใช้เวลาน้อยกว่าในการแบ่งช่วง (end-of-segment latency) เมื่อเปรียบเทียบกับระบบ VAD แบบเดิมในการถอดคำของเสียงแบบ Long-Form บน Youtube

2.2.2 Smart Speech Segmentation using Acousto-Linguistic Features with look-ahead [9]

งานวิจัย Smart Speech Segmentation using Acousto-Linguistic Features with look-ahead เสนอวิธีการแบ่งประโยคสำหรับ ASR (Automatic Speech Recognition) ที่มีประสิทธิภาพ โดยใช้ข้อมูลทั้งด้าน acoustic และภาษาศาสตร์ ร่วมกับเทคนิค look-ahead งานวิจัยนี้ เริ่มต้นด้วยการอธิบายถึงข้อจำกัดของวิธีการแบ่งประโยคแบบดั้งเดิม เช่น การใช้ silence timeout หรือ voice activity detector (VAD) ซึ่งทำงานโดยอาศัยข้อมูล acoustic เพียงอย่างเดียว โมเดลจึงเสนอวิธีการแบ่งประโยคแบบใหม่ ที่ใช้โมเดล deep learning เรียนรู้จากทั้งข้อมูล acoustic และภาษาศาสตร์ ซึ่งข้อมูล acoustic ประกอบด้วย spectral features และ prosodic features ข้อมูลภาษาศาสตร์ประกอบด้วย part-of-speech tags และ n-gram language model โมเดล deep learning ที่ใช้เป็นโมเดลแบบ Recurrent neural network (RNN) ที่ถูกปรับแต่งสำหรับงานแบ่งประโยค และเทคนิค look-ahead ช่วยให้โมเดลสามารถคาดเดาคำถัดไปได้ที่จะพูด ซึ่งช่วยเพิ่มประสิทธิภาพในการแบ่งประโยค

Locale	Model	BLEU	Gain
en-GB	v1	35.5	
	v2	36	+0.5
	v3	36.9	+1.4
es-ES	v1	41.4	
	v2	41.6	+0.2
	v3	41.8	+0.4
el-GR	v1	34.5	
	v2	35.1	+0.7
	v3	35.3	+0.8
fr-FR	v1	38.4	
	v2	39.1	+0.7
	v3	39.6	+1.2
it-IT	v1	44.7	
	v2	45.7	+1.0
	v3	46	+1.3
ro-RO	v1	40.3	
	v2	40.9	+0.6
	v3	41.5	+1.2

รูปที่ 2.6 ผลลัพธ์ BLEU Score ของ Smart Speech segmentation

ผลลัพธ์งานวิจัยแสดงให้เห็นว่าโมเดลนี้มีประสิทธิภาพดีกว่าวิธีการแบบดั้งเดิม เห็นได้ชัดว่างานวิจัยนี้สามารถเพิ่มผลลัพธ์ค่า BLEU Score ที่บ่งบอกว่าโมเดล ASR นั้น ๆ มีประสิทธิภาพมากยิ่งขึ้น

บทที่ 3

การดำเนินงาน

การดำเนินการเพื่อพัฒนาการตัดแบ่งส่วนเสียงนั้น มีขั้นตอนการดำเนินการ ดังนี้

3.1 เตรียมการพัฒนาโมเดล

3.1.1 โมเดลที่ใช้ร่วมกับการพัฒนาการตัดแบ่งส่วนเสียง

การพัฒนาเทคนิคการแบ่งส่วนเสียงเริ่มต้นด้วยการเลือกโมเดล ASR ที่จะใช้ร่วมในการปรับปรุงประสิทธิภาพ โดยศึกษาช่วงก่อนที่โมเดลจะนำไฟล์เสียงไปแปลงเป็นข้อความ (Speech-to-Text) และพัฒนาเทคนิคการแบ่งส่วนเสียงในช่วงนั้น ผู้พัฒนาเลือกใช้โมเดลดังต่อไปนี้

- 1) Whisper
- 2) Wav2Vec2

3.1.2 เตรียมข้อมูลสำหรับการวัดประสิทธิภาพของโมเดล

ข้อมูลที่ใช้ในการทดสอบต้องเป็นไฟล์เสียงที่มีคำอธิบายข้อความที่ถูกต้องและเชื่อถือได้ ข้อมูลที่เลือกมานี้อาจต้องมีการปรับปรุงและทำความสะอาด (clean) เพื่อให้เหมาะสมกับการทดสอบโมเดลอย่างมีประสิทธิภาพสูงสุด โดยข้อมูลสำหรับการทดสอบนี้ประกอบด้วย ThonburianV3 มีพื้นฐานมาจากโมเดล Whisper แต่มีความเชี่ยวชาญด้านภาษาไทยมากกว่า

- 1) ไฟล์เสียงที่นำมาจาก YouTube พร้อมกับไฟล์คำบรรยาย (Subtitle) ของเสียงเหล่านั้น
- 2) ใช้ไฟล์เสียงจำนวน 2 ไฟล์ที่เป็นเสียงภาษาอังกฤษ พร้อมคำบรรยายภาษาอังกฤษ เพื่อสร้างชุดข้อมูลสำหรับการทดสอบประสิทธิภาพของโมเดล

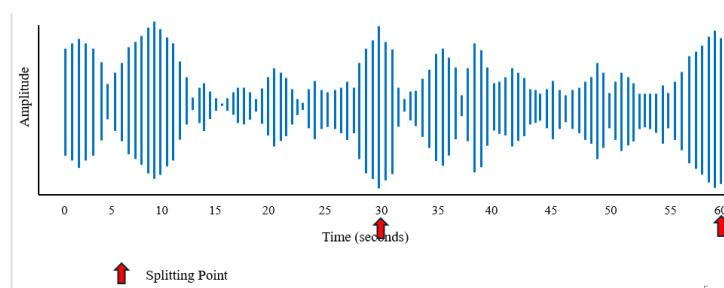
3.2 ขั้นตอนการพัฒนาการตัดแบ่งเสียง

3.2.1 คิดค้นและพัฒนา อัลกอริทึม ที่จะใช้ในการทำ Speech segmentation

การพัฒนาอัลกอริทึมสำหรับการแบ่งส่วนเสียง (Speech Segmentation) เริ่มจากการคิดค้นวิธีใหม่ ๆ ที่สามารถใช้เพิ่มประสิทธิภาพของการแปลงเสียงเป็นข้อความ โดยการพัฒนา Speech Segmentation จะถูกนำไปเพิ่มในขั้นตอนก่อนที่โมเดลจะทำการประมวลผลเสียงผ่าน Encoder

วิธีการที่สามารถใช้ในการแบ่งส่วนเสียงเพื่อพัฒนาอัลกอริทึม Speech Segmentation มีหลายแนวทางดังนี้

3.2.1.1 วิธีที่ 1 Fix-sized Audio chunk (วิธีดั้งเดิม)



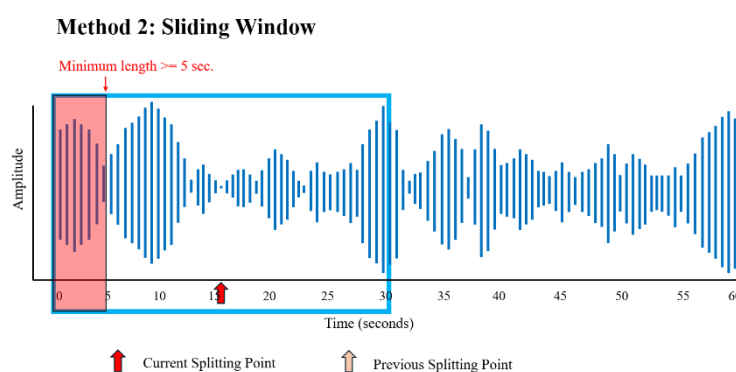
รูปที่ 3.1 ตัวอย่างการตัดแบ่งเสียงแบบ Fix-sized Audio chunk

หลักการของ Fix-sized Audio chunk คือการตัดแบ่งไฟล์เสียงออกเป็น Chunk ที่มีความยาวเท่ากันตามระยะเวลาที่กำหนด เช่น ทุก 30 วินาที

ข้อดีของ Fix-sized Audio chunk คือ วิธีนี้มีความง่ายและรวดเร็ว เพราะเพียงแค่กำหนดขนาดของ chunk แล้วตัดตามขนาดนั้น

ปัญหาของ Fix-sized Audio chunk คือ มีโอกาสสูงที่จะตัดเสียงในขณะที่พูดกลางคำ ทำให้โมเดล ASR อาจรู้จำคำผิดพลาด นอกจากนี้วิธีนี้ยังไม่คำนึงถึงตำแหน่งที่แอมพลิจูดของเสียงต่ำสุด จึงอาจพลาดจุดตัดที่เหมาะสม

3.2.1.2 วิธีที่ 2 Sliding Windows

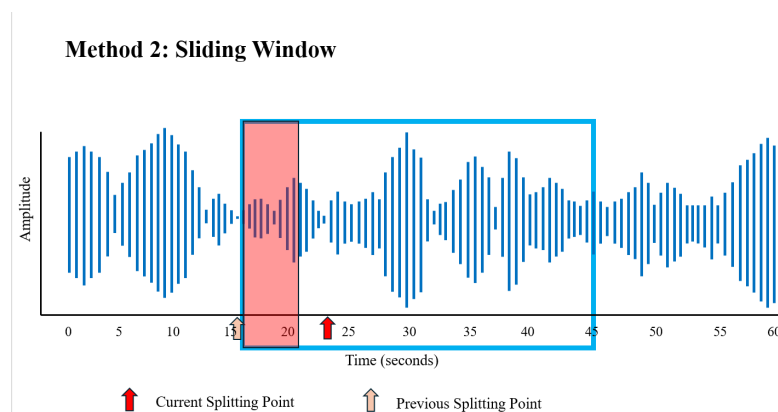


รูปที่ 3.2 ตัวอย่างการตัดแบ่งเสียงแบบ Sliding Windows ขั้นตอนที่ 1

หลักการของ Sliding Windows คือการเลื่อนหาจุดที่มี amplitude ต่ำที่สุดในรอบความยาวที่กำหนด เพื่อใช้ในการตัด โดยกำหนด minimum length ไว้ที่ 5 วินาทีและ

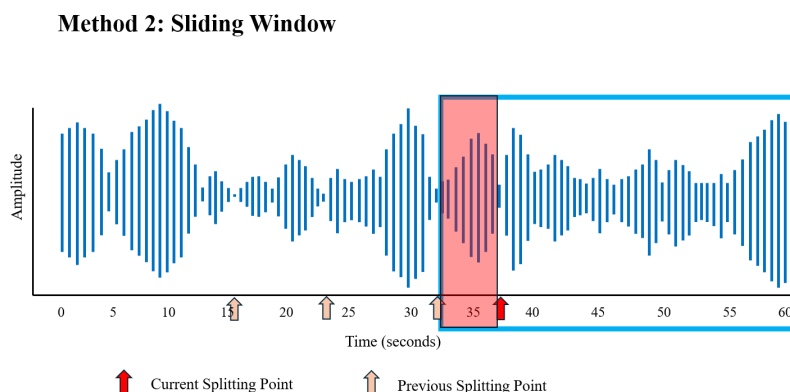
maximum length ไว้ที่ 30 วินาที เพื่อไม่ให้ขนาดของ chunk สั้นและยาวเกินไป เนื่องจากกำหนดให้จุดตัดไม่สามารถอยู่ในกรอบ ของ minimum length และนอกกรอบ maximum length ได้

โดยในตัวอย่างจะเห็นได้ว่าจุดที่ amplitude ต่ำที่สุดคือจุดที่ถูกศรีสีแดงชี้อยู่ จะถูกใช้เป็นจุดตัด



รูปที่ 3.3 ตัวอย่างการตัดแบ่งเสียงแบบ Sliding Windows ขั้นตอนที่ 2

เมื่อได้จุดตัดแรกแล้วจะทำการเลื่อนจุดเริ่มต้นของกรอบ window ไปที่จุดตัดแรกและทำการกำหนด minimum length และ maximum length ใหม่อีกครั้ง เพื่อหาจุดตัดจุดถัดไปในการตัดแบ่งเสียง



รูปที่ 3.4 ตัวอย่างการตัดแบ่งเสียงแบบ Sliding Windows ขั้นตอนที่ 3

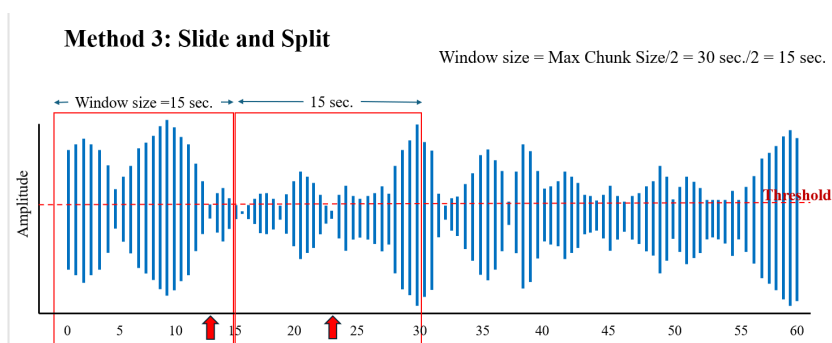
ทำแบบนี้ต่อไปเรื่อย ๆ จนสิ้นสุดความยาวของเสียงจะทำให้เราได้ chunk ของเสียงที่มีจุดตัดที่ amplitude ต่ำที่สุด

ข้อดีของ Sliding Windows คือมีความยืดหยุ่นในการหาจุดตัด โดยสามารถปรับขนาดของหน้าต่างได้

ปัญหาของ Sliding Windows คือ

- 1) จำนวนชิ้นส่วนที่ได้อาจมากเกินไป ทำให้การคำนวณเพิ่มขึ้น
- 2) ชิ้นส่วนเสียงมีแนวโน้มจะมีขนาดเล็ก
- 3) อาจพลาดจุดแอมพลิจูดต่ำสุดในช่วงเวลาที่กำหนด

3.2.1.3 วิธีที่ 3 Slide and split



รูปที่ 3.5 ตัวอย่างการตัดแบ่งเสียงแบบ Slide and split ขั้นตอนที่ 1

หลักการของ slide and split คือ

- 1) ใช้หน้าต่างสองชุด (คือหน้าต่างซ้ายและขวา)ที่มีความยาวครึ่งหนึ่งของขนาดกรอบที่ต้องการ (เช่น 15 วินาที สำหรับขนาดกรอบสูงสุด 30 วินาที) โดยจัดเรียงจากซ้ายไปขวา
- 2) ในแต่ละหน้าต่าง ให้ทำการหา amplitude ที่ต่ำกว่าค่า threshold และอยู่ทางขวาสุด เพื่อใช้เลือกเป็นจุดตัด
- 3) หากหน้าต่างไหนไม่มีจุด amplitude ที่ต่ำกว่าค่า threshold ให้ข้ามโดยไม่มีจุดตัด
- 4) จากตัวอย่างจุดที่ต่ำกว่าค่า threshold และอยู่ทางขวาสุดในแต่ละหน้าต่าง คือจุดที่ถูกสกรีนสีแดงชี้อยู่

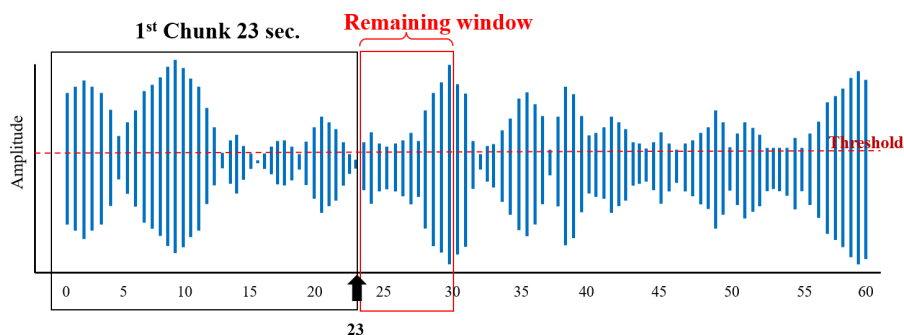
กฎของการเลือกจุดตัด

- 1) เลือกจุดตัดของหน้าต่างทางขวาก่อนเสมอ (ถ้ามีจุดที่ตัดได้) หากหน้าต่างทางขวาไม่มีจุดตัดให้เลือกหน้าต่างทางซ้ายแทน

- 2) หากทั้ง 2 หน้าต่างไม่มีจุดที่ต่ำกว่า threshold หรือจุดที่ตัดได้ เลย ให้เลือกจุดที่ amplitude ต่ำที่สุดของหน้าต่างขวา

หลักการ การเลื่อนหน้าต่าง หลังจากเลือกจุดตัดตามกฎการเลือกแล้ว

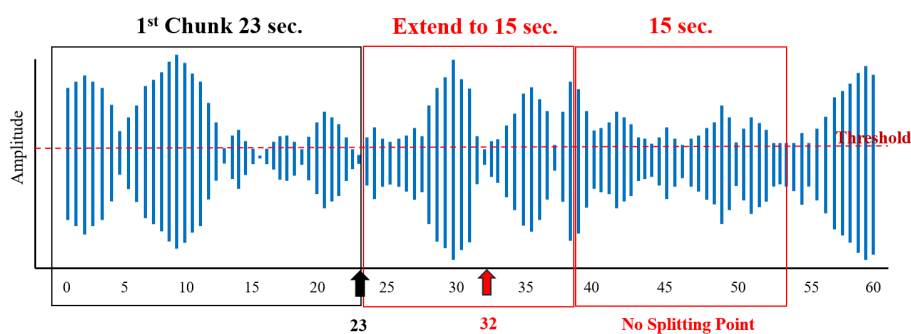
Method 3: Slide and Split



รูปที่ 3.6 ตัวอย่างการตัดแบ่งเสียงแบบ Slide and split ขั้นตอนที่ 2

หากจุดตัดอยู่หน้าต่างขวาให้ขยายหน้าต่างที่เหลืออยู่ของหน้าต่างด้านขวาออกไปให้เป็น 15 วินาที กลายเป็นหน้าต่างซ้ายใหม่

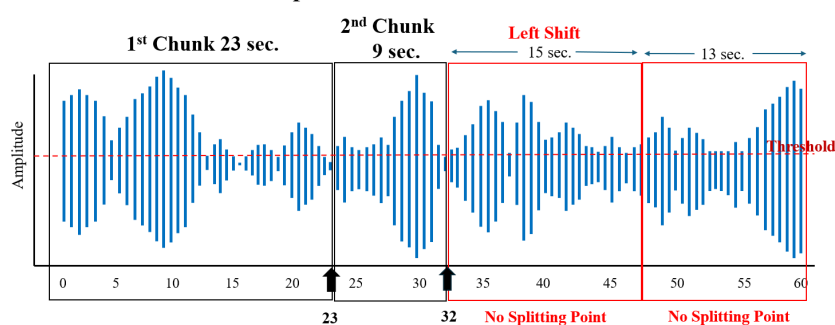
Method 3: Slide and Split



รูปที่ 3.7 ตัวอย่างการตัดแบ่งเสียงแบบ Slide and split ขั้นตอนที่ 3

และเพิ่มหน้าต่างที่ 2 อีก 15 วินาทีกลายเป็นหน้าต่างขวาใหม่ จะได้เป็นกรอบ 30 วินาที 2หน้าต่างดังเดิม

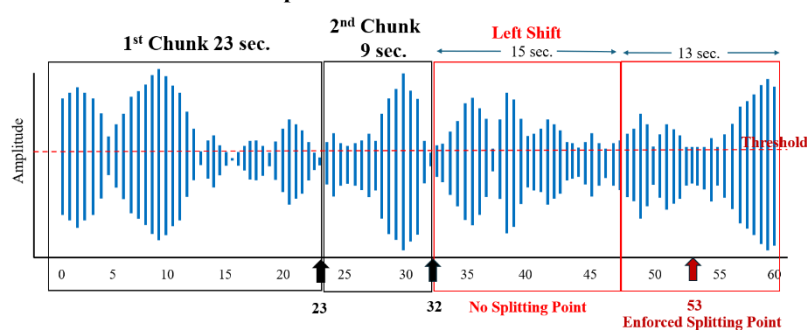
Method 3: Slide and Split



รูปที่ 3.8 ตัวอย่างการตัดแบ่งเสียงแบบ Slide and split ขั้นตอนที่ 4

หากจุดตัดอยู่ในหน้าต่างด้านซ้าย ให้เลื่อนจุดเริ่มต้นของกรอบหน้าต่างทั้งหมดไปที่จุดตัด และหาจุดตัดจุดต่อไป

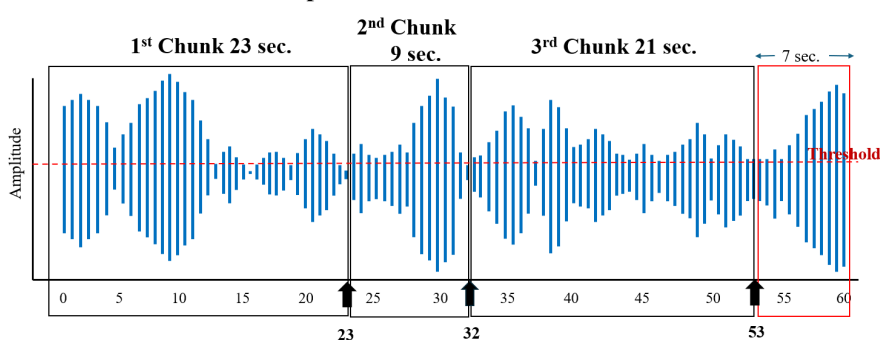
Method 3: Slide and Split



รูปที่ 3.9 ตัวอย่างการตัดแบ่งเสียงแบบ Slide and split ขั้นตอนที่ 5

หากทั้ง 2 หน้าต่างไม่มีจุดที่ amplitude ต่ำกว่าค่า threshold เลย ให้ตัดที่จุดที่ amplitude ต่ำที่สุดของหน้าต่างด้านขวา และเลื่อนกรอบด้วยวิธีเลื่อนกรอบของหน้าต่างขวา

Method 3: Slide and Split

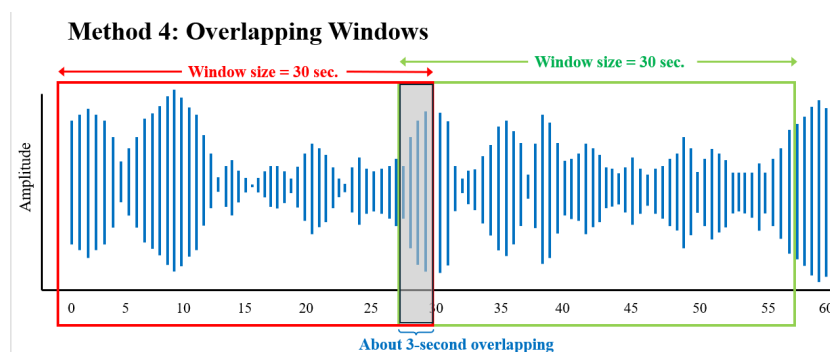


รูปที่ 3.10 ตัวอย่างการตัดแบ่งเสียงแบบ Slide and split ขั้นตอนที่ 6

ข้อดีของ slide and split คือ ลดจำนวนของ chunk และทำให้ chunk มีขนาดกลางไปจนถึงขนาดใหญ่

ข้อเสียของ slide and split อาจจะมีการตัดกลางคำ ทำให้การถอดความจากเสียงพูดเกิดความผิดพลาดได้

3.2.1.4 วิธีที่ 4 Overlapping Windows



รูปที่ 3.11 ตัวอย่างการตัดแบ่งเสียงแบบ Overlapping Windows ขั้นตอนที่ 1

หลักการของ Overlapping Windows คือ ใช้กรอบหน้าต่างที่มีการทับซ้อนกันในการตัดเสียง โดยแต่ละหน้าต่างมีขนาดเท่ากัน (เช่น 30 วินาที) จากนั้นใช้ Levenshtein distance เพื่อวัดความแตกต่างระหว่างข้อความสองชุดที่ได้จากการถอดความ และ Merge ข้อความเข้าด้วยกัน

Levenshtein distance เป็นการวัดความแตกต่างระหว่างสตริงสองตัว (หรือข้อความสองชุด) โดยนับจำนวนขั้นตอนขั้นต่ำที่ต้องใช้ในการแก้ไขข้อความหนึ่งให้กลายเป็นข้อความอีกข้อความหนึ่ง ขั้นตอนเหล่านี้ประกอบด้วย การแทรก (insert), ลบ (delete), หรือ แทนที่ (substitute) อักขระ (ดูบทที่ 2 หัวข้อ 2.1.10 หน้า 10)

การ Merge ข้อความมีวิธีการ คือ การค่อย ๆ ขยับข้อความที่ได้จากการถอดความจากทั้งสองหน้าต่าง ให้ ซ้อนหรือ Overlaps กันมากขึ้นทีละสเต็ป โดยจะใช้ส่วนท้ายของข้อความชุดแรกและส่วนต้นของข้อความชุดที่สอง เพื่อคำนวณหา Levenshtein distance ที่เป็น 0 หรือค่าที่สุทธระหว่างข้อความทั้งสองชุด

ตัวอย่างการ Merge ข้อความ

Example

- Red Window Text: “Once upon a time when the flakes”
- Green Window Text: “when the flakes of snow were falling”

รูปที่ 3.12 ตัวอย่างประโยค

Step 1

Distance = 5

Once upon a time when the flakes
when the flakes of snow were falling

Step 2

Distance = 7

Once upon a time when the flakes
when the flakes of snow were falling

Step 3

Distance = 0

Once upon a time when the flakes
when the flakes of snow were falling

Step 4

Distance = 8

Once upon a time when the flakes
when the flakes of snow were falling

รูปที่ 3.13 ตัวอย่างการตัดแบ่งเสียงแบบ Overlapping Windows ขั้นตอนที่ 2

ในที่นี้จะเห็นได้ว่าใน Step ที่ 3 มีค่า Levenshtein distance เป็น 0 หมายความว่าส่วนที่ซ้อนกันของข้อความทั้งสองชุดไม่มีความแตกต่างกัน ดังนั้นผลลัพธ์ข้อความนี้จะได้ “Once upon a time when the flakes of snow were falling”

Step 1

Distance = 4

Once upon a time when the fake
hen the flakes of snow were falling

Wrong predictions by ASR

Step 2

Distance = 5

Once upon a time when the fake
hen the flakes of snow were falling

Step 3

Distance = 3

Once upon a time when the fake
hen the flakes of snow were falling

Step 4

Distance = 11

Once upon a time when the fake
hen the flakes of snow were falling

รูปที่ 3.14 ตัวอย่างการตัดแบ่งเสียงแบบ Overlapping Windows ขั้นตอนที่ 3

ในกรณีที่ข้อความที่ได้นั้นถูกตัดตรงกลางระหว่างคำจะทำให้ผลลัพธ์จากการถอดความที่ได้นั้นไม่ถูกต้องดังที่เห็นในตัวอย่าง ส่งผลให้ค่า Levenshtein distance ไม่มีทางเป็น 0

ในกรณีนี้ เราจะเลือก Step ที่ 3 เพราะมี Levenshtein distance น้อยที่สุด

Once upon a time when the fake
hen the flakes of snow were falling

Use 'flakes' instead.

Use 'when' instead.

รูปที่ 3.15 ตัวอย่างการตัดแบ่งเสียงแบบ Overlapping Windows ขั้นตอนที่ 4

เมื่อมี conflict (มีคำที่แตกต่างกันในสองชุดข้อความ) เกิดขึ้น วิธีแก้ปัญหาคือ เราจะไม่เลือกคำที่ถูกตัด นั่นคือ คำสุดท้ายของชุดข้อความแรก และ คำแรกของชุดข้อความที่สอง แต่เราจะเลือกคำในตำแหน่งเดียวกันของอีกชุดข้อความแทน ดังในตัวอย่าง เราจะเลือกคำว่า flakes แทน fake และ เลือกคำว่า when แทน hen

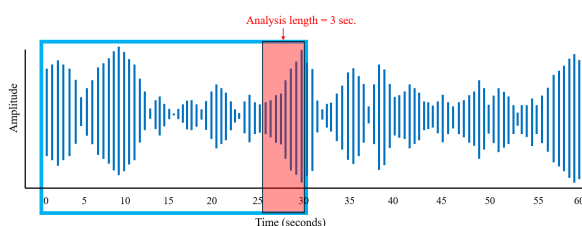
Result: “Once upon a time when the flakes when the flakes of snow were falling”

รูปที่ 3.16 ตัวอย่างการตัดแบ่งเสียงแบบ Overlapping Windows ขั้นตอนที่ 5

ดังนั้นผลลัพธ์ที่ได้จะเป็น “Once upon a time when the flakes of snow were falling” ซึ่งเป็นผลลัพธ์ที่ต้องการ

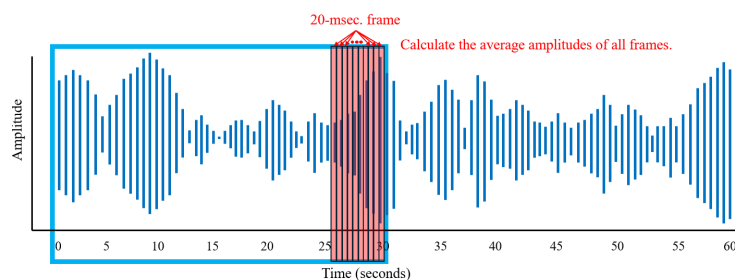
3.2.1.5 วิธีที่ 5 Minimum Average Amplitude

หลักการของ Minimum Average Amplitude จะใช้การ average amplitude ในแต่ละ frame เพื่อหาจุดที่มีค่า average amplitude ต่ำที่สุด และใช้เป็นจุดตัด โดยมีขั้นตอนดังนี้



รูปที่ 3.17 ตัวอย่างการตัดแบ่งเสียงแบบ Minimum Average Amplitude

ทำการแบ่ง window ออกเป็น 30 วินาทีและกำหนด analysis length ขนาด 3 วินาทีนับจากวินาทีสุดท้ายของ window ถอยเข้ามา 3 วินาทีจะได้ช่วงตามรูป



รูปที่ 3.18 ตัวอย่างการตัดแบ่งเสียงแบบ Minimum Average Amplitude

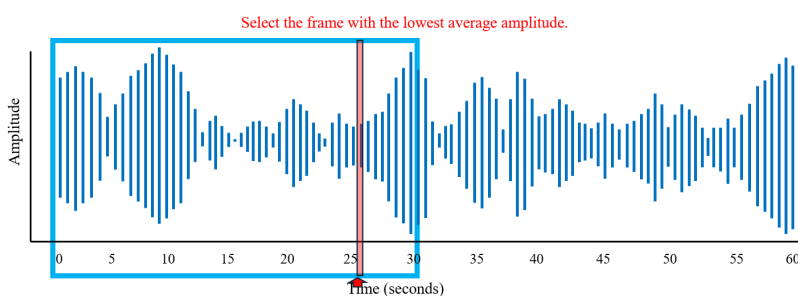
แบ่ง frame ในการคำนวณ average amplitude เป็น frame ละ 20ms และทำการคำนวณ average amplitude ในแต่ละ frame โดยมีสูตรการคำนวณดังนี้

$$E = \frac{1}{N} \sum_{i=1}^N |x_i| \quad (3.1)$$

E = average amplitude ใน frame นั้นๆ

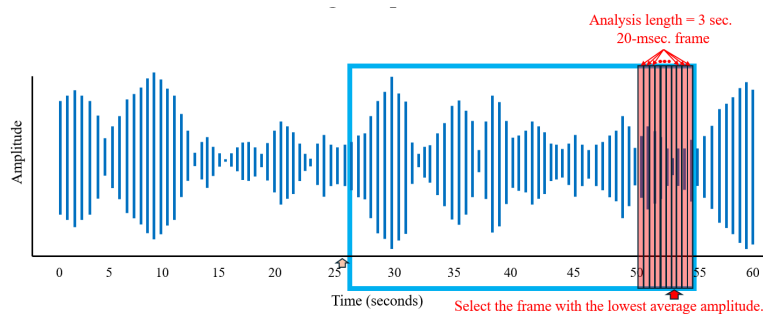
N = จำนวนตัวอย่างเสียงในช่วง 20 ms

X_i = ค่าความดังของตัวอย่างที่ i



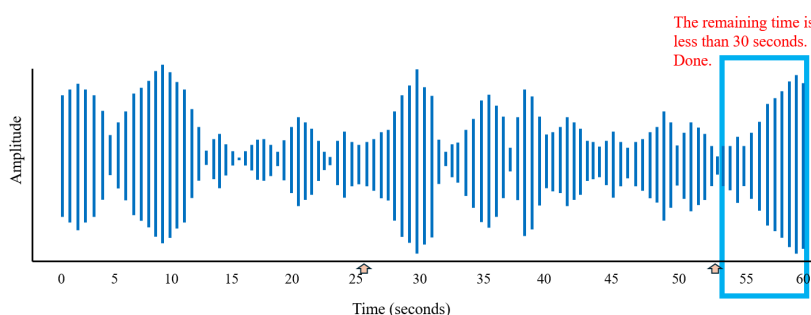
รูปที่ 3.19 ตัวอย่างการตัดแบ่งเสียงแบบ Minimum Average Amplitude

เลือก frame ที่มีค่า average amplitude ต่ำที่สุด และใช้เป็นจุดตัด



รูปที่ 3.20 ตัวอย่างการตัดแบ่งเสียงแบบ Minimum Average Amplitude

เลื่อนจุดเริ่มต้นของ window ไปที่จุดตัดต่ำสุด และเริ่มกระบวนการใหม่จนสิ้นสุดไฟล์เสียง



รูปที่ 3.21 ตัวอย่างการตัดแบ่งเสียงแบบ Minimum Average Amplitude

หากไฟล์เสียงช่วงสุดท้ายเหลือความยาวไม่ถึง 30 วินาทีหรือสั้นกว่าขนาด window จะเลือกใช้ไฟล์เสียงส่วนที่เหลือทั้งหมด

ข้อดีของ Minimum Average Amplitude คือ ลดจำนวนของ chunk และทำให้ chunk มีขนาดที่ไม่เล็เกินไป และสามารถคำนวณจุดตัดที่มี average amplitude ในช่วง analysis length ได้อย่างแม่นยำ

ข้อเสียของ Minimum Average Amplitude ช่วงที่ average amplitude ต่ำที่สุดอาจอยู่นอก analysis length

3.2.2 ดำเนินการตรวจสอบข้อมูลและทำความสะอาดข้อมูล

คำอธิบายเสียงที่ดาวน์โหลดจาก YouTube อาจมีบางคำหรือบางประโยคที่ไม่สอดคล้องกับเสียงจริง จึงจำเป็นต้องปรับแก้เพื่อให้เนื้อหาตรงกับเสียงมากที่สุด นอกจากนี้ ยังต้องกรองอักขระที่ไม่จำเป็น เช่น สัญลักษณ์พิเศษ เพื่อรักษาเฉพาะข้อมูลสำคัญที่มีประโยชน์ต่อการตรวจสอบประสิทธิภาพของการแบ่งส่วนเสียง

3.2.3 ดำเนินการพัฒนา Algorithm การแบ่งส่วนเสียง

ในขั้นตอนนี้ ได้มีการพัฒนาอัลกอริทึมสำหรับการแบ่งส่วนเสียง ซึ่งประกอบด้วยวิธี Sliding Windows, Slide and Split, Overlapping Windows และ Minimum Average Amplitude โดยใช้ภาษา Python เนื่องจากโค้ดที่ใช้เรียกโมเดล ASR เขียนด้วยภาษา Python การใช้ภาษาเดียวกันสำหรับการพัฒนาทำให้สามารถนำมาใช้งานร่วมกันได้อย่างสะดวกและไม่ซับซ้อน

3.2.4 การพัฒนา Algorithm ให้สามารถนำไปใช้งานกับโมเดล

การนำอัลกอริทึมการแบ่งเสียงไปใช้ร่วมกับโมเดล ASR เริ่มต้นโดยการเรียกใช้อัลกอริทึมการแบ่งเสียงก่อน จากนั้นจึงนำเสียงที่แบ่งได้ไปผ่านกระบวนการประมวลผลด้วยโมเดล ASR เพื่อให้ได้ผลลัพธ์เป็นข้อความที่เป็นตัวอักษร

ในขั้นตอนแรกของการทำงาน โปรแกรมจะทำการโหลดข้อมูลเสียงและคำอธิบาย จากนั้นจึงกรองข้อมูลคำอธิบายให้เหลือเฉพาะตัวอักษรที่จำเป็นสำหรับการตรวจสอบค่า Word Error Rate (WER) และ Character Error Rate (CER) เมื่อกรองข้อมูลเสร็จแล้ว จะนำเสียงไปผ่านกระบวนการ Voice Activity Detection (VAD) เพื่อคัดกรองเสียงพูดที่ต้องการ จากนั้นจึงนำเสียงที่ผ่านการคัดกรองไปใช้กับอัลกอริทึมการแบ่งเสียงแบบต่าง ๆ ที่ได้กล่าวไว้ข้างต้น เพื่อแบ่งเสียงออกเป็นส่วน ๆ

เมื่อได้เสียงที่แบ่งออกเป็นส่วน ๆ แล้ว จะนำเข้าสู่กระบวนการประมวลผลด้วยโมเดล ASR เพื่อให้โมเดลทำการทำนายว่าเสียงเหล่านี้มีเนื้อหาคำพูดว่าอย่างไร สุดท้ายเมื่อได้ผลลัพธ์ในรูปแบบข้อความจากโมเดล ASR จะนำข้อความที่ได้มาเปรียบเทียบกับข้อความต้นฉบับในคำอธิบาย เพื่อคำนวณค่า WER และ CER

3.2.5 การทดสอบประสิทธิภาพการตัดแบ่งเสียงโดยทดสอบผ่านโมเดล

ในการทดสอบประสิทธิภาพของโมเดล จะใช้ตัวชี้วัด Word Error Rate (WER) และ Character Error Rate (CER) เพื่อประเมินความแม่นยำ โดยจะทำการทดสอบทั้งหมด 5 รอบ ซึ่งแต่ละรอบจะใช้วิธีการแบ่งเสียงที่แตกต่างกันตามอัลกอริทึมที่มีอยู่ ขั้นตอนคือการนำข้อความที่ได้จากโมเดล ASR ของแต่ละวิธีการแบ่งเสียงไปเปรียบเทียบกับข้อความต้นฉบับที่ใช้เป็นข้อมูลอ้างอิง เพื่อคำนวณค่า WER และ CER ของแต่ละอัลกอริทึม

3.2.6 จัดทำเอกสารและสรุปผลการทดลอง

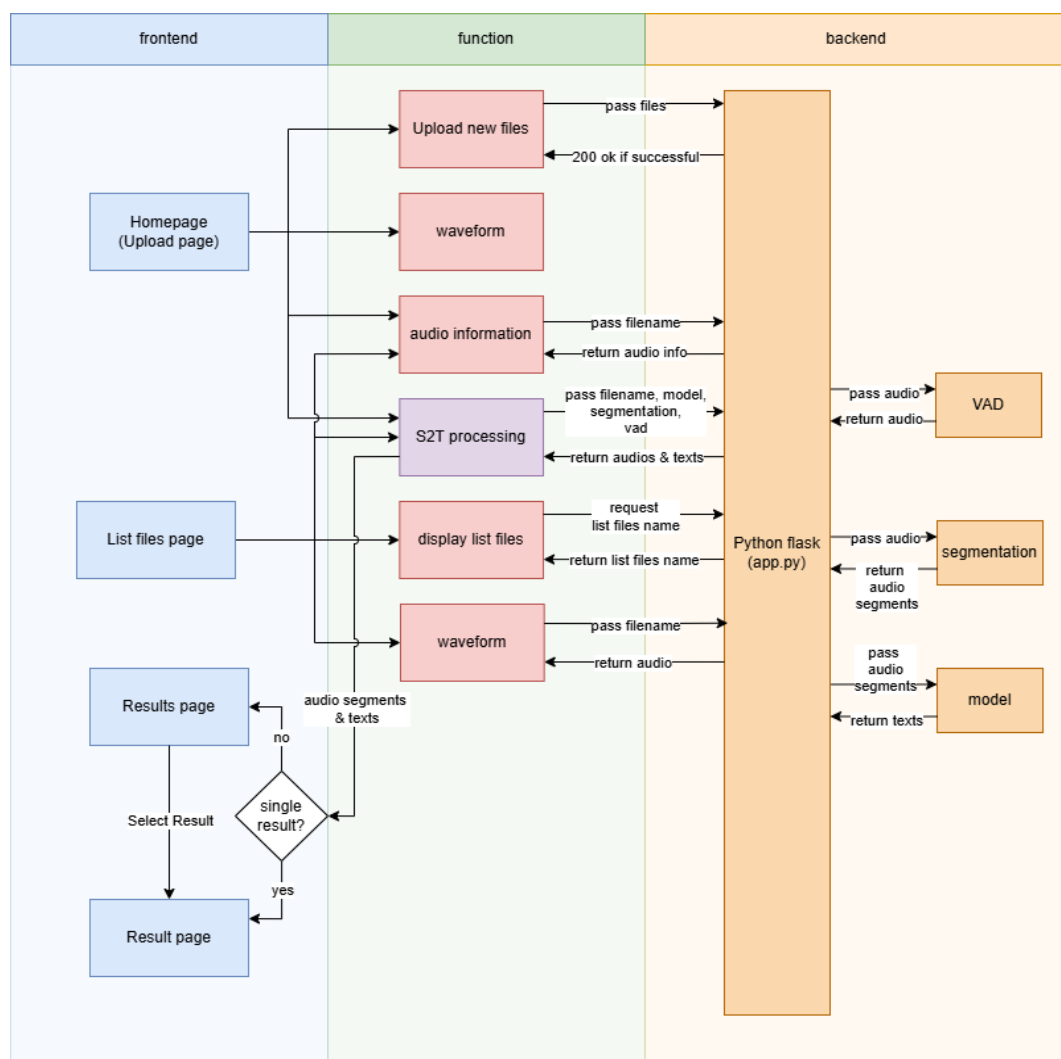
ผลการทดสอบข้างต้นจะถูกนำมาวิเคราะห์และเปรียบเทียบกันสำหรับทุกโมเดลที่ใช้ในการทดลอง เพื่อประเมินประสิทธิภาพการทำงานในแต่ละกรณี

3.3 การออกแบบและพัฒนาเว็บแอปพลิเคชัน

เป็นการออกแบบเว็บแอปพลิเคชันเพื่อช่วยในการตรวจสอบผลลัพธ์และทดสอบประสิทธิภาพการถอดความของโมเดลหลังจากที่ผ่านการแบ่งส่วนเสียงด้วยอัลกอริทึมที่พัฒนาขึ้น แอปพลิเคชันนี้จะสามารถแสดงข้อมูล เช่น รูปคลื่นเสียง (Waveform), ค่า Word Error Rate (WER) ของแต่ละไฟล์เสียง และค่า WER รวมของแต่ละวิธีการแบ่งเสียง

3.3.1 โครงสร้างสถาปัตยกรรมของเว็บแอปพลิเคชัน

การพัฒนาเว็บแอปพลิเคชันเพื่อทดสอบประสิทธิภาพของโมเดลจะใช้ Vue.js ในการพัฒนา Frontend และ Python Flask สำหรับ Backend โดยมีโครงสร้างดังนี้



รูปที่ 3.22 โครงสร้างสถาปัตยกรรมของเว็บแอปพลิเคชัน

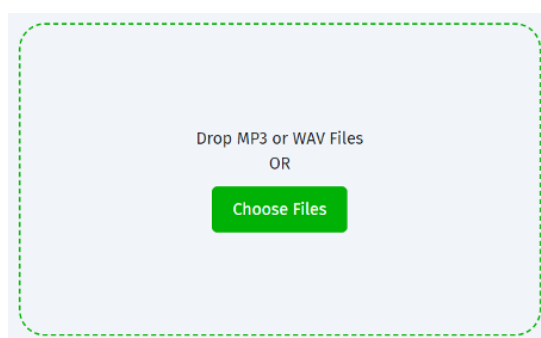
3.3.2 การพัฒนาหน้าเว็บแอปพลิเคชัน

3.3.2.1 หน้า Home



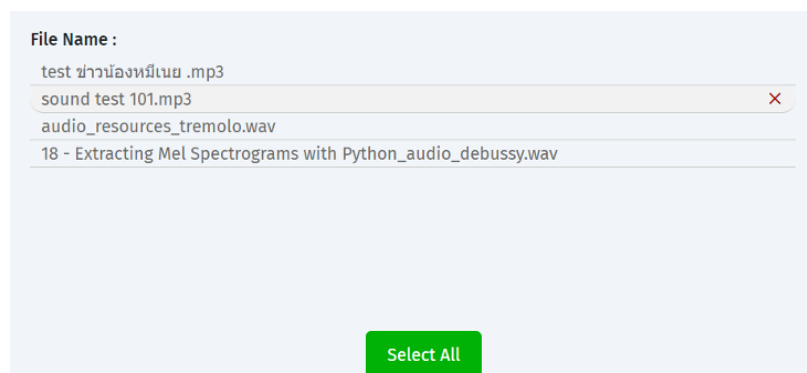
รูปที่ 3.23 เว็บแอปพลิเคชันหน้า Home

เป็นหน้าสำหรับการวางไฟล์เสียงที่ต้องการอัปโหลดเสียงเข้ามาใน
เว็บไซต์



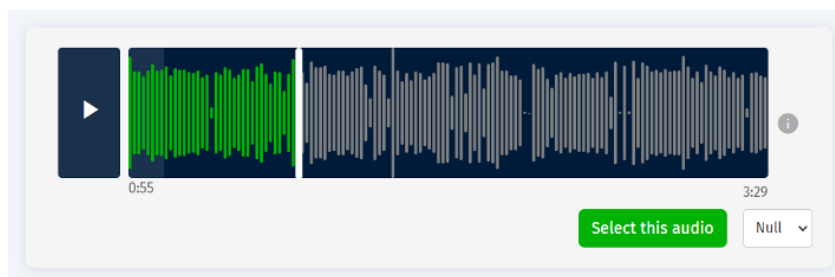
รูปที่ 3.24 เว็บแอปพลิเคชันส่วน DropZone

ส่วน DropZone สำหรับอัปโหลดไฟล์เสียงนามสกุล .mp3 และ .wav เพื่อ
นำไปถอดความจากไฟล์เสียง



รูปที่ 3.25 เว็บแอปพลิเคชันส่วน File name

ส่วน File name สำหรับแสดงชื่อไฟล์ทั้งหมดที่จะนำไปถอดความจากไฟล์เสียง สามารถกดที่ชื่อไฟล์เสียงเพื่อพรีวิว waveform เสียงที่เลือก, กดปุ่มกากบาทหลังบรรทัดเพื่อยกเลิกการอัปโหลด และ ปุ่ม Select All เพื่อถอดความจากไฟล์เสียงทุกไฟล์ที่มีชื่ออยู่ใน File name



รูปที่ 3.26 เว็บแอปพลิเคชันส่วน Waveform

ส่วน Waveform ประกอบด้วยปุ่ม play, ปุ่ม pause สำหรับใช้เล่นเสียงจากไฟล์ที่เลือก, โชน waveform สำหรับดูระดับเสียง ณ เวลานั้น ๆ รองรับระบบ Trim Crop เสียง และ ปุ่ม info



รูปที่ 3.27 ตัวอย่างการ Trim ของ Waveform

การ Trim จะสามารถเลือกช่วงที่เราไม่ต้องการ และไม่นำช่วงเสียงนั้นเข้าสู่การถอดความ



รูปที่ 3.28 ตัวอย่างการ Crop ของ Waveform

การ Crop จะเป็นการเลือกช่วงไฟล์เสียงที่เราต้องการ เข้าสู่การถอดความ



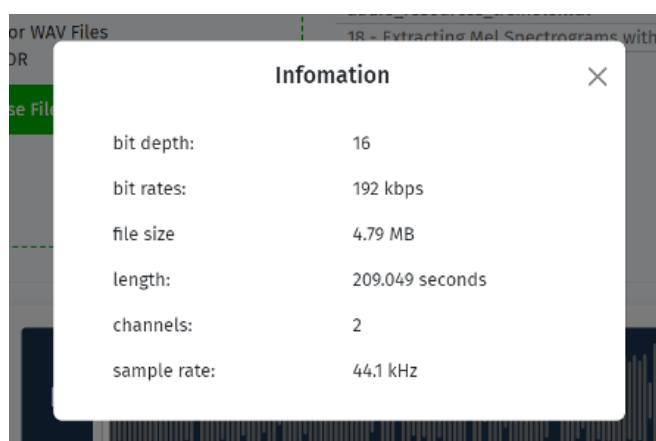
รูปที่ 3.29 ตัวอย่างการ Reset Region

การ Trim และ Crop จะมีปุ่ม Reset Region เพื่อเป็นการรีเซ็ต การ Trim และ การ Crop ออกไป



รูปที่ 3.30 ปุ่ม info ของเสียงที่กำลังแสดงอยู่

ปุ่ม info จะแสดง Pop-up ข้อมูลของเสียง



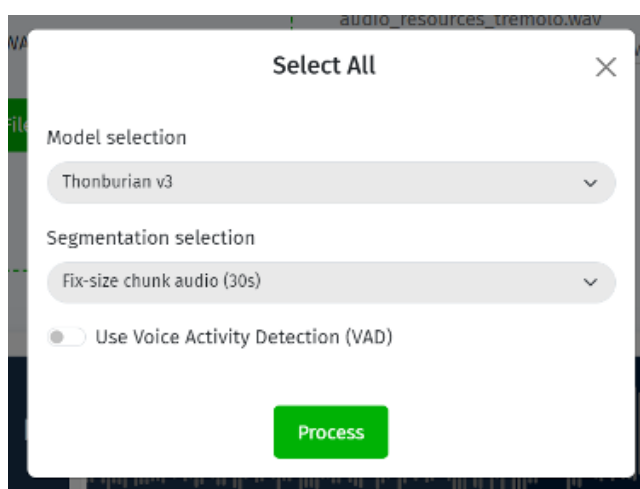
รูปที่ 3.31 ตัวอย่าง Pop-up ข้อมูลของไฟล์เสียง

Pop-up หลังจากกดปุ่ม info จะแสดงข้อมูลของเสียง ได้แก่ bit depth, bit rates, file size, length, channels และ sample rate



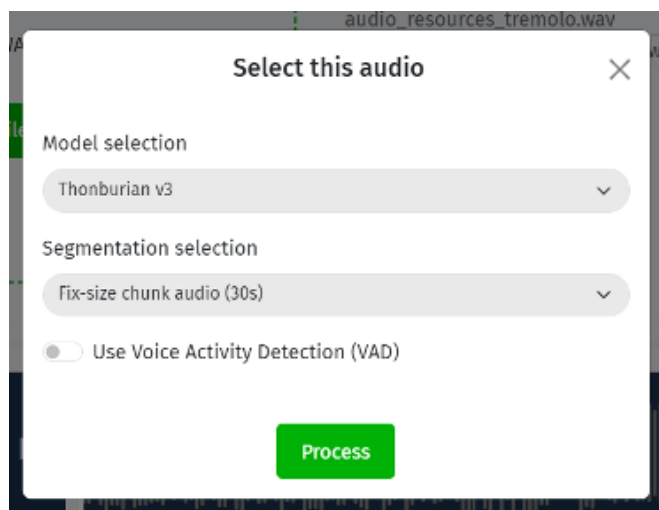
รูปที่ 3.32 ปุ่ม Select All และ ปุ่ม Select this audio

ปุ่ม Select All และ Select this audio กดเพื่อแสดง Pop-up โดยปุ่ม Select All จะเป็นการเลือกไฟล์เสียงทั้งหมด เข้าสู่การ Process ต่อไป และ ปุ่ม Select this audio จะเป็นการเลือกไฟล์ที่กำลังแสดง waveform อยู่เข้าสู่การ Process ต่อไป



รูปที่ 3.33 ตัวอย่าง Pop-up การกด Select All

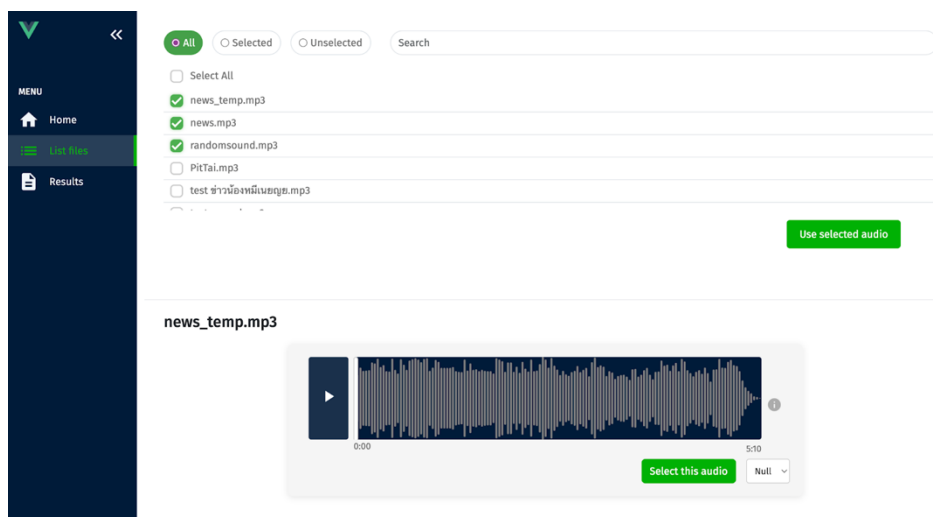
Pop-up หลังจากกดปุ่ม Select All จะแสดง Pop-up สำหรับเลือกโมเดล ถอดความจากไฟล์เสียง วิธีการทำ Segmentation และต้องการใช้ Voice Activity Detection (VAD) หรือไม่ โดยเมื่อกดปุ่ม Process จะเป็นการเลือกไฟล์เสียงทั้งหมดที่ใส่เข้ามา



รูปที่ 3.34 ตัวอย่าง Pop-up การกด Select this audio

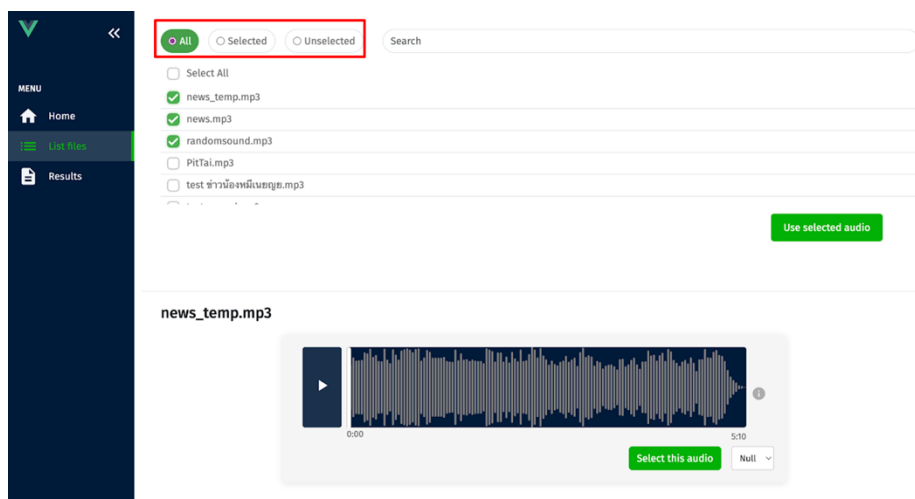
Pop-up หลังจากกดปุ่ม Select this audio จะแสดง Pop-up สำหรับเลือกโมเดลถอดความจากไฟล์เสียง วิธีการทำ Segmentation และต้องการใช้ Voice Activity Detection (VAD) หรือไม่ โดยเมื่อกดปุ่ม Process จะเป็นการเลือกไฟล์เสียงที่กำลังแสดง Waveform ให้เห็น

3.3.2.2 หน้า List File



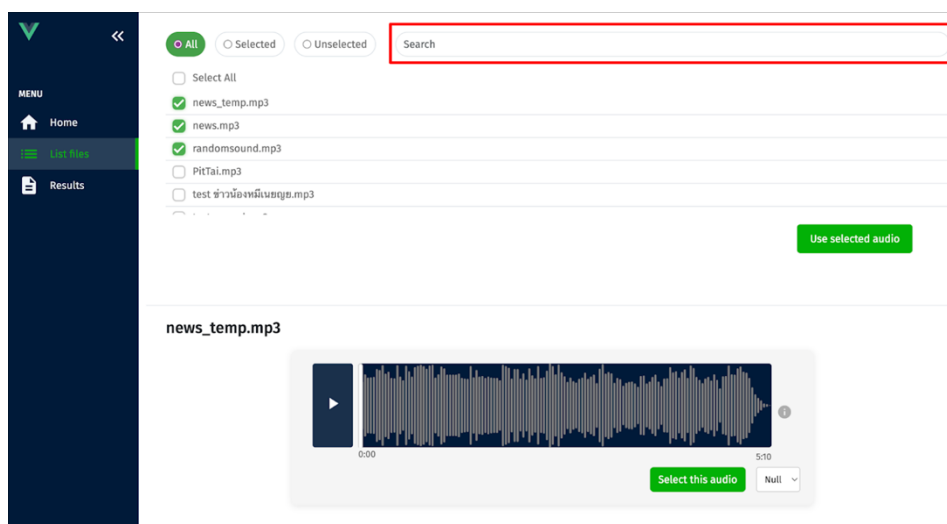
รูปที่ 3.35 เว็บแอปพลิเคชันหน้า List File

เป็นหน้าที่จะแสดงชื่อไฟล์ที่ผ่านการอัปโหลดเข้ามาในเว็บไซด์แล้ว และจะนำไปผ่านการถอดความไฟล์เสียงต่อไป



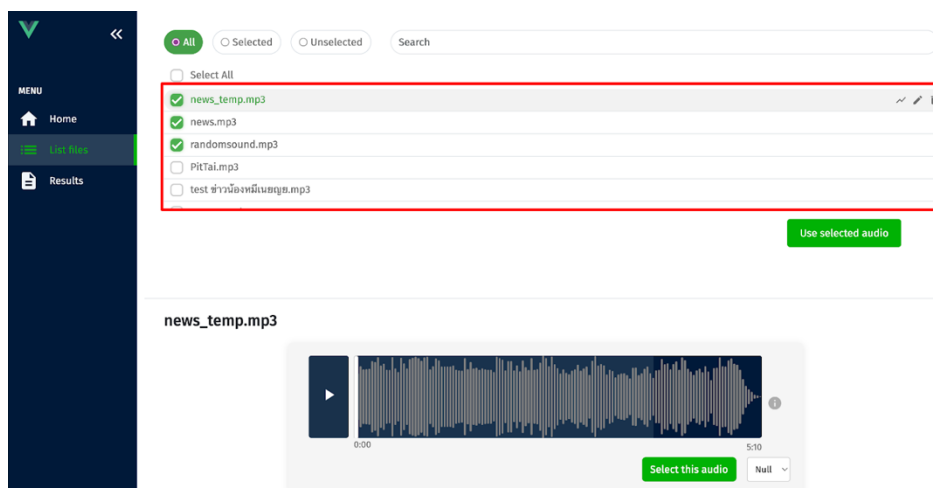
รูปที่ 3.36 เว็บแอปพลิเคชันส่วน filter ข้อมูลไฟล์

filter ประกอบไปด้วย 3 ส่วน ได้แก่ ไฟล์ทั้งหมดภายใน web server ,ไฟล์
ที่เลือก และ ไฟล์ที่ยังไม่เลือก



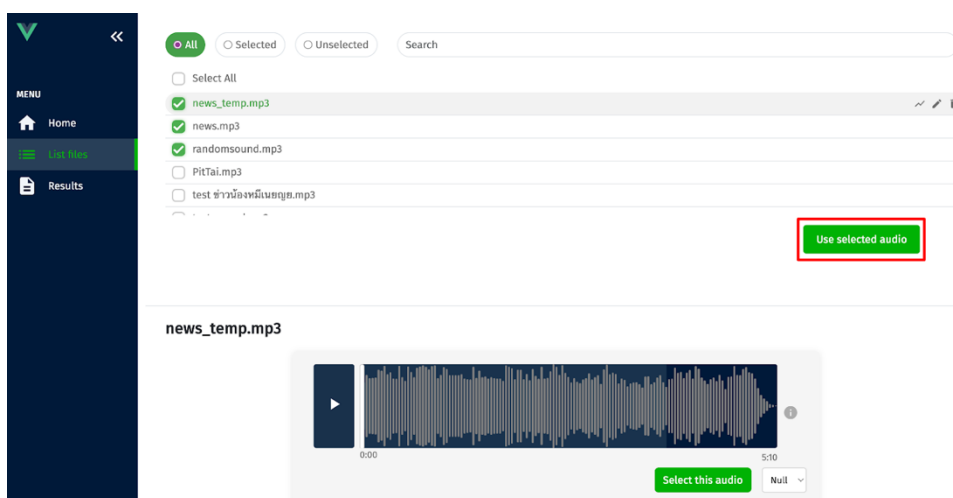
รูปที่ 3.37 เว็บแอปพลิเคชันส่วน Search ข้อมูลไฟล์

Search ใช้งานชื่อไฟล์ภายใน web server



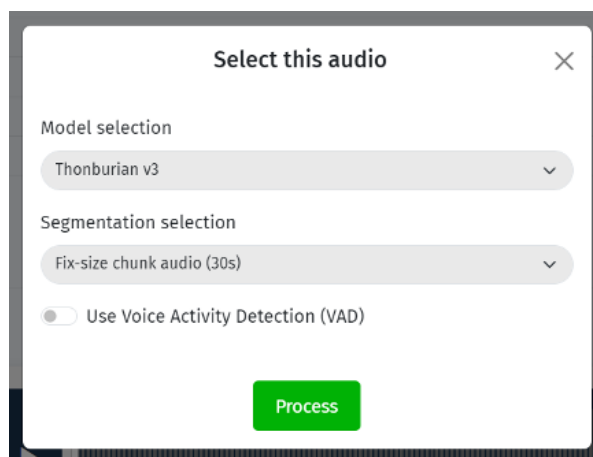
รูปที่ 3.38 เว็บแอปพลิเคชันส่วนรายชื่อไฟล์

รายชื่อไฟล์ มีรายชื่อไฟล์ทั้งหมดตรงตาม filter และปุ่ม Use select audio สำหรับเลือกทุกไฟล์ และ option เพิ่มเติม ได้แก่ การเรียกดู waveform ,การแก้ไขชื่อไฟล์ และ การลบไฟล์ออกจาก web server



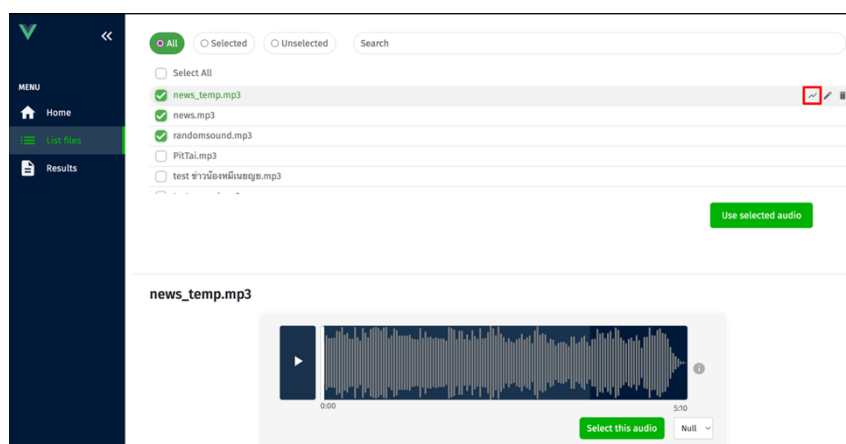
รูปที่ 3.39 ปุ่ม Use selected audio

ปุ่ม Use select audio กดเพื่อแสดง Pop-up



รูปที่ 3.40 ตัวอย่าง Pop-up หลังกดปุ่ม Use select audio

Pop-up หลังกดปุ่ม Use select audio จะแสดง Pop-up สำหรับเลือกโมเดลถอดความจากไฟล์เสียง วิธีการทำ Segmentation และต้องการใช้ Voice Activity Detection (VAD) หรือไม่ โดยเมื่อกดปุ่ม Process จะเป็นการเลือกไฟล์เสียงที่กด Select ไว้



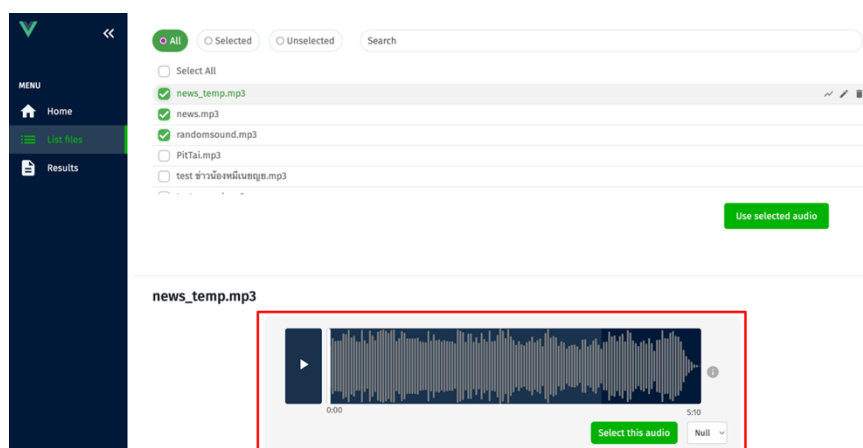
รูปที่ 3.41 ปุ่มสำหรับแสดง Waveform เสียงนั้น ๆ

กดที่ icon ดังรูป จะเป็นการเรียกดู Waveform



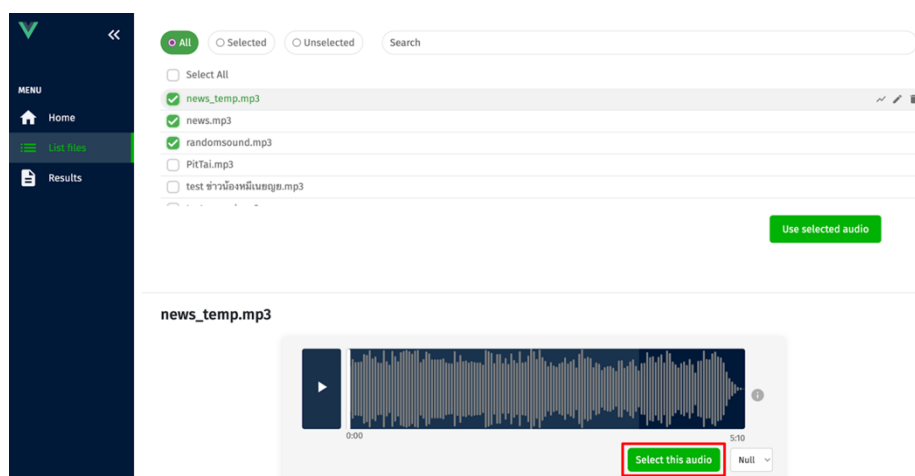
รูปที่ 3.42 หน้าต่าง Loading

หน้า Loading เป็นหน้าที่ขึ้นชั่วคราวสำหรับการรอหลังจากต้องเรียก waveform ขึ้นมาแสดง



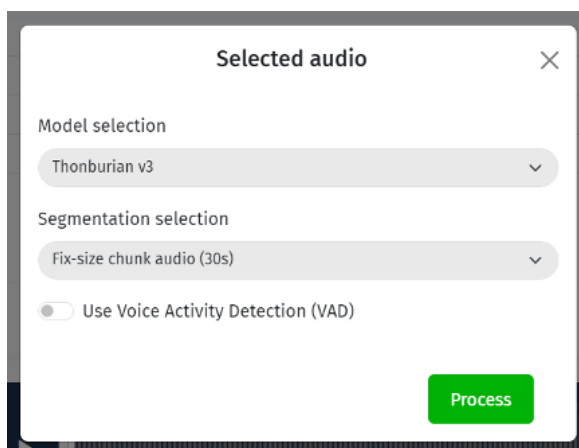
รูปที่ 3.43 เว็บแอปพลิเคชันส่วน Waveform ของหน้า List File

ส่วน Waveform จะทำการพรีวิว Waveform ไฟล์เสียงที่เลือก โดยจะสามารถทำฟังก์ชันต่าง ๆ ได้เหมือนกับในหน้า Home



รูปที่ 3.44 ปุ่ม Select this audio

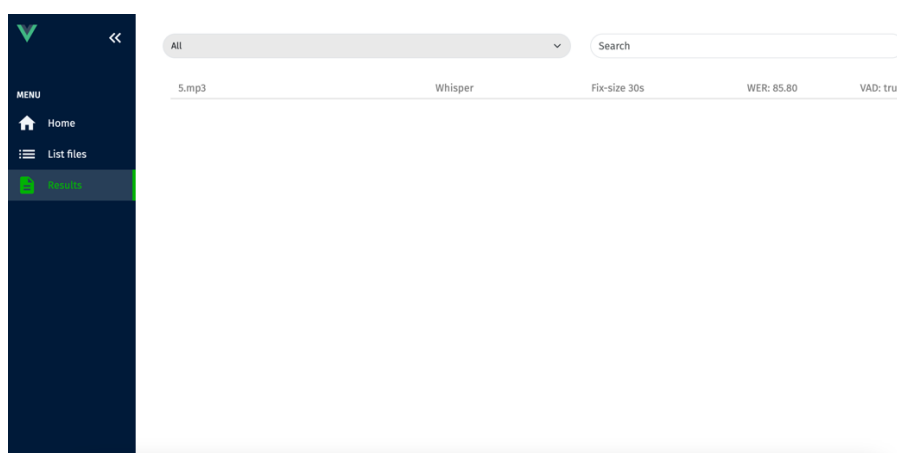
ปุ่ม select this audio กดเพื่อแสดง Pop-up



รูปที่ 3.45 ตัวอย่าง Pop-up หลังกดปุ่ม Select this audio

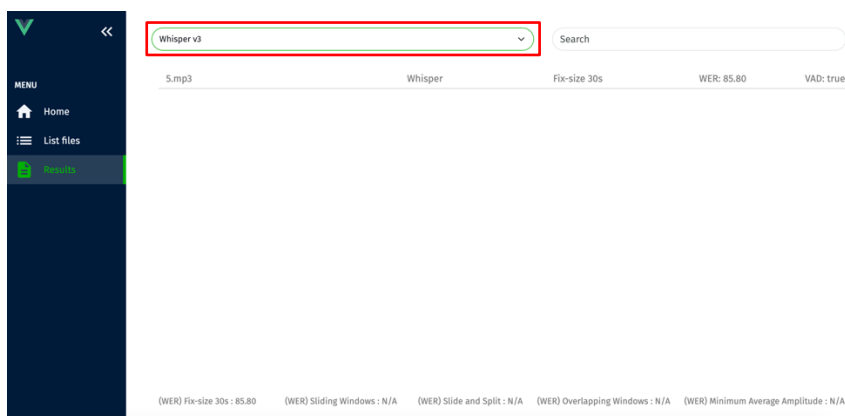
Pop-up หลังกดปุ่ม Select this audio จะแสดง Pop-up สำหรับเลือกโมเดลถอดความจากไฟล์เสียง วิธีการทำ Segmentation และต้องการใช้ Voice Activity Detection (VAD) หรือไม่ โดยเมื่อกดปุ่ม Process จะเป็นการเลือกไฟล์เสียงที่กด Select ไว้

3.3.2.3 หน้า Result



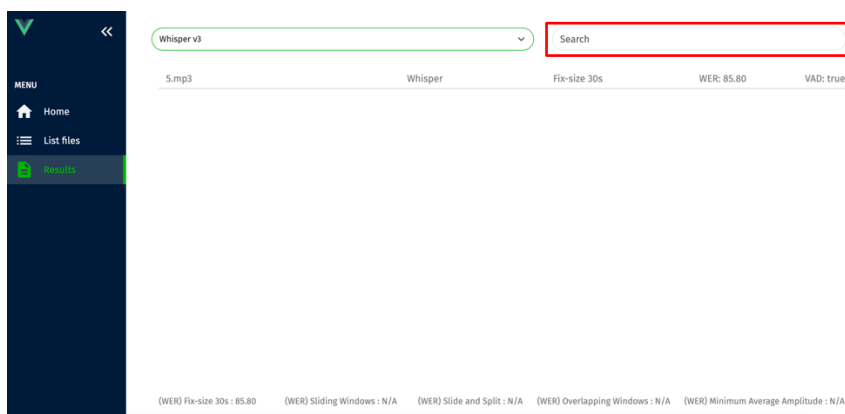
รูปที่ 3.46 เว็บแอปพลิเคชันหน้า Result

เป็นหน้าแสดงรายชื่อไฟล์ทั้งหมดที่เคยผ่านการถอดความจากไฟล์เสียงแล้ว และสามารถดูผลลัพธ์เดิมของไฟล์นั้น ๆ ได้ มีรายละเอียดดังนี้



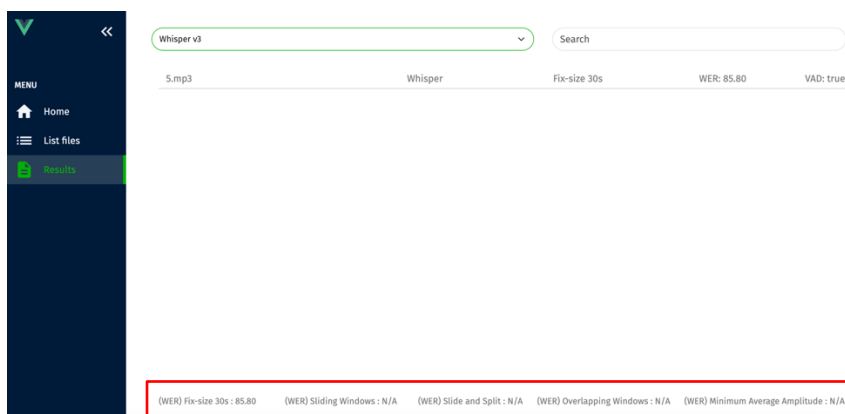
รูปที่ 3.47 เว็บแอปพลิเคชันหน้า Result ส่วน filter ข้อมูลไฟล์

ใช้คัดกรองไฟล์ที่เคยที่ได้รับการถอดความจากโมเดลออกมาแล้ว โดยคัดกรองจากการถอดความด้วยโมเดลชนิดนั้นๆ



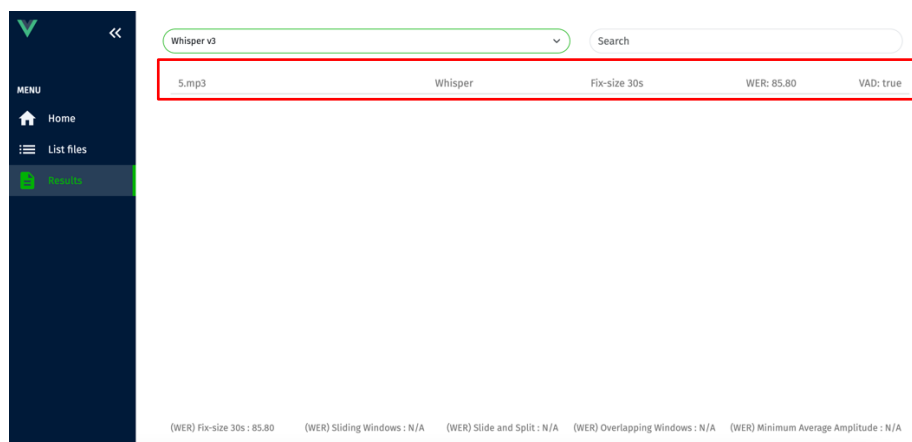
รูปที่ 3.48 เว็บแอปพลิเคชันหน้า Result ส่วน Search ข้อมูลไฟล์

ใช้หาไฟล์ที่เคยที่ได้รับการถอดความจากโมเดลออกมาแล้ว



รูปที่ 3.49 เว็บแอปพลิเคชันหน้า Result ส่วนการแสดงค่าเฉลี่ยของ WER

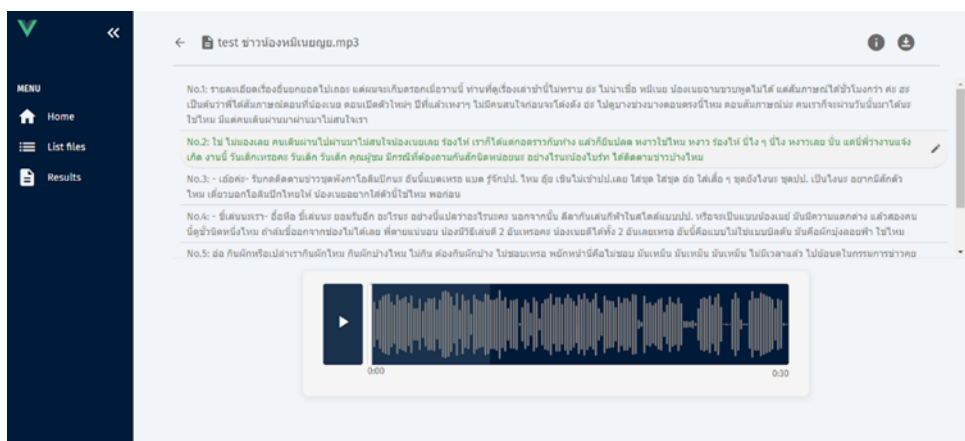
ใช้สำหรับแสดงค่าเฉลี่ยของ WER ของแต่ละ Segmentation โดยถ้าทำการถอดความจากไฟล์เสียงหลายไฟล์ ค่าที่แสดงจะเป็นค่าเฉลี่ย หากไม่มี จะแสดงเป็น N/A



รูปที่ 3.50 เว็บแอปพลิเคชันหน้า Result ส่วนรายชื่อไฟล์

รายชื่อไฟล์ทั้งหมดที่เคยผ่านการถอดความจากไฟล์เสียงแล้ว และสามารถผลลัพธ์ของไฟล์นั้น ๆ ได้ (File result details)

3.3.2.4 หน้า File result details



รูปที่ 3.51 เว็บแอปพลิเคชันหน้า File result details

เป็นหน้าแสดงข้อมูล text ทั้งหมดที่ผ่านจากการประมวลผลและได้รับ
การถอดความเสียงจากโมเดล



รูปที่ 3.52 เว็บแอปพลิเคชันส่วนข้อความที่ได้รับการถอดความ

ส่วนของ output สำหรับข้อความที่ได้จากการถอดความจากไฟล์เสียง สามารถคลิกเพื่อฟังเสียง เพื่อดูว่าข้อความตรงกับเสียงหรือไม่

3.3.3 การออกแบบ API ของเว็บแอปพลิเคชัน

3.3.3.1 UploadAudioAPI (POST: /upload_files)

API นี้ใช้สำหรับอัปโหลดไฟล์เสียงไปยังเซิร์ฟเวอร์ ทั้งแบบไฟล์เดียวและหลายไฟล์ เพื่อนำไปประมวลต่อ

- 1) Request ส่งไฟล์เสียงในรูปแบบ multipart/form-data โดยมีไฟล์เสียงที่อนุญาตคือ wav, mp3, ogg

ตัวอย่าง 3.1 การร้องขอสำหรับอัปโหลดไฟล์เสียง

```
{
  "audio_transcriptions": [
    {
      "audio": "base64_encoded_audio_data",
      "transcription": "transcribed text"
    }
  ]
}
```

2) Response

- หากสำเร็จ

ตัวอย่าง 3.2 การตอบกลับเมื่ออัปโหลดสำเร็จ

```
{
  "message": "File uploaded successfully"
}
```

- หากไม่สำเร็จ

ตัวอย่าง 3.3 การตอบกลับเมื่อไม่สามารถอัปโหลดได้

```
{
  "error": "Invalid file type"
}
```

3.3.3.2 ListAudioFilesAPI (GET: /ListAudioFiles)

API นี้แสดงรายการไฟล์เสียงทั้งหมดในเซิร์ฟเวอร์ที่อัปโหลดไว้

- 1) Request ไม่ต้องใช้ข้อมูลเพิ่มเติม
- 2) Response รายการชื่อไฟล์เสียงทั้งหมด

ตัวอย่าง 3.4 การร้องขอสำหรับการแสดงไฟล์เสียงทั้งหมดที่อัปโหลดไว้

```
[
  "file1.wav",
  "file2.mp3"
]
```

3.3.3.3 DeleteAudioAPI (DELETE: /delete_audio)

API นี้ใช้สำหรับลบไฟล์เสียงในเซิร์ฟเวอร์ที่ต้องการRequest ไม่ต้องใช้ข้อมูลเพิ่มเติม

- 1) Request ต้องส่งชื่อไฟล์ที่จะลบในรูปแบบ JSON

ตัวอย่าง 3.5 การร้องขอสำหรับลบไฟล์เสียงในเซิร์ฟเวอร์ที่ต้องการ

```
{
  "filename": "file1.wav"
}
```

2) Response

- หากลบสำเร็จ

ตัวอย่าง 3.6 การตอบกลับเมื่อลบไฟล์เสียงสำเร็จ

```
{
  "message": "File file1.wav deleted successfully"
}
```

- หากหาไฟล์ไม่เจอ

ตัวอย่าง 3.7 การตอบกลับเมื่อไม่มีไฟล์เสียงที่ต้องการลบ

```
{
  "error": "File not found"
}
```

3.3.3.4 UpdateAudioNameAPI (PUT: /update_name_audio)

API นี้ใช้สำหรับเปลี่ยนชื่อไฟล์เสียงที่มีอยู่ในเซฟเวอร์

- 1) Request ส่งชื่อไฟล์เก่าและชื่อไฟล์ใหม่ในรูปแบบ JSON

ตัวอย่าง 3.8 การร้องขอสำหรับการเปลี่ยนชื่อไฟล์เสียง

```
{
  "old_filename": "file1.wav",
  "new_filename": "newfile.wav"
}
```

2) Response

- หากเปลี่ยนชื่อสำเร็จ

ตัวอย่าง 3.9 การตอบกลับเมื่อเปลี่ยนชื่อสำเร็จ

```
{
  "message": "File renamed from file1.wav to
newfile.wav successfully"
}
```


- หากหาไฟล์ไม่เจอ

ตัวอย่าง 3.10 ตอบกลับเมื่อไม่มีไฟล์เสียงที่ต้องการเปลี่ยนชื่อ

```
{
  "error": "File not found"
}
```

3.3.3.5 GetAudioFileAPI (POST: /get_audio)

API นี้ใช้สำหรับดึงข้อมูลไฟล์เสียงในรูปแบบ Base64

- 1) Request ส่งชื่อไฟล์ที่ต้องการดึงในรูปแบบ JSON

ตัวอย่าง 3.11 ร้องขอสำหรับดึงข้อมูลไฟล์เสียงในรูปแบบ base64

```
{
  "filename": "file1.wav"
}
```

- 2) Response

- หากเจอไฟล์

ตัวอย่าง 3.12 ตอบกลับเมื่อดึงข้อมูลสำเร็จ

```
{
  "filename": "file1.wav",
  "content": "Base64_encoded_string",
  "content_type": "audio/wav"
}
```

- หากหาไฟล์ไม่เจอ

ตัวอย่าง 3.13 การตอบกลับเมื่อไม่มีไฟล์เสียงที่ต้องการดึงข้อมูล

```
{
  "error": "File not found"
}
```

3.3.3.6 SpeechToTextAPI (POST: /stt)

API นี้ใช้สำหรับการแปลงไฟล์เสียงเป็นข้อความโดยใช้โมเดลต่าง ๆ

Request ส่งชื่อไฟล์ที่ต้องการดึงในรูปแบบ JSON

- 1) Request ส่งข้อมูลไฟล์เสียง, โมเดลที่ต้องการใช้, วิธีการ segment และ การใช้ Voice Activity Detection (VAD) ในรูปแบบ JSON

ตัวอย่าง 3.14 การร้องขอสำหรับแปลงไฟล์เสียงเป็นข้อความโดยใช้โมเดลต่าง ๆ

```
{
  "file_paths": ["file1.wav", "file2.mp3"],
  "model": "whisper",
  "segmentation": "method_name",
  "vad": true,
  "start_time": 1000,
  "end_time": 5000,
  "cut_method": "crop"
}
```

- 2) Response ส่งผลการแปลงข้อความและไฟล์เสียงที่ถูกประมวลผลแล้ว

ตัวอย่าง 3.15 การตอบกลับเมื่อแปลงไฟล์เสียงเป็นข้อความสำเร็จ

```
{
  "results": [
    {
      "file_path": "file1.wav",
      "text": "Transcribed text",
      "model_name": "whisper",
      "segmentation_method": "method_name",
      "use_vad": true,
      "wer": wer
    }
  ],
  "audio_files": [
    {
      "file_format": "wav",
      "content": ["Base64_encoded_audio_chunk_1",
        "Base64_encoded_audio_chunk_2"]
    }
  ]
}
```

3.3.3.7 AudioInfoAPI (POST: /audio_info)

API นี้ใช้เพื่อดึงข้อมูลเกี่ยวกับไฟล์เสียง เช่น ความถี่ตัวอย่าง, ขนาดไฟล์, ความยาวไฟล์ เป็นต้น

- 1) Request ส่งชื่อไฟล์เสียงที่ต้องการในรูปแบบ JSON

ตัวอย่าง 3.16 การร้องขอสำหรับดึงข้อมูลของไฟล์เสียง

```
{  
  "file_path": "file1.wav"  
}
```

2) Response ส่งข้อมูลไฟล์เสียง**ตัวอย่าง 3.17 การตอบกลับเมื่อดึงข้อมูลไฟล์เสียงสำเร็จ**

```
{  
  "num_channels": "2",  
  "sampling_rate": "44.1 kHz",  
  "bitrate": "128 kbps",  
  "bit_depth": "16",  
  "file_size": "2.34 MB",  
  "length": "180.0 seconds"  
}
```

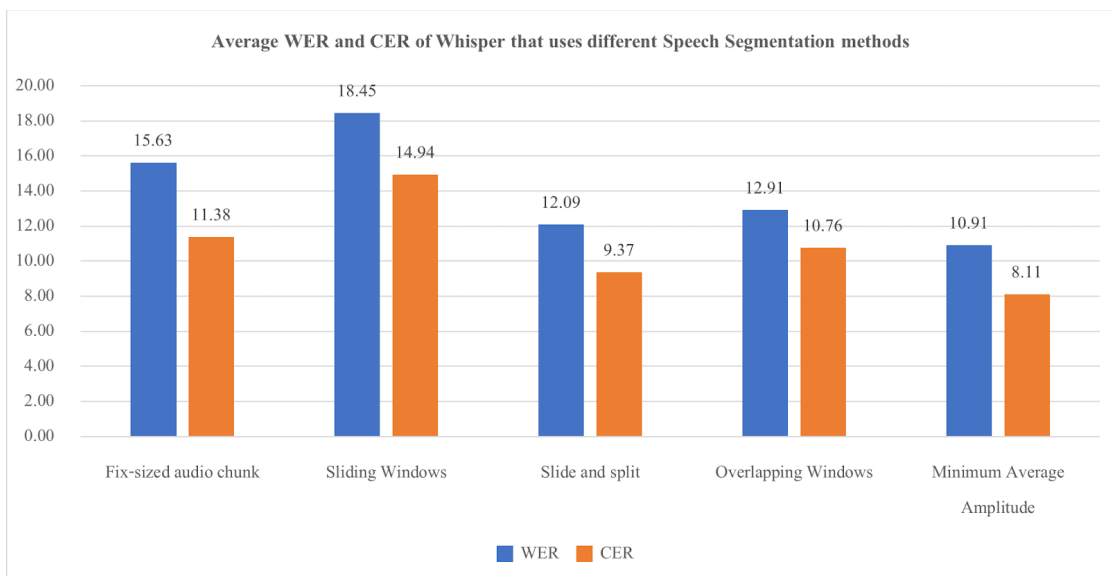
บทที่ 4

ผลการดำเนินงาน

จากการทดลองเพื่อเพิ่มประสิทธิภาพของโมเดล ASR ผ่านการพัฒนาเทคนิคการแบ่งส่วนเสียง ได้มีการคิดค้นและพัฒนาวิธีการแบ่งส่วนเสียงใหม่ทั้งหมด 4 วิธี และนำไปเปรียบเทียบกับวิธีการที่ใช้อยู่ในปัจจุบัน การทดลองนี้วัดประสิทธิภาพของโมเดล ASR ด้วยค่าตัวชี้วัด Word Error Rate (WER) และ Character Error Rate (CER) ซึ่งผลการทดลองแสดง ดังนี้

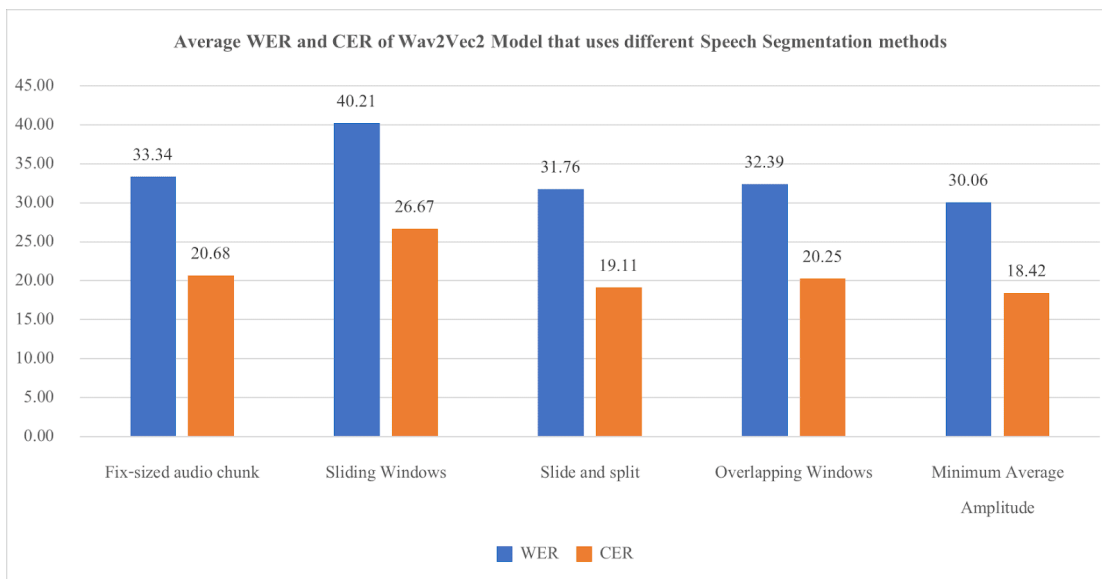
4.1 ประสิทธิภาพของโมเดลจากค่า WER และ CER

4.1.1 Whisper



ตารางที่ 4.1 ตารางเปรียบเทียบประสิทธิภาพการถอดความของโมเดล Whisper ด้วยวิธีการตัดแบ่งเสียงวิธีต่าง ๆ

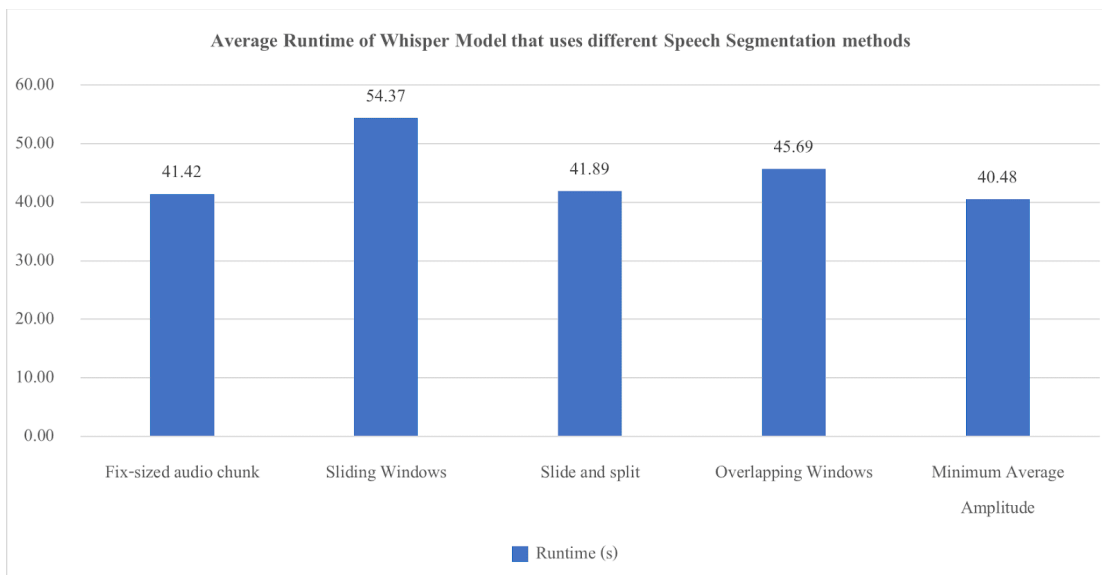
4.1.2 Wav2Vec2



ตารางที่ 4.2 ตารางเปรียบเทียบประสิทธิภาพการถอดความของโมเดล Wav2Vec2 ด้วยวิธีการตัดแบ่งเสียงวิธีต่าง ๆ

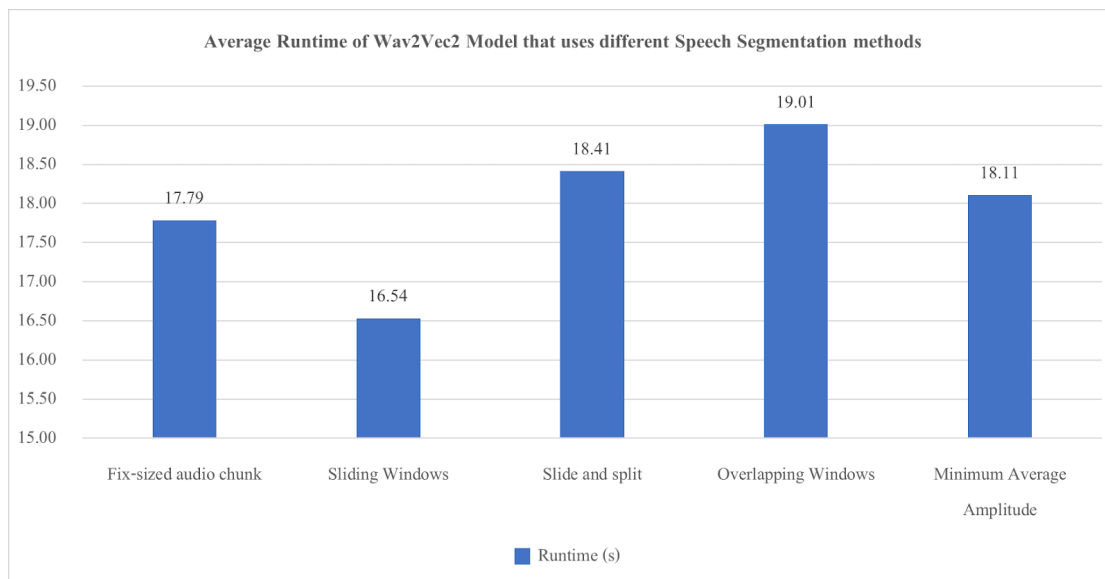
4.2 ประสิทธิภาพเวลาที่ใช้ในการทำงานของแต่ละการตัดแบ่งเสียง

4.2.1 Whisper



ตารางที่ 4.3 ตารางเปรียบเทียบระยะเวลาในการถอดความของโมเดล Whisper ด้วยวิธีการตัดแบ่งเสียงวิธีต่าง ๆ

4.2.2 Wav2vec2

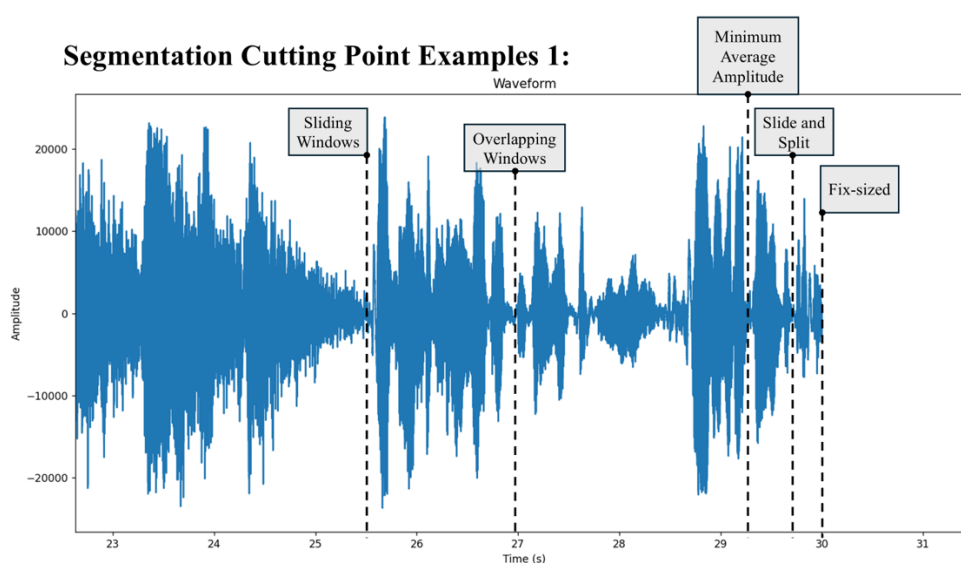


ตารางที่ 4.4 ตารางเปรียบเทียบระยะเวลาในการถอดความของโมเดล Wav2Vec2 ด้วยวิธีการตัดแบ่งเสียงวิธีต่าง ๆ

4.3 ตัวอย่างการเปรียบเทียบจุดตัดและข้อความจากแต่ละวิธีการตัดแบ่งเสียง

ผลลัพธ์ตัวอย่างเปรียบเทียบจุดตัดที่วิธีการตัดแบ่งเสียงเลือก และผลลัพธ์ของข้อความที่โมเดลถอดข้อความจากแต่ละวิธีการตัดแบ่งเสียง โดยมี 6 ตัวอย่าง ดังนี้

4.3.1 ผลลัพธ์จากตัวอย่างที่ 1



รูปที่ 4.1 ตัวอย่างเปรียบเทียบจุดตัดที่วิธีการตัดแบ่งเสียงเลือก จากผลลัพธ์ตัวอย่างที่ 1

Segmentation Cutting Point Examples 1:

Reference:

if we succeed which spoiler we will we will have upgraded the picture quality the sound quality and the smarts For Better

Fix-sized:

if we succeed which spoiler we will we will have upgraded the pitch quality the sound quality and the smarts for better

Sliding Windows:

if we succeed with spoiler we will we will have upgraded the picture quality the sound quality and the smart

Slide and Split:

if we succeed which spoiler we will we will have upgraded the picture quality the sound quality and the smarts for better

Overlapping Windows:

if we succeed which spoiler we will we will have upgraded the picture quality the sound quality and the smarts for better

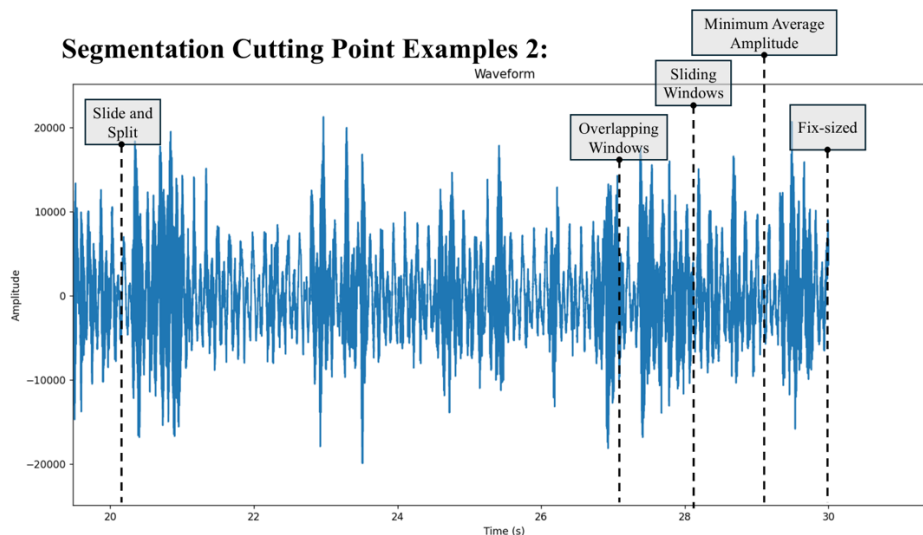
Minimum Average Amplitude:

if we succeed which spoiler we will we will have upgraded the picture quality the sound quality and the smarts for better

- Red Text = Text Before Cutting Point
- Green Text = Text After Cutting Point

รูปที่ 4.2 ข้อความที่โมเดลถอดข้อความจากแต่ละวิธีการตัดแบ่งเสียง จากผลลัพธ์ตัวอย่างที่ 1

4.3.2 ผลลัพธ์จากตัวอย่างที่ 2



รูปที่ 4.3 ตัวอย่างเปรียบเทียบจุดตัดที่วิธีการตัดแบ่งเสียงเลือก จากผลลัพธ์ตัวอย่างที่ 2

Segmentation Cutting Point Examples 2:

Reference:

who wants to be in the hot seat first? I kinda would. Great. That is perfect for us.

Fix-sized:

who wants to be in the hot seat first I got wood great that is perfect for us

Sliding Windows:

who wants to be in the world in the hot seat first I kind of would great that is perfect for us

Slide and Split:

who wants to be in the hot seat first I kind of would great that is perfect for us

Overlapping Windows:

who wants to be in the hot seat first I got would great that is perfect for us

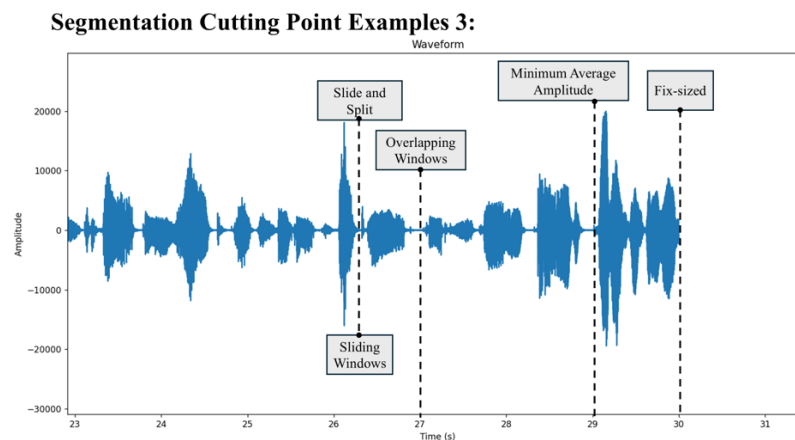
Minimum Average Amplitude:

who wants to be in the hot seat first I kind of would great that is perfect for us

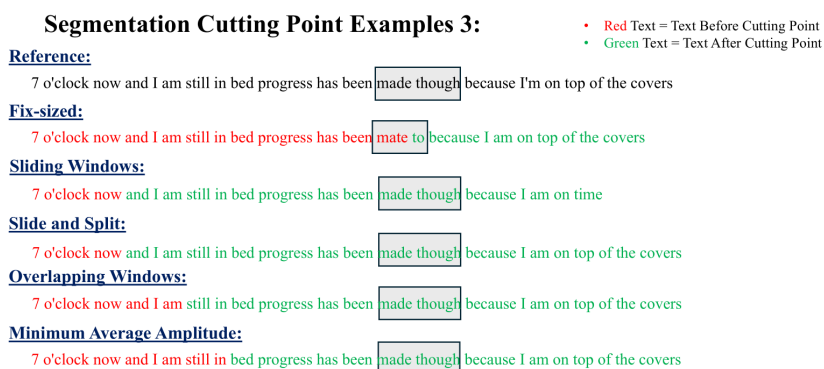
- Red Text = Text Before Cutting Point
- Green Text = Text After Cutting Point

รูปที่ 4.4 ข้อความที่โมเดลถอดข้อความจากแต่ละวิธีการตัดแบ่งเสียง จากผลลัพธ์ตัวอย่างที่ 2

4.3.3 ผลลัพธ์จากตัวอย่างที่ 3

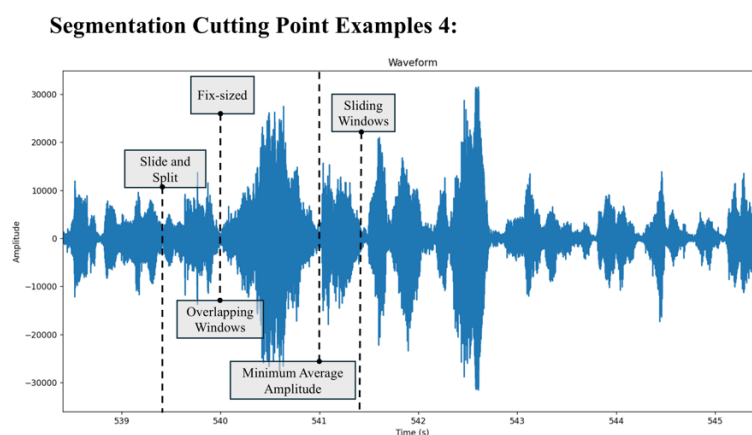


รูปที่ 4.5 ตัวอย่างเปรียบเทียบจุดตัดที่วิธีการตัดแบ่งเสียงเลือก จากผลลัพธ์ตัวอย่างที่ 3



รูปที่ 4.6 ข้อความที่โมเดลถอดข้อความจากแต่ละวิธีการตัดแบ่งเสียง จากผลลัพธ์ตัวอย่างที่ 3

4.3.4 ผลลัพธ์จากตัวอย่างที่ 4



รูปที่ 4.7 ตัวอย่างเปรียบเทียบจุดตัดที่วิธีการตัดแบ่งเสียงเลือก จากผลลัพธ์ตัวอย่างที่ 4

Segmentation Cutting Point Examples 4:

Reference:

where they get caught up all the time. I am a red. I am out. It is mesmerizing. You are out?

Fix-sized:

where they get caught up all the time abg I am out It is mesmerized you are out

Sliding Windows:

where they get caught up all the time I am red oh I am out smesmerize you are out

Slide and Split:

where they get caught up all the time I am out It is mesmerized you are out

Overlapping Windows:

where they get caught up all the time I am red I am out I am out It is mesmerized you are out

Minimum Average Amplitude:

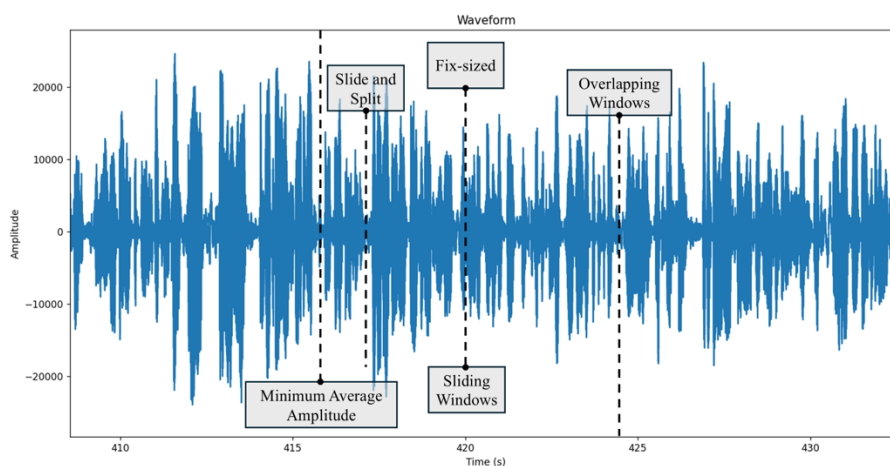
where they get caught up all the time I am a red I am out It is mesmerized you are out

- Red Text = Text Before Cutting Point
- Green Text = Text After Cutting Point

รูปที่ 4.8 ข้อความที่โมเดลถอดข้อความจากแต่ละวิธีการตัดแบ่งเสียง จากผลลัพธ์ตัวอย่างที่ 4

4.3.5 ผลลัพธ์จากตัวอย่างที่ 5

Segmentation Cutting Point Examples 5:



รูปที่ 4.9 ตัวอย่างเปรียบเทียบจุดตัดที่วิธีการตัดแบ่งเสียงเลือก จากผลลัพธ์ตัวอย่างที่ 5

Segmentation Cutting Point Examples 5:

Reference:

this right here is going to be the base of our TV and it is going to be heavy as all frick

Fix-sized:

this right here is going to be the base of our tv going to be heavy as all fric

Sliding Windows:

this right here is going to be the base of our tv it is going to be heavy as all fric

Slide and Split:

this right here going to be the base of our tv and it is going to be heavy as all frick

Overlapping Windows:

this right here is going to be the base of our tv and it is go to be heavy as all frick

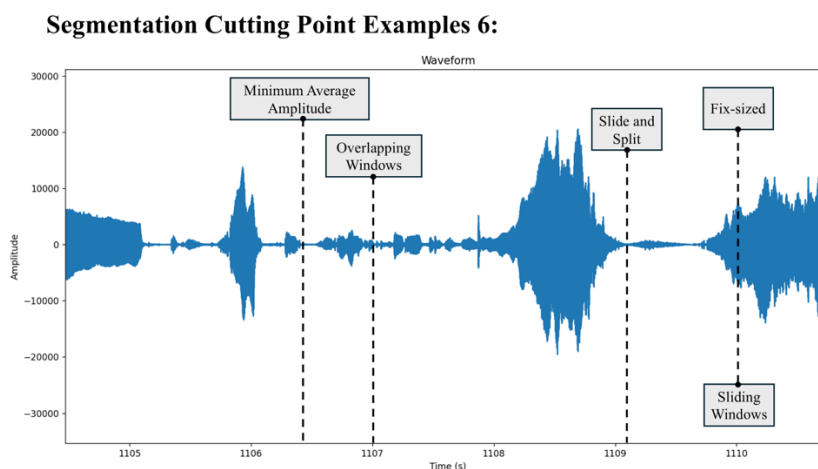
Minimum Average Amplitude:

this right here is going to be the base of our tv and it is going to be heavy as all frick

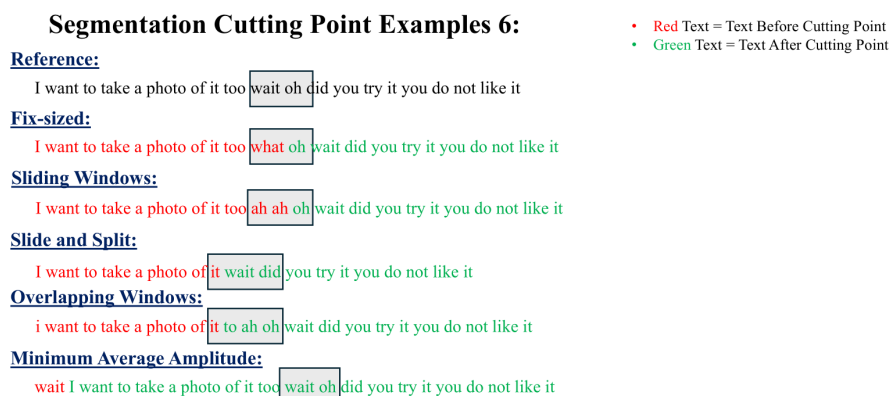
- Red Text = Text Before Cutting Point
- Green Text = Text After Cutting Point

รูปที่ 4.10 ข้อความที่โมเดลถอดข้อความจากแต่ละวิธีการตัดแบ่งเสียง จากผลลัพธ์ตัวอย่างที่ 5

4.3.6 ผลลัพธ์จากตัวอย่างที่ 6



รูปที่ 4.11 ตัวอย่างเปรียบเทียบจุดตัดที่วิธีการตัดแบ่งเสียงเลือก จากผลลัพธ์ตัวอย่างที่ 6



รูปที่ 4.12 ข้อความที่โมเดลถอดข้อความจากแต่ละวิธีการตัดแบ่งเสียง จากผลลัพธ์ตัวอย่างที่ 6

จากรูปที่ 57, 59, 61 ภายในสี่เหลี่ยมที่เป็นช่วงที่ Fix-sized ตัดแบ่งเสียงพอดี ทำให้ได้คำออกมาผิด ซึ่งเมื่อเทียบกับวิธีการตัดแบ่งเสียงอื่นๆ ที่เลือกจุดตัดต่างกันออกไป จะพบว่าวิธีการตัดแบ่งเสียงอื่นๆ จะทำให้ผลลัพธ์ภายในสี่เหลี่ยมออกมาถูกต้อง แต่บางวิธีตัดแบ่งเสียงอย่างเช่น Sliding Windows ที่จุดตัดของวิธีนี้ก็กลับกลายเป็นจุดที่ผิดเอง

และจากรูปที่ 63, 65, 67 จากเห็นว่าในบางกรณีนั้น Sliding Windows, Slide and Split และ Overlapping Windows ไม่สามารถช่วยให้ผลลัพธ์ออกมาดีขึ้นได้ แต่ Minimum Average Amplitude เป็น Segmentation เดียวที่สามารถทำให้ผลลัพธ์ใกล้เคียงหรือเหมือนกับข้อความต้นฉบับได้ดีที่สุด

บทที่ 5

สรุปผลการดำเนินงาน

5.1 ผลลัพธ์การทดลอง

จากการทดลองเพื่อเพิ่มประสิทธิภาพของโมเดล ASR ด้วยการพัฒนาการตัดแบ่งส่วนเสียงนั้น ทำให้ทราบว่า การพัฒนาการตัดแบ่งส่วนเสียงสามารถทำให้คะแนน Word Error Rate (WER) และ Character error rate (CER) ของโมเดล Whisper และ Wav2Vec2 ไปในทิศทางที่ดียิ่งขึ้น

ถึงแม้ว่า Wav2Vec2 จะไม่สามารถให้ผลลัพธ์ที่ดีมากนัก เนื่องจากตัวความสารถของโมเดลเอง โดยแม้ Segmentation จะช่วยเรื่องเสียงให้แล้ว แต่ในบางกรณีโมเดลก็ยัง Prediction ผิดอยู่ดี

ซึ่งจะขอแบ่งอธิบายแต่ละ Segmentation ตามหัวข้อ ดังนี้

5.1.1 Fix-sized Audio chunk

สำหรับ Segmentation นี้ เป็นวิธีการแบ่งเสียงที่ใช้เป็นวิธีมาตรฐานสำหรับการถอดความจากไฟล์เสียง โดยมีจุดมุ่งหวังให้ Segmentation อื่นๆ ให้ผลลัพธ์ที่มีประสิทธิภาพมากกว่าวิธีนี้

5.1.2 Sliding Windows

เป็นวิธีการตัดแบ่งเสียงที่ให้ผลลัพธ์ที่มีประสิทธิภาพน้อยที่สุดเมื่อเทียบกับวิธีการตัดแบ่งเสียงวิธีอื่นๆ เพราะวิธีนี้นั้นใช้ความเรียบง่ายในการค้นหาจุดตัด คือการสนใจเพียงแค่แอมพลิจูดที่ต่ำที่สุดที่พบ ซึ่งทำให้อาจไปตัดโดนกลางคำง่ายกว่าวิธีการอื่นๆ และด้วยเหตุผลนี้ทำให้ช่วงเสียงที่ตัดแบ่งได้มีมากกว่าวิธีการอื่นอย่างเห็นได้ชัด ทำให้ใช้เวลาในการประมวลผลเพื่อถอดข้อความทั้งหมดนานที่สุด

5.1.3 Slide and Split

สามารถแบ่งจุดหาช่วงตัดได้ดี เพราะมีตัวเลือกการตัดจากการแบ่งสอง Windows แต่ด้วยการที่มี Threshold กำหนด เมื่อเจอบางเสียงบางช่วงที่หาจุดตัดที่ดีภายใต้ Threshold ไม่ได้ เช่นเสียงที่มีเพลงประกอบด้านหลัง ทำให้บางครั้งอาจตัดไปโดนกลางคำอยู่ดี ซึ่งเป็นสาเหตุที่วิธีการตัดแบ่งเสียงนี้ไม่ใช่วิธีการที่ดีที่สุด

5.1.4 Overlapping Windows

เป็นวิธีการตัดแบ่งเสียงที่ไปทำงาน Post Processing ซึ่งหมายความว่าสามารถตัดแปลงข้อความหลังจากโมเดลทำการถอดข้อความ แต่วิธีนี้จะมีประสิทธิภาพ ขึ้นอยู่กับความสามารถของโมเดลด้วย แม้จะสามารถช่วยแก้ไขข้อความที่ผิดได้ในหลายกรณี แต่ในบางกรณี

ที่โมเดลถอดข้อความผิดทั้งประโยคก็มีโอกาสทำให้โมเดลหาจุดที่ข้อความส่วนควรจะซ้อนทับกันไม่เจอ และทำให้วิธีการตัดแบ่งเสียงนี้ให้ผลลัพธ์ที่ผิด

5.1.5 Minimum Average Amplitude

เป็นวิธีการตัดแบ่งเสียงที่คำนวณจุดที่ต้องการจะตัดจากการนำช่วงเสียงเล็กๆ เท่ากับ 20ms ไปคำนวณกับค่าแอมพลิจูด ทำให้หาจุดตัดที่หลบการตัดโดนกลางคำได้ละเอียด และมีประสิทธิภาพ ซึ่งวิธีการนี้ ให้ผลลัพธ์ของ WER และ CER ได้ดีที่สุดเมื่อเทียบกับวิธีการตัดแบ่งเสียงวิธีการอื่นๆ

บรรณานุกรม

[1] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey and I. Sutskever. “Robust Speech Recognition via Large-Scale Weak Supervision” Available form:

<https://arxiv.org/pdf/2212.04356.pdf>

[2] Alexei Baevski, Henry Zhou, Abdelrahman Mohamed, Michael Auli. “wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations” Available form:

<https://arxiv.org/abs/2006.11477>

[3] Y. Zhang, W. Han, J. Qin, Y. Wang, A. Bapna, Z. Chen, N. Chen, B. Li and et al. “Google USM: Scaling Automatic Speech Recognition Beyond 100 Languages” Available form:

<https://arxiv.org/pdf/2303.01037.pdf>

[5] A. Gulati, J. Qin, C.C. Chiu, N. Parmar, Y. Zhang, J. Yu, W. Han, S. Wang and et al. “Conformer: Convolution-augmented Transformer for Speech Recognition” Available form

<https://arxiv.org/abs/2005.08100>

[6] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser and I. Polosukhin. “Attention Is All You Need” Available form:

<https://arxiv.org/pdf/1706.03762.pdf>

[7] Haytham Fayek. “Speech Processing for Machine Learning: Filter banks, Mel-Frequency Cepstral Coefficients (MFCCs) and What's In-Between” Available form:

<https://haythamfayek.com/2016/04/21/speech-processing-for-machine-learning.html>

[8] W. R. Huang, S. Chang, D. Rybach, R. Prabhavalkar, T. N. Sainath, C. Allauzen, C. Peyser and Z. Lu. “E2E Segmenter: Joint Segmenting and Decoding for Long-Form ASR” Available form:

<https://arxiv.org/pdf/2204.10749.pdf>

[9] P. Behre, N. Parihar, S. Tan, A. Shah, E. Sharma, G. Liu, S. Chang, H. Khalil and et al.
 “Smart Speech Segmentation using Acousto-Linguistic Features with look-ahead” Available
 form:

<https://arxiv.org/pdf/2210.14446.pdf>

[10] “Common Voice” Available form:

<https://commonvoice.mozilla.org/>

[11] T. Bäckström, O. Räsänen, A. Zewoudie, P.P. Zarazaga, L. Koivusalo, S. Das, E.G. Mellado,
 M.B. Mansali and et al. “Voice Activity Detection (VAD)” Available
 form:

https://speechprocessingbook.aalto.fi/Recognition/Voice_activity_detection.html#introduction

[12] April Craig. “รู้จัก Voice Activity Detection สิ่งสำคัญต่อการสื่อสารในห้องประชุม”
 Available form:

<https://conceptocomunicacion.com/voice-activity-detection/>

ภาคผนวก

บทความวิจัย

Speech Segmentation Methods for Improving Automatic Speech Recognition

Kietikul Jearanaitanakij¹, Saranthorn Pornchaiwattanakorn¹, Kampanat Chinnarach¹,
Thanaphat Praphatpornkul¹, Vipada Varasajja¹, Tiwat Kullawan² and Chankit
Yongpiyakul²

¹Department of Computer Engineering, School of Engineering, King Mongkut's
Institute of Technology Ladkrabang, Bangkok 10520 THAILAND

² Dataxet Limited, Bangkok, 10330, Thailand

Abstract

Transcribing speech files into text can be challenging due to the audio-length constraint of Automatic Speech Recognition (ASR) models like Whisper and Wav2Vec2. Most models split audio clips into smaller segments, for instance, a 30-second segment for Whisper. This enables GPUs with limited memory to transcribe the speech segments into text. However, they often misinterpret words at the splitting points of an audio file since the segmentation frequently occurs within a single word, resulting in a higher word error rate. We introduce four speech segmentation methods that allow for flexible segmentation of a speech file into varying lengths, all within the 30-second limit. Our primary objective is to decrease the word and character error rates produced by the ASR models. We also compare our proposed techniques using well-known ASR models such as Whisper and Wav2Vec2. Our experiments on 10 video clips from YouTube and their corresponding subtitles demonstrate that most of our speech segmentation approaches significantly improve the prediction error. The best method, i.e., Minimum Average Amplitude, is recommended since it is the most effective in reducing WER and CER while maintaining a negligible impact on the ASR runtime, making it a valuable tool for enhancing ASR performance.

Keywords: Speech segmentation, Automatic speech recognition, Whisper, Wav2Vec2

Introduction

Automatic Speech Recognition (ASR), also known as Speech-to-text, is a field of study that aims to transform raw audio into a sequence of corresponding words. ASR is a challenging yet exciting field that continues to evolve, and it plays a crucial role in applications like voice assistants, transcription services, and more. Speech segmentation is one important preprocessing step in most deep-learning-based ASR models. It aims to cut a long audio file into multiple segments, each with a length within a certain period, e.g., 30 seconds. Understanding the boundaries between words or

phonemes is crucial in natural language processing (NLP) and ASR. Speech is a continuous signal, in which pauses rarely separate words. Moreover, while some words occasionally occur in isolation, many do not (Aslin et al., 1996). To avoid deteriorating the quality of ASR models when translating speech at both ends of an audio segment, a speech segmentation must properly identify the boundaries between words, syllables, or phonemes in spoken natural languages. Unlike written text, spoken language lacks explicit boundaries (such as spaces) between words. Variability in speech rate, accents, and coarticulation makes segmentation challenging. Research works relevant to speech segmentation are categorized into 3 classes based on their fundamental techniques.

In acoustic-based speech segmentation, spoken language is segmented into multiple segments using acoustic cues (White 2018). These cues included pauses, pitch changes, and duration variations between words. The absence of consistent boundaries between spoken words posed a challenge, as speech sounds are contextually variable. Behre et al. (2022) combined acoustic and language information to enhance segmentation quality. By leveraging acoustic features and language context, including a single word as a look-ahead further boosts segmentation quality, the model achieves better segmentation accuracy. Mittmann and Barbosa (2022) developed an automatic speech segmentation tool that utilizes multiple acoustic parameters for identifying boundaries within speech segments focusing on spontaneous speech in Brazilian Portuguese. They extracted prosodic features from the audio signal, including parameters related to duration, fundamental frequency (F0), and intensity. Their model can identify boundaries of elementary linguistic units such as utterances and tonal units.

Prosodic-cue-based speech segmentation employs prosodic cues, e.g., pauses, and intonation patterns, to separate speech into pieces. Namjoshi et al. (2012) shed light on the interplay between linguistic exposure, prosodic cues, and speech segmentation. They investigated the impact of recent linguistic exposure and native language (L1) on listeners' ability to extract prosodic cues from unfamiliar speech streams. They found that listeners can effectively extract prosodic cues from unfamiliar speech streams, especially when other information (lexical, syntactic, or semantic) is unavailable. Lexical information seems to play a stronger role than prosodic information in Second language (L2) speech segmentation. Moreover, a transfer of L1 prosodic cues to L2 segmentation is limited. White et al. (2012) investigated how listeners interpret segmentation cues in conversational speech compared to read speech. Two types of cues were considered: semantic likelihood (the plausibility of word boundaries based on meaning) and cross-boundary diphone phonotactics (acoustic-phonetic cues related to phoneme transitions across word boundaries). Their experimental results indicate that semantic plausibility affected segmentation behavior, regardless of speech style. However, there was no effect of cross-boundary phonotactics. Tremblay et al. (2019) investigated how the fine-grained phonetic realization of tonal cues impacts speech segmentation when the cues signal the same word boundary in the native and unfamiliar languages. They discovered that Korean listeners' segmentation improved as the phrase-initial low (L) tone was lowered. Korean listeners tune in to low-level cues that conform to the Korean intonational grammar when the phrase-initial L tone is not canonical phonologically.

The last category of speech segmentation is based on statistical and machine learning. Yurovsky et al. (2012) aimed to understand how children acquire language by simultaneously learning speech segmentation (identifying word boundaries) and mapping words to objects in their environment. They recorded a corpus of child-

directed speech during naturalistic free-play interactions between parents and children. They identified two consistent regularities in this corpus's sentence structure of naming events. According to an artificial language created from these regularities, either regularity alone was sufficient for successful learning. Child-directed speech structure was crucial in scaffolding simultaneous speech segmentation and word learning. Palmer et al. (2018) employed statistical learning (SL) to support word segmentation and language acquisition in infants and young adults. SL involves detecting regularities in the input, allowing learners to identify word boundaries based on frequency patterns. Participants were exposed to continuous speech streams at different speech rates, both with and without cognitive load. Remarkably, speech segmentation by SL was resilient to aging, although age effects were visible under more challenging conditions. Venkatesh et al. (2021) proposed a novel approach for enhancing speech and music detection in radio broadcasts. They created synthetic data resembling radio signals to improve the performance of audio classification and segmentation models. By artificially generating labeled data, they addressed challenges related to training deep learning models using copyrighted material. The study provides insights into the effectiveness of synthetic data for radio signal analysis and aims to enhance the accuracy of speech and music detection in broadcast content. Bhati et al. (2022) proposed the segmental contrastive predictive coding (SCPC) framework that bridges the gap between speech segmentation and representation learning, leveraging frame and segment-level information. It starts by learning frame-level representations from raw speech waveforms using a convolutional neural network (CNN). Then, a differentiable boundary detector identifies variable-length segments (e.g., phones or word-like units). Finally, a segment encoder extracts representations for each segment, allowing joint training of frame-level and segment-level encoders. SCPC provides variable rate representations, adapting to the content of different segments. Dal Ben et al. (2022) explored whether learners can leverage the structure of linguistic input to perform speech segmentation and cross-situational word learning simultaneously. Their experiments indicate that participants could map words to objects but struggled with speech segmentation when the speech stream had conflicting statistics. In addition, adults correctly identified non-words but still could not discriminate between words and part-words. They suggested that learners can track multiple sources of statistical information to find and map words to objects in noisy environments. Kamper and Niekerk (2024) proposed improvements to a self-supervised method for segmenting unlabeled speech into word-like segments. The authors begin with a two-stage duration-penalized dynamic programming method, which achieves zero-resource segmentation without explicitly learning an explicit lexicon. In the initial stage, they replace contrastive predictive coding features with HuBERT, a superior self-supervised feature. Following word segmentation in the second stage, they compute an acoustic word embedding for each segment by averaging HuBERT features. These embeddings are then clustered using K-means to construct a lexicon. Their approach demonstrates effective full-coverage segmentation, achieving state-of-the-art performance on the ZeroSpeech benchmark.

This research proposes four speech segmentation techniques of ASR. We conduct the experiments on 10 audio clips approximately 30 minutes each and their corresponding English subtitles. Most of the proposed methods can significantly reduce the word and character error rates of two well-known ASRs (Whisper, and Wav2Vec2) with a negligible impact on the ASR runtime.

The rest of this paper is organized as follows. Section 1 provides the fundamental pieces of knowledge necessary to understand this research. Section 2 describes the speech dataset and four proposed speech segmentation methods, including traditional speech segmentation. Section 3 contains the experiments and comparison among all techniques along with discussions. Finally, section 4 concludes the performance of the proposed method, including its limitations.

Fundamental Knowledge

1. Whisper

The Whisper model (Radford et al., 2019) and his colleagues, is a robust speech recognition system trained using large-scale weak supervision. It leverages 680,000 hours of multilingual and multitask audio data sourced from the internet to achieve high performance. The Whisper model employs a simple end-to-end architecture based on the encoder-decoder Transformer framework. Its architecture, as shown in Fig. 1, is composed of three components. First, audio input is divided into 30-second segments and converted into Log-Mel spectrograms. Next, these spectrograms are fed into the encoder, which processes the audio data into a sequence of hidden states. This encoding process allows Whisper to effectively transform raw audio into a format that can be used for various speech recognition tasks. Lastly, the decoder then predicts the corresponding text using the hidden states from the encoder along with a sequence of previously generated tokens to handle tasks like language identification, phrase-level timestamps, multilingual transcription, and translation. A self-attention mechanism helps the decoder focus on different parts of the input sequence, allowing it to generate contextually relevant outputs. Whisper also excels in zero-shot transfer, meaning it can generalize well to new tasks without additional fine-tuning. This model approaches human-level accuracy and robustness in speech recognition, making it a significant advancement in the field. The authors have also made the models and inference code open-source, encouraging further research and development in robust speech processing.

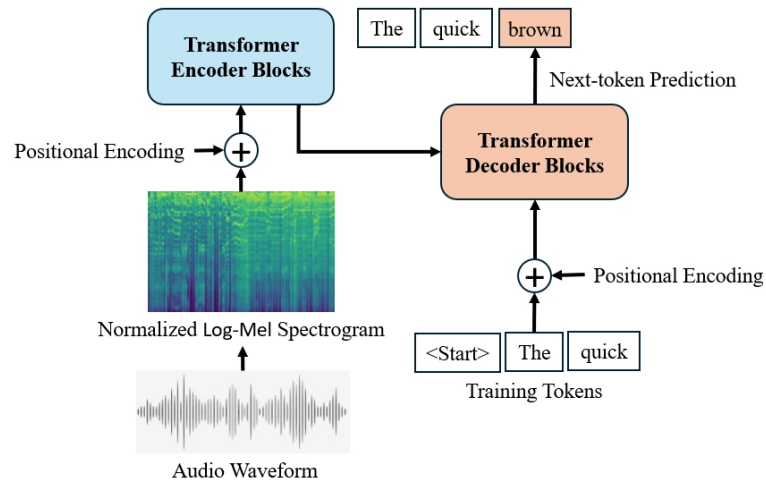


Fig. 1 Whisper Architecture

2. Wav2Vec2

Wav2Vec2 is a seminal framework in speech processing, particularly influential in the realm of automatic speech recognition (ASR). Developed by Facebook AI, Wav2Vec2 utilizes self-supervised learning to pre-train speech representations on large volumes of

unlabeled audio data, thereby reducing the reliance on labeled datasets. The model learns robust speech features by predicting future speech samples from past ones using a contrastive loss function. The original Wav2Vec model, introduced by Schneider et al. (2019), demonstrated significant improvements in ASR performance by leveraging these pre-trained representations. Subsequent iterations, such as Wav2Vec 2.0 (Baevski et al., 2020), have further enhanced the framework's capabilities, achieving state-of-the-art results on various benchmarks through the use of transformer architectures and discrete unit quantization.

3. Voice Activity Detection

Voice Activity Detection (VAD) is a crucial technology in speech-processing applications, such as mobile communication, multimedia transmission, forensic science, and voice recognition (Graf et al., 2015). The primary function of VAD is to distinguish between speech and non-speech segments in an audio signal, which is essential for enhancing the efficiency and accuracy of these applications. Early VAD methods relied on simple energy-based features and zero-crossing rates, which were often susceptible to noise. To address these limitations, more advanced techniques (Ball, 2023) have been developed, including Mel-Frequency Cepstral Coefficients (MFCCs), autocorrelation-based features, and linear prediction residuals. Machine learning algorithms, such as Gaussian Mixture Models (GMMs) and Hidden Markov Models (HMMs), have also been employed to improve robustness and accuracy. Recently, deep learning approaches have gained popularity, leveraging large datasets and complex neural network architectures to enhance VAD performance. Despite these advancements, challenges remain, particularly in noisy environments and with varying speaker characteristics.

4. Levenshtein Distance

The Levenshtein distance (Levenshtein, 1966), also known as edit distance, is a metric used in information theory, linguistics, and computer science to measure the difference between two sequences. It calculates the minimum number of single-character edits—insertions, deletions, or substitutions—required to transform one string into another. Named after the Soviet mathematician Vladimir Levenshtein, who introduced it in 1965, this distance metric is particularly useful in applications like spell checkers, DNA sequencing, and natural language processing, where it helps to quantify how similar or different two strings are. For example, the Levenshtein distance between the words “neto” and “melt” is 3, as it requires three edits: substituting ‘n’ with ‘m’, deleting ‘o’, and inserting ‘l’ at the end. The calculation involves constructing a matrix where each cell represents the cost of transforming the input strings' substrings, and the matrix's final value gives the Levenshtein distance. This method allows for efficient computation even for longer strings, making it a robust tool for various computational tasks.

5. Word Error Rate (WER)

WER (Klakow & Peters, 2002) is a widely used metric for evaluating the performance of speech recognition and machine translation systems. It measures the accuracy of these systems by comparing the transcribed output to a reference transcript, calculating the number of errors as a proportion of the total words. Specifically, WER is computed by summing the number of substitutions, deletions, and insertions needed to transform the system's output into the reference transcript, and then dividing by the total number of words in the reference. The word error rate can be computed as:

$$\text{WER} = (S + D + I) / N$$

Where S is the number of substitutions, D is the number of deletions, I is the number of insertions, and N is the number of words in the reference.

This metric ranges from 0 to 1, where 0 indicates a perfect match with no errors, and 1 signifies complete dissimilarity. For example, if a speech recognition system transcribes “hello world” as “hello word,” the WER would be 0.5, accounting for one substitution error out of two words. WER is derived from the Levenshtein distance but operates at the word level rather than the character level, making it particularly useful for assessing the overall accuracy of spoken language systems. Despite its utility, WER does not account for the severity of different types of errors or their impact on comprehension, which can vary significantly. Therefore, while a lower WER generally indicates better performance, it is often supplemented with other metrics to provide a more comprehensive evaluation of a system’s effectiveness

6. Character Error Rate (CER)

CER is a crucial metric used to evaluate the performance of automatic speech recognition (ASR) systems. It measures the system's accuracy by calculating the percentage of incorrectly predicted characters, including substitutions, deletions, and insertions. CER shares the same formula as WER but considers characters instead of words.

Unlike WER, which operates at the word level, CER focuses on individual characters. It is particularly useful for identifying finer-grained errors such as mispronunciations and erroneous phonemes. To compute CER, the total number of character errors is divided by the total number of characters in the reference text, providing a ratio that reflects the system’s precision. A CER of 0 indicates perfect accuracy, while higher values signify more errors. This metric is essential for applications where precise character recognition is critical, such as in transcribing names, technical terms, or languages with complex orthographies. Despite its utility, CER can be challenging to optimize due to the variability in speech patterns, accents, and background noise. Advances in deep learning and neural network architectures have significantly improved CER by enabling more robust and adaptive ASR systems.

Proposed Speech Segmentations for ASR

1. Process Overview

To provide readers with an understanding of the overall process before presenting our speech segmentation method, Figure 2 illustrates all components of the speech transcription, with our proposed speech segmentation method positioned in the middle.

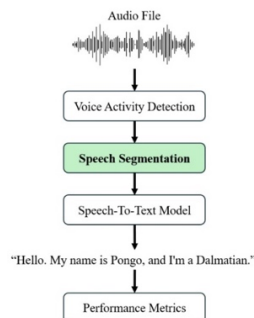


Fig. 2 System overview

In the preprocessing stage, the audio file is trimmed to include only human speech using a Voice Activity Detection (VAD) tool. The human speech segments are then processed using the proposed speech-segmentation method to produce speech segments no longer than 30 seconds. This reduces the burden on the Speech-to-Text model and allows for predictable model processing times. Once preprocessing is complete, the speech segments are passed to the speech transcription model, which outputs text. We then compare this output against various metrics such as Word Error Rate (WER) and Character Error Rate (CER) to measure performance against ground truths.

2. Proposed Speech Segmentations

We propose four speech segmentation techniques: Sliding Window, Right-split First, Overlapping Windows, and Minimum Average Amplitude. Besides for techniques, we use the fixed-length audio segment method as a default audio segmentation in ASRs.

2.1 Fixed-length audio segment (Default audio segmentation in ASRs)

This method is straightforward, dividing the speech audio file into segments, each 30 seconds long, as shown in Figure 3, with red arrows indicating the cut points. The advantage of this method is its simplicity and speed of application. However, a drawback is that it may cut in the middle of the speech, potentially causing the transcription model to produce a wrong prediction.

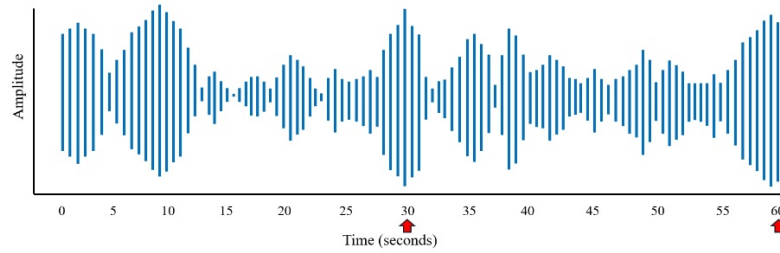


Fig. 3 Fixed-length audio segment

2.2 Sliding Window

To mitigate the issue of cutting through speech in the fixed-length method, we create a 30-second window starting from the leftmost unprocessed point of the waveform. We select the lowest amplitude point within the window for cutting, then shift the window to the left, starting from the last cut point, and select the next lowest amplitude point. This process is repeated until the entire audio file is segmented. We set a minimum cut length of 5 milliseconds to prevent the small fragment segments. This duration can be adjusted as needed. An excessive number of segments will cause the speech transcription model to spend unnecessary time switching these small segments. Figure 4 illustrates the steps of the sliding window method.

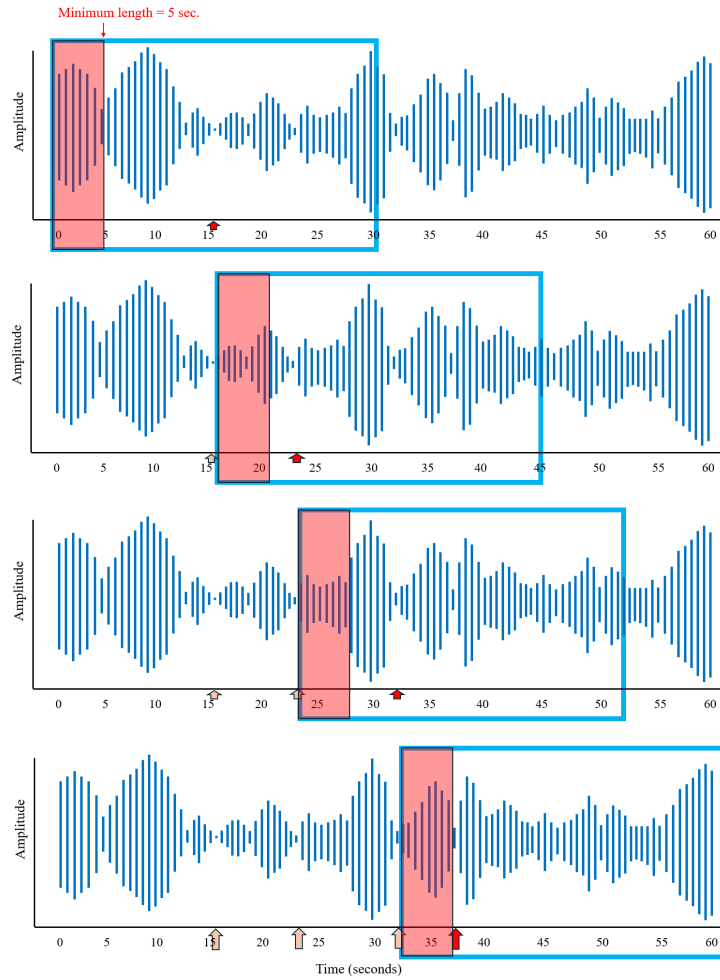


Fig. 4 Sliding Window method

Although this method can reduce the likelihood of cutting through speech inside a word, it results in a larger number of audio segments compared to the fixed-length method and tends to create smaller audio segments. Moreover, this method may miss the lowest amplitude point if it occurs within the first 5 seconds.

2.3 Right-split First

To reduce the number of audio segments in the Sliding Window method, the right-split first method creates two smaller windows of 15 seconds each. Then, it selects the lowest amplitude cut point within each window, ensuring the amplitude is below a specified threshold to match the gaps between words. We recommend experimenting with a suitable threshold for the best result. If a window cannot find a cut point below the threshold, it is left without a cutting point. Next, both windows are considered: if there are cut points, the cut point from the right window is always chosen. If the right window has no cut point, the cut point from the left window is selected. If neither window has a cut point, the lowest amplitude point in the right window is forced as the cut point. Figure 5 illustrates the steps of the Right-Split First method.

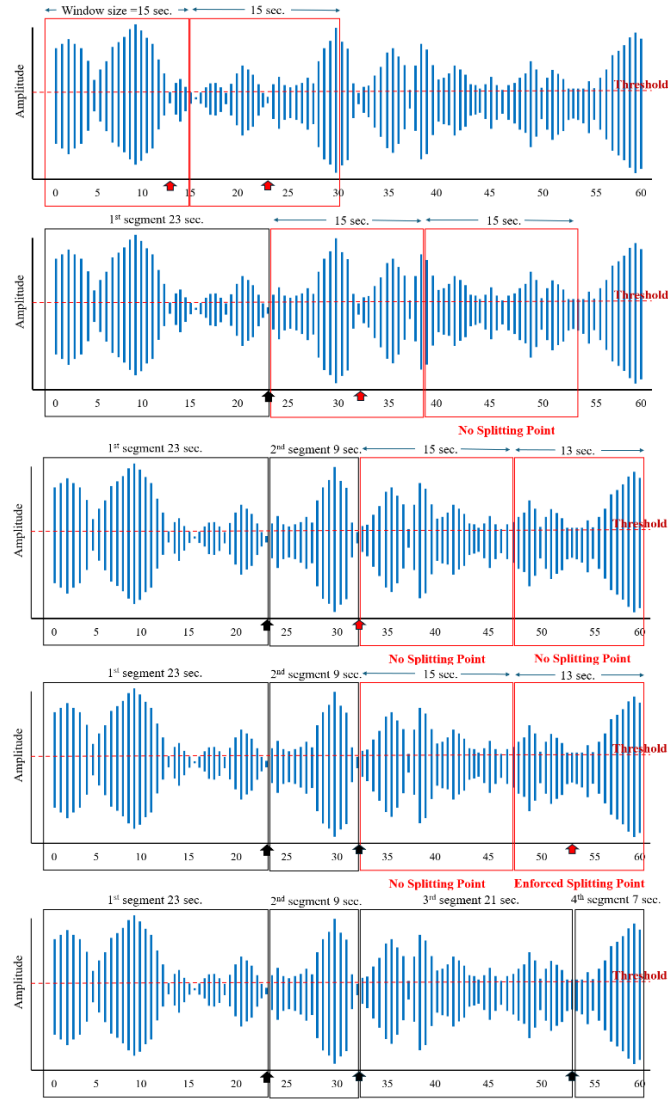


Fig. 5 Right-split First method

2.4 Overlapping Windows

The Right-Split First method seems to be the most effective for speech segmentation. Not only does it result in fewer segments, but the segments are also not excessively small. However, this method does not solve the problem of cuts occurring in the middle of words. The next method is the Overlapping Windows method. We create two windows, each 30 seconds long, with a slight overlap. In this case, we use a 3-second overlap. Subsequently, we input the audio segments from both windows into the transcription model. Word segmentation is necessary for languages without whitespace between words, such as Thai. We chose the newmm engine from PyThaiNLP for word segmentation; however, other engines like AttaCut can also be used. Once we have the text segmented by whitespace, we align the texts word by word, as illustrated in Figure 6, for better understanding.

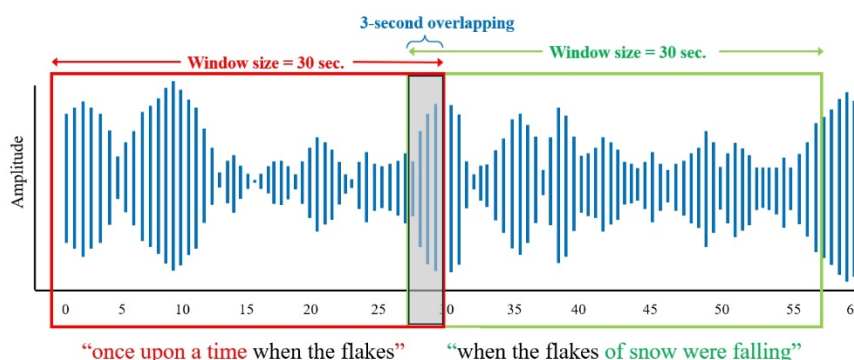


Fig. 6 Two overlapping windows of the Right-split First method

Suppose the full text of both windows, is "once upon a time when the flakes of snow were falling." The transcription model's output for the red window is "once upon a time when the flakes" and for the green window is "when the flakes of snow were falling," with a 3-second overlap of the text "when the flakes." The process of identifying overlapping text requires the steps shown in Figure 7.

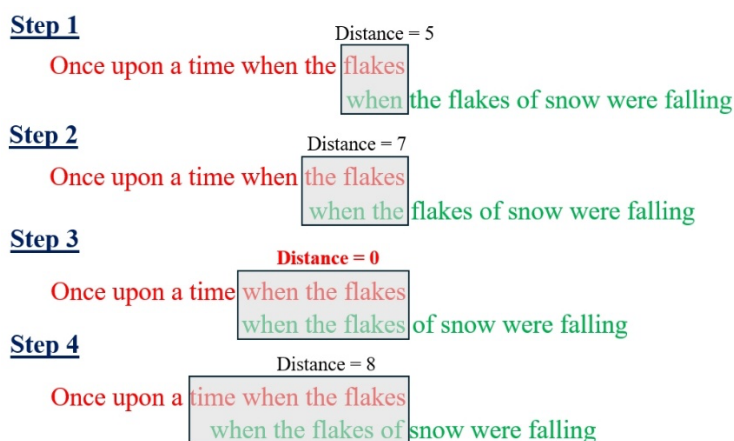


Fig. 7 Steps to determine the overlapping text

Figure 7 illustrates the process to determine the overlapping text. Each time the texts are aligned, the Levenshtein distance is calculated. The step where the Levenshtein distance equals zero is selected as the point where two texts are overlapped. In step 3, the Levenshtein distance is zero, leading to the conclusion that "when the flakes" is the overlapping text, allowing us to merge the two texts.

In cases where the edges of the two windows cut through words, such as the right edge of the red window cutting through the word "flakes" and the left edge of the green window cutting through the word "when", as shown in Figure 8. The transcription model may incorrectly predict some words so that "flake" might be transcribed as "fake" and "when" as "hen." In this case, the Levenshtein distance will never be zero. Therefore, we must calculate the Levenshtein distance for all overlap scenarios and select the case with the lowest Levenshtein distance. The leftmost word should be selected from the left text and the rightmost word from the right text, as these are not cut through the middle of the words.

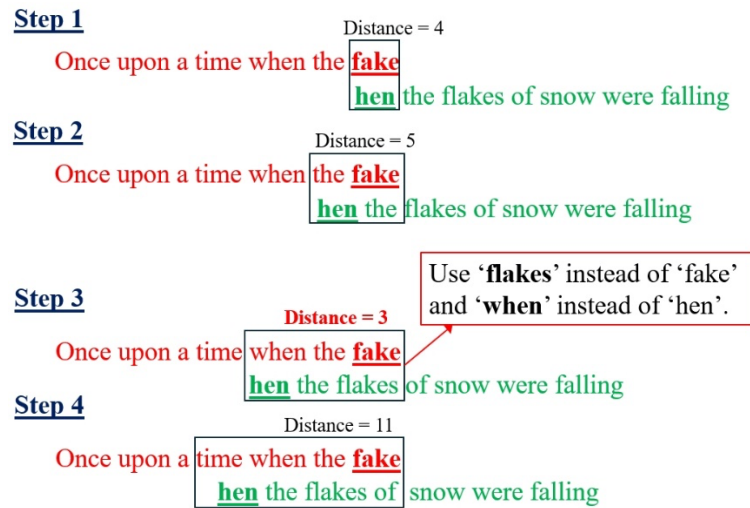


Fig. 8 Methods for handling speech segmentation occurring at the mid-word

2.5 Minimum Average Amplitude

The final method is simple yet more effective than other methods. In addition, it requires less preprocessing time compared to more complex methods. The Minimum Average Amplitude method operates similarly to the Sliding window method by sliding a 30-second window from left to right. For any given window, we analyze the average amplitude of 150 small frames, each 20 milliseconds long, during the last 3 seconds of the window. We then select the frame with the lowest average amplitude as the splitting point and slide the new window to start after the splitting point to analyze the next 30-second segment.

The reason for using a frame size of 20 milliseconds is based on experimental results comparing frame sizes of 10, 20, 30, 40, and 50 milliseconds, where the 20-millisecond frame size provided the most accurate ASR predictions. The simulation of the Minimum Average Amplitude method is shown in Figure 9. It can be observed that this method generates fewer splitting points compared to other methods, resulting in segments with lengths ranging from 27 to 30 seconds. This ensures efficient GPU utilization, as it avoids processing small segments that cause context switching overhead.

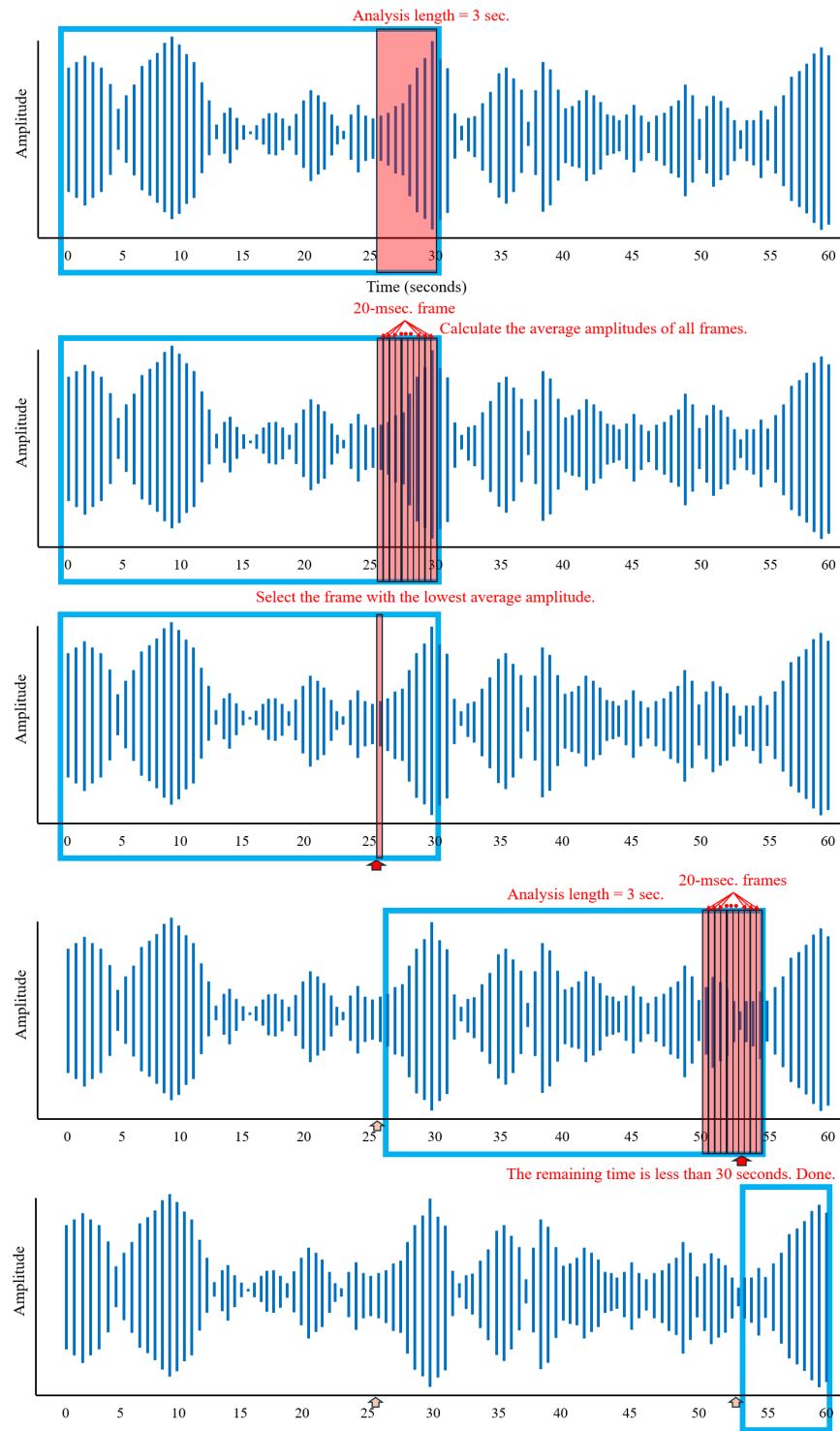


Fig. 9 Minimum Average Amplitude method

Experimental Results and Comparisons

We conducted experiments using 10 audio clips, each with a duration of approximately 30 minutes. These audio clips were sourced from YouTube channels that provide English subtitles. We selected renowned ASR systems, Whisper (small model) and Wav2Vec2, to demonstrate the improvement in word and character error rates after applying six different speech segmentation methods. The comparison of the averages of word and character error rates is presented in Figure 10 for Whisper and Figure 11 for Wav2Vec2.

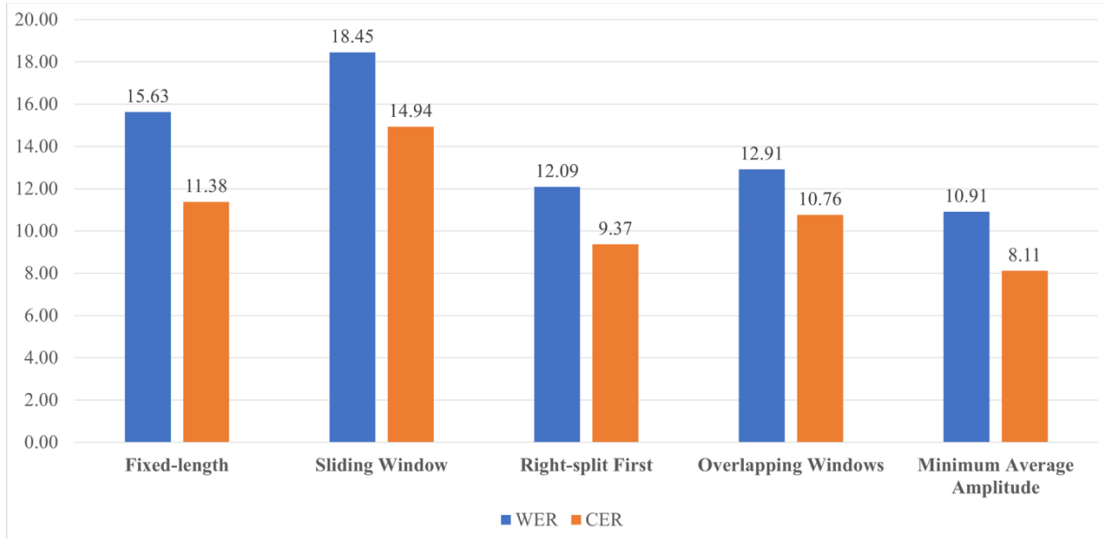


Fig. 10 Averages WER and CER of Whisper

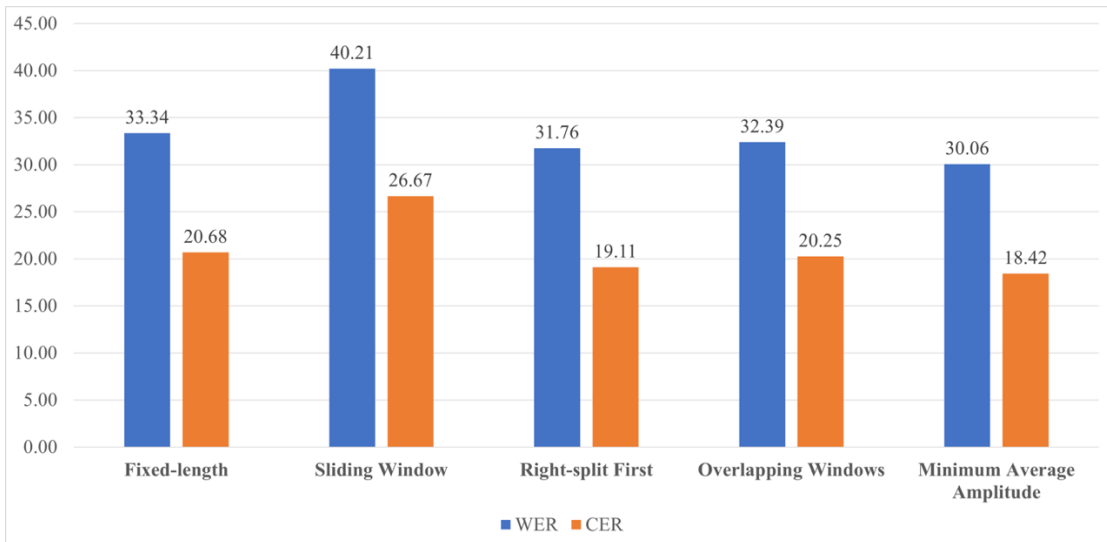


Fig. 11 Averages WER and CER of Wav2Vec2

The experimental results of WER and CER indicate that the Minimum Average Amplitude method achieves approximately 9-40% and 13-45% lower WER and CER, respectively, compared to other audio segmentation techniques. While almost all of the proposed methods improve the WER and CER of the default audio segmentation (Fixed-length), the Sliding Window method results in increased errors. This is because the Sliding Window method tends to create a large number of splitting points, causing the audio files to be divided into very small segments. Consequently, the ASR models

receive insufficient contextual information to accurately predict the next words, leading to a higher number of prediction errors.

To illustrate the impact of preprocessing steps on the overall time required for ASR predictions, Figures 12 and 13 present the average time, which includes both preprocessing time and ASR prediction time. It can be observed that almost all of the proposed methods take only slightly more time than the Fixed-length (default) method. In contrast, the Sliding Window method significantly increases the ASR prediction time.

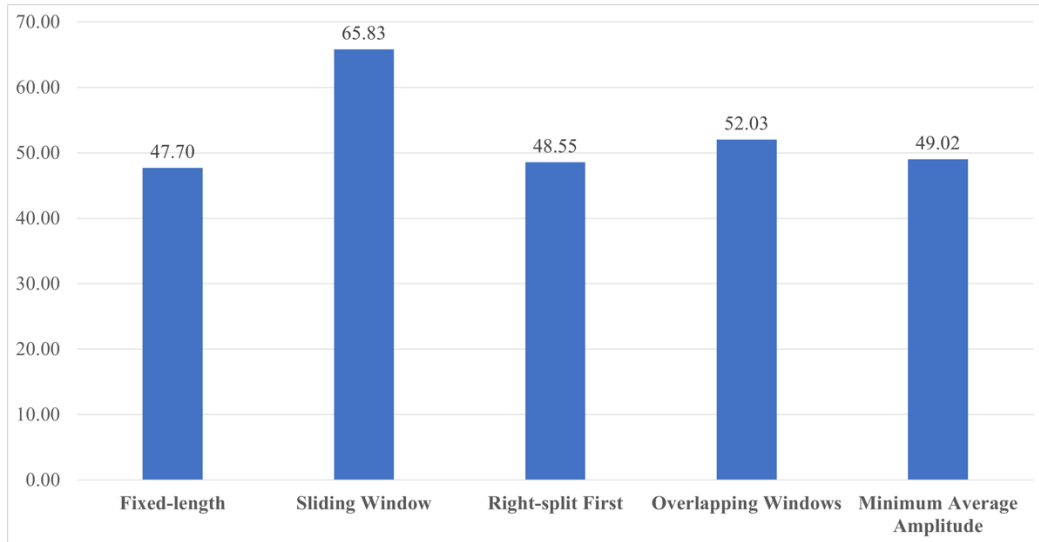


Fig. 12 Average runtime (in seconds) per a 30-minute audio clip of Whisper

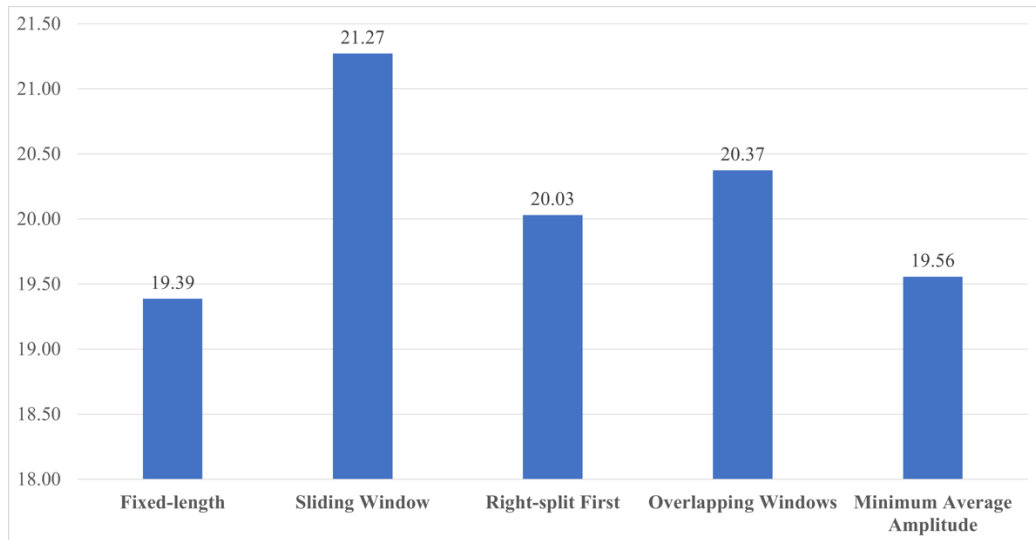


Fig. 13 Average runtime (in seconds) per a 30-minute audio clip of Wav2Vec2

To visualize the results of speech segmentation from various techniques, we present three comparison samples of text produced by Whisper using five speech segmentation methods in Figures 14-16. The first line of each figure is the reference subtitle obtained from the original audio clip, and the red text in each method indicates incorrect word predictions.

The Fixed-length and Sliding Window methods predict a relatively high number of incorrect words across all samples. Additionally, the Overlapping Windows method results in the repetition of the phrase "I am out," as shown by the underlined text in Figure 15. Although the Minimum Average Amplitude method predicts one incorrect word, "mesmerized," in Figure 15, it is considered to have fewer prediction errors compared to the other methods.

Reference:

who wants to be in the hot seat first? I kinda would. Great. That is perfect for us.

Fixed-length:

who wants to be in the hot seat first I **got wood** great that is perfect for us

Sliding Window:

who wants to be in the **world in the** hot seat first I kind of would great that is perfect for us

Right-split First:

who wants to be in the hot seat first I kind of would great that is perfect for us

Overlapping Windows:

who wants to be in the hot seat first I **got** would great that is perfect for us

Minimum Average Amplitude:

who wants to be in the hot seat first I kind of would great that is perfect for us

Fig. 14: Visualized comparison of sample 1

Reference:

where they get caught up all the time. I am a red. I am out. It is mesmerizing. You are out?

Fixed-length :

where they get caught up all the time **abg** I am out It is **mesmerized** you are out

Sliding Window:

where they get caught up all the time I am red **oh** I am out **smesmerize** you are out

Right-split First:

where they get caught up all the time **I am out** It is **mesmerized** you are out

Overlapping Windows:

where they get caught up all the time I am red I am out I am out It is **mesmerized** you are out

Minimum Average Amplitude:

where they get caught up all the time I am a red I am out It is **mesmerized** you are out

Fig. 15: Visualized comparison of sample 2

Reference:

this right here is going to be the base of our TV and it is going to be heavy as all frick

Fixed-length :

this right here is going to be the base of our tv **going** to be heavy as all **fric**

Sliding Window:

this right here is going to be the base of our tv **it** is going to be heavy as all **fric**

Right-split First:

this right here **going** to be the base of our tv and it is going to be heavy as all frick

Overlapping Windows:

this right here is going to be the base of our tv and it is **go** to be heavy as all frick

Minimum Average Amplitude:

this right here is going to be the base of our tv and it is going to be heavy as all frick

Fig. 16: Visualized comparison of sample 3

Conclusion

This research introduces four speech segmentation techniques designed to enhance the performance of Automatic Speech Recognition (ASR) systems. To evaluate the effectiveness of these methods, we conducted a series of experiments using 10 audio clips, each with a duration of approximately 30 minutes, accompanied by their corresponding English subtitles. The primary objective was to assess the impact of these segmentation techniques on the word error rate (WER) and character error rate (CER) of two prominent ASR systems, Whisper and Wav2Vec2. The experimental results demonstrate that the majority of the proposed methods significantly reduce both WER and CER, thereby improving the accuracy of the ASR systems. Importantly, these improvements were achieved without a substantial increase in the total runtime, which includes both preprocessing and ASR prediction times. This balance between accuracy and efficiency is crucial for practical applications of ASR technology. Among the proposed methods, we particularly recommend the Minimum Average Amplitude method for speech segmentation preprocessing. This method has proven highly effective in reducing WER and CER, making it a valuable tool for enhancing ASR performance. Furthermore, the Minimum Average Amplitude method has a minimal impact on the total runtime of the ASR process, ensuring that the system remains efficient and responsive. The findings of this research highlight the potential of advanced speech segmentation techniques to significantly improve the accuracy of ASR systems while maintaining operational efficiency. Future work will explore the application of these methods to a broader range of audio datasets and ASR systems to further validate their effectiveness and versatility.

References

- [1] Aslin, R. N., Woodward, J. Z., LaMendola, N. P., & Bever, T. G. (1996). Models of word segmentation in fluent maternal speech to infants. In J. L. Morgan & K. Demuth (Eds.), *Signal to syntax: Bootstrapping from speech to grammar in early acquisition* (pp. 117–134). Lawrence Erlbaum Associates, Inc.

- [2] Behre, P., Parihar, N., Tan, S., Shah, A., Sharma, E., Liu, G., Chang, S., Khalil, H., Basoglu, C., & Pathak, S.D. (2022). Smart Speech Segmentation using Acousto-Linguistic Features with look-ahead. ArXiv, abs/2210.14446.
- [3] Bhati, S., Villalba, J., Želasko, P., Moro-Velazquez, L., & Dehak, N. (2022). "Unsupervised Speech Segmentation and Variable Rate Representation Learning Using Segmental Contrastive Predictive Coding," in *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 30, pp. 2002-2014, 2022, doi: 10.1109/TASLP.2022.3180684.
- [4] Dal Ben, R., Prequero, I. T., de Hollanda Souza, D., & Hay, J. (2022, August 1). Speech segmentation and cross-situational word learning in parallel. <https://doi.org/10.31234/osf.io/pf4nr>
- [5] Kamper, H., & Niekerk, B.V. (2024). Revisiting speech segmentation and lexicon learning with better features. ArXiv, abs/2401.17902.
- [6] Mittmann, M., & Barbosa, P. (2022). An automatic speech segmentation tool based on multiple acoustic parameters. *CHIMERA: Revista de Corpus de Lenguas Romances y Estudios Lingüísticos*. 3. 133-147. 10.15366/chimera2016.3.2.003.
- [7] Namjoshi, J., Tremblay, A., Broersma, M., Kim, S., & Cho, T. (2012). Use of Prosodic Cues in Speech Segmentation: The Effect of Recent Linguistic Exposure.
- [8] Palmer, S. D., Hutson, J., & Mattys, S. L. (2018). Statistical learning for speech segmentation: Age-related changes and underlying mechanisms. *Psychology and aging*, 33(7), 1035–1044. <https://doi.org/10.1037/pag0000292>
- [9] Tremblay, A., Cho, T., Kim, S., & Shin, S. (2019). Phonetic and phonological effects of tonal information in the segmentation of Korean speech: An artificial-language segmentation study. *Applied Psycholinguistics*, 40(5), 1221–1240. doi:10.1017/S0142716419000237
- [10] Venkatesh, S., Moffat, D., Kirke, A., Shakeri, G., Brewster, S.A., Fachner, J., Odell-Miller, H., Street, A.J., Farina, N., Banerjee, S., & Miranda, E.R. (2021). Artificially Synthesising Data for Audio Classification and Segmentation to Improve Speech and Music Detection in Radio Broadcast. *ICASSP 2021 - 2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 636-640.
- [11] White, L., Mattys, S. L., & Wiget, L. (2012). Segmentation cues in conversational speech: robust semantics and fragile phonotactics. *Frontiers in Psychology*, 3, 375. <https://doi.org/10.3389/fpsyg.2012.00375>
- [12] White, L., “Segmentation of Speech”, in Shirley-Ann Rueschemeyer, and M. Gareth Gaskell (eds), *The Oxford Handbook of Psycholinguistics*, 2nd edn, Oxford Library of Psychology (2018; online edn, Oxford Academic, 10 Sept. 2018), <https://doi.org/10.1093/oxfordhb/9780198786825.013.1>
- [13] Yurovsky, D., Yu, C., & Smith, L. B. (2012). Statistical speech segmentation and word learning in parallel: scaffolding from child-directed speech. *Frontiers in psychology*, 3, 374. <https://doi.org/10.3389/fpsyg.2012.00374>
- [14] Radford, A., Kim, J. W. , Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. “Robust Speech

Recognition via Large-Scale Weak Supervision” Available from: <https://arxiv.org/pdf/2212.04356.pdf>

[15] Schneider, S., Baevski, A., Collobert, R., & Auli, M. (2019). wav2vec: Unsupervised pre-training for speech recognition. arXiv preprint arXiv:1904.05862.

[16] Graf, S., Herbig, T., Buck, M., & Schmidt, G. (2015). Features for voice activity detection: a comparative analysis. EURASIP Journal on Advances in Signal Processing, 2015(91).

[17] Ball, J. (2023). Voice Activity Detection (VAD) in Noisy Environments. arXiv. <https://doi.org/10.48550/arXiv.2312.05815>

[18] Levenshtein, V. I. (1966). Binary codes capable of correcting deletions, insertions, and reversals. Soviet Physics Doklady, 10(8), 707-710.

[19] Klakow, D., & Peters, J. (2002). Testing the correlation of word error rate and perplexity. Speech Communication, 38(1–2), 19–28.

[20] Baevski, A., Zhou, H., Mohamed, A., & Auli, M. (2020). wav2vec 2.0: A framework for self-supervised learning of speech representations. arXiv. <https://doi.org/10.48550/arXiv.2006.11477>